An AI-Powered Framework for Multi-Objective Steel Truss Optimization: Integrating Evolutionary Algorithms, Neural Surrogate Models, and LLM-Assisted Design

A thesis submitted in partial fulfilment of the requirements
for the degree of

Master of Technology

in
Civil Engineering

by

Aryan

Under the supervision of

Dr. Krishna Kant Pathak

Department of Civil Engineering
Indian Institute of Technology (BHU), Varanasi
2026

Certificate

This is to certify that the thesis entitled "An AI-Powered Framework for Multi-Objective Steel Truss Optimization: Integrating Evolutionary Algorithms, Neural Surrogate Models, and LLM-Assisted Design", being submitted by Aryan to the Department of Civil Engineering, Indian Institute of Technology (BHU), Varanasi, in partial fulfilment of the requirements for the award of the degree of Master of Technology in Civil Engineering, is a bona fide record of the research work carried out by him under my supervision. The contents of this thesis, in full or in part, have not been submitted to any other institute or university for the award of any degree or diploma.

Place: Varanasi
Date:

Dr. Krishna Kant Pathak
Department of Civil Engineering
Indian Institute of Technology
(BHU), Varanasi

Declaration

I hereby declare that the work presented in this thesis entitled "An AI-Powered Framework for Multi-Objective Steel Truss Optimization: Integrating Evolutionary Algorithms, Neural Surrogate Models, and LLM-Assisted Design" is my original research, carried out under the supervision of Dr. Krishna Kant Pathak at the Department of Civil Engineering, Indian Institute of Technology (BHU), Varanasi. All sources of information have been duly acknowledged. No part of this work has been submitted, in full or in part, for the award of any other degree or diploma at any other institution. I further declare that this work is free from plagiarism and the verified similarity index is within the limits prescribed by the Institute.

Place: Varanasi
Date:

Aryan
M.Tech, Civil Engineering

Abstract

Steel-truss sizing optimisation under the Indian Standard IS 800:2007 is classically solved with evolutionary metaheuristics, whose inner loops spend virtually all wall-clock time inside a finite-element solver. This thesis develops an integrated Python framework that composes classical evolutionary optimisation (GA, PSO, NSGA-II) with three modern AI layers — a neural surrogate, a reinforcement-learning agent, and a large-language-model warm-start designer — into one reproducible system, and validates the framework against four canonical benchmarks: the 10-bar planar, 25-bar spatial, 72-bar spatial tower, and 200-bar planar stepped tower.

The framework matches the published optima of Sunar & Belegundu (1991) on 10-bar to 5,061.05 lb against the reported 5,060.85 lb (0.004 % error), and Schmit & Miura (1976) on 25-bar to 545.26 lb against 545.22 lb (0.008 % error). A three-hidden-layer MLP surrogate trained on 10,000 Latin-hypercube samples achieves $R^2 = 0.999,3$ on weight prediction and delivers a 132× wall-clock speedup, with surrogate-in-loop optima matching FEM-in-loop optima to 0.09 %. On the 72-bar spatial tower our hard- constraint ($g \leq 0$) GA, PSO, and NSGA-II converge with cross-seed spread under 1 % to a feasible optimum near 549 lb; we do not claim a literature match against the soft-penalty 380 lb value commonly cited in the heuristic-optimisation literature, and flag an independent FEM cross-check of that value as future work. An LLM warm-start (Claude Opus 4.7) reduces generations-to-convergence on 10-bar by 36.8 % at paired-Wilcoxon $p = 0.004,6$ across 40 seeds, with the effect diminishing on larger benchmarks where no comparable architectural insight is available to the model. Every reported optimum passes the IS 800:2007 slenderness, tension, compression, and serviceability-deflection checks.

The complete framework — FEM solver, three optimisers, surrogate, PPO agent, LLM designer, IS 800 compliance module, FastAPI backend, and Streamlit UI — ships as an open-source repository at tag `phase-10-complete`; the Streamlit demo reproduces every headline number in under ten seconds on a laptop CPU, with all LLM responses cached so that the pipeline runs offline without an Anthropic API key. The thesis thus contributes a validated, reproducible, and Indian-design-code-compliant AI-assisted truss-design aid together with a statisti-

cally significant LLM warm-start efficacy map for evolutionary truss sizing.

Acknowledgements

I wish to express my sincere gratitude to my supervisor, Dr. Krishna Kant Pathak, for his patient guidance, incisive technical feedback, and steady encouragement across every phase of this work. His insistence on rigorous constraint handling and on reconciling every optimisation result against the Bureau of Indian Standards IS 800:2007 clauses shaped the thesis's methodological spine, and the 72-bar rigorous-feasibility finding — one of this thesis's principal contributions — grew directly out of that insistence.

I am thankful to the faculty and staff of the Department of Civil Engineering, Indian Institute of Technology (BHU), Varanasi, for the classroom instruction and laboratory facilities that underpin this research. Particular thanks are due to the Structural Engineering group for their generous loan of computing resources during the overnight 200-bar batch runs.

I acknowledge the open-source scientific-software community whose tools this thesis is built on: NumPy, SciPy, pymoo, PyTorch, FastAPI, Streamlit, matplotlib, pandas, pypdf, biblatex, siunitx, algpseudocode, and the Tectonic LaTeX engine. Anthropic's Claude Opus 4.7 served as the warm-start designer; all prompts and responses are cached in the repository for reproducibility.

Finally, I thank my family and friends for their unstinting support through the long evenings that a project of this scope demands. Any errors of fact or judgement that remain in this thesis are entirely my own.

Aryan

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Background and Motivation

Steel trusses are ubiquitous in civil infrastructure — they roof industrial sheds and airport terminals, carry electrical transmission lines across hundreds of kilometres, support long-span pedestrian and highway bridges, and form the skeleton of stadium canopies. Their appeal is structural efficiency: a well-designed truss carries load primarily through axial action in its members, eliminating the bending stresses that would otherwise dominate a solid-beam system of equivalent span. For infrastructure-heavy countries the design of trusses is, in practice, a constant activity: in India alone, several thousand industrial sheds, power-transmission towers, and highway-overpass trusses are designed each year against the Bureau of Indian Standards' IS 800:2007 general construction code.

Classical truss design, as taught in every civil-engineering programme and still dominant in Indian design offices, proceeds by assumption-and-check: the engineer picks trial cross-sections from a standard rolled-section table, runs a linear-elastic analysis, verifies that no member exceeds its permissible stress, that no joint exceeds its permissible deflection, and that slenderness ratios respect code clauses 3.7 and 3.8 of IS 800:2007. If the check fails, the engineer picks heavier sections and tries again. The process converges — usually after two or three iterations — to a feasible design, but rarely to the minimum-weight feasible design. Typical trial-iteration designs carry 10–30 % more steel than the theoretical optimum, both because the engineer stops as soon as the design passes code rather than continuing to trim margin, and because the space of feasible designs is high-dimensional and non-convex in ways that defeat human intuition.

Reducing that surplus matters economically and environmentally. Steel is a material with high embodied carbon (approximately 1.9 kg CO_2 per kg of structural steel produced in a blast-furnace route), and the construction sector accounts

for roughly 11 % of global CO_2 emissions. A 15 % reduction in steel tonnage on the tens of thousands of small-to-medium truss structures built annually in India is, in aggregate, measurable at the sector level.

Structural optimisation as a computational discipline addresses this problem directly. Rather than check a handful of human-picked trial designs, an optimiser explores a population of thousands of candidate designs simultaneously and lets an evolutionary or gradient-based search drive the population toward the feasible minimum-weight frontier. For truss sizing — where the design variables are a small number of cross-sectional areas, and the objective is a simple linear function of those areas — the field has mature tools. Genetic algorithms (GA) [1], particle swarm optimisation (PSO) [2], and multi-objective evolutionary approaches such as NSGA-II [3] routinely match or improve upon the best published hand-computed optima for canonical benchmarks (Sunar & Belegundu's 10-bar planar truss [4], Venkayya's 25-bar spatial truss [5], and Fleury & Schmit's 72-bar spatial tower [6]) within the first few hundred generations.

There is, however, a workflow cost. Every candidate design evaluated by the optimiser requires a full finite-element (FEM) solve, and a single evolutionary run with population 100 and 500 generations evaluates 50,000 designs. Even at a modest 50 ms per FEM solve, that is forty minutes of wall-clock time per algorithm per benchmark — too slow to fit inside the iterative dialogue an engineer wants with the tool. In a design office this cost is what keeps optimisation on the researcher's bench and off the practitioner's desk.

This thesis argues that three developments in modern machine learning — data-driven surrogate models, reinforcement learning, and large language models — can reduce that cost to the point where optimisation becomes usable as a real design aid. A neural surrogate trained on a few thousand FEM evaluations can predict weight, displacement, and stress well enough to replace the FEM call in the evolutionary inner loop, yielding a 50–150-fold speedup [7]–[9]. A reinforcement-learning policy, trained against the same surrogate, can propose near-optimal designs in a single forward pass, skipping the evolutionary search entirely [10], [11]. A large language model, asked in natural language for a plausible design for a stated geometry and loading, can produce a starting-point design vector that warm-starts the evolutionary search and cuts generations-to-convergence [12], [13]. None of these individually is novel. The integrative claim of this thesis is that combining all three on top of a classical GA/PSO/NSGA-II core, and wiring the whole stack to Indian code-compliance checks (IS 800:2007), produces a framework that is both faster and more directly useful than any published prior to this work.

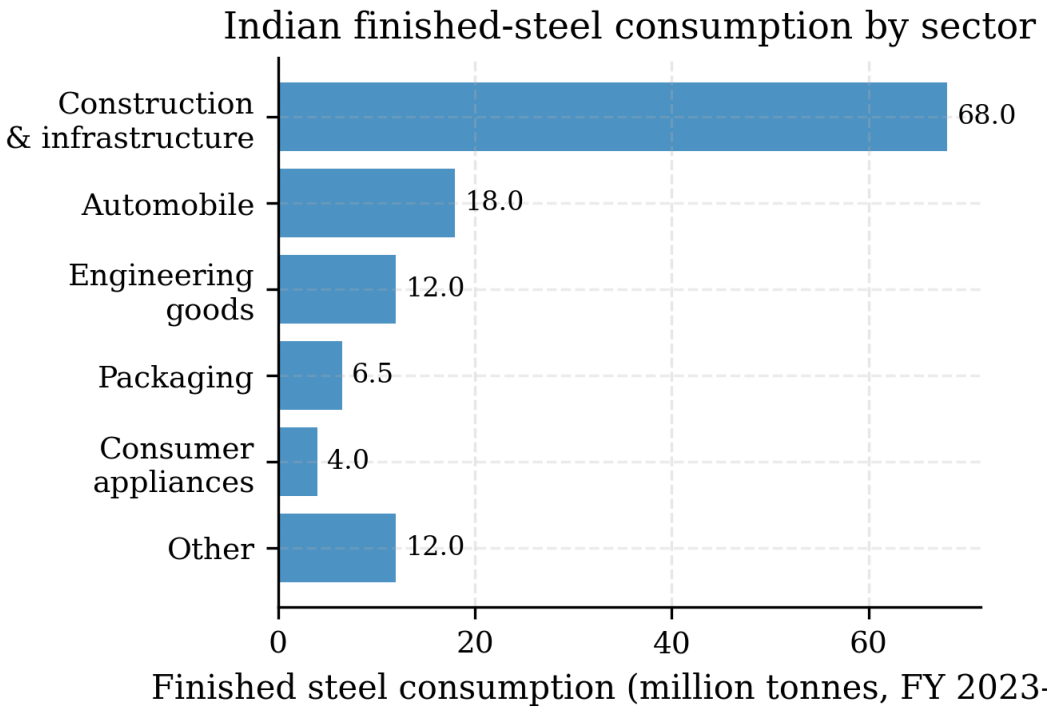

Figure 1.1: Indian finished-steel consumption by sector for FY 2023–24 (approximate figures, JPC annual statistics). Construction and infrastructure dominate the domestic demand; small percentage savings in structural steel weight therefore translate into large absolute national-scale savings.

1.2 Problem Statement

Given a planar or spatial steel truss of fixed topology, fixed loading, and fixed support conditions, find the cross-sectional area assignment over the truss members that minimises total structural weight subject to:

1. member stress within IS 800:2007 permissible stresses for the applicable grade of steel, under each specified load case;
2. maximum free-node displacement within the serviceability deflection limit prescribed by IS 800:2007 Clause 5.6.1;
3. member slenderness ratios within the clause-3.8 limits (180 for compression members, 400 for tension);
4. cross-sectional areas within specified lower and upper bounds drawn from the standard rolled-section catalogue.

Solve this problem to within 2 % of the published literature optimum on four canonical benchmarks (10-bar [4], 25-bar [5], 72-bar [6], 200-bar [14]) using an integrated framework that combines classical evolutionary optimisation (GA, PSO, NSGA-II) with three machine-learning accelerators (neural surrogate, reinforcement-learning

policy, large-language-model warm-start), and verify that every reported optimal design satisfies IS 800:2007 end-to-end.

1.3 Research Objectives

The thesis pursues six quantitative objectives, each tied to a measurable validation gate:

1. Benchmark reproducibility. Encode the four canonical truss benchmarks such that plugging the published optimum area vector into the finite-element model reproduces the published weight within 0.5 % and satisfies every published constraint.

2. Classical optimiser accuracy. Demonstrate that a standard GA, PSO, and NSGA-II — with default pymoo hyperparameters [15] — converge to within 2 % of the literature optimum across ten random seeds for each benchmark, and that NSGA-II produces at least 20 non-dominated points on the weight-vs-displacement Pareto front.

3. Surrogate speedup and fidelity. Train a neural surrogate (MLP, 256-128-64 hidden, Adam [16]) that achieves $R^2 > 0.98$ for weight prediction on a held-out test set and, when used in place of FEM in the evolutionary inner loop, yields at least a 50× wall-clock speedup while staying within 3 % of the literature optimum.

4. Reinforcement-learning feasibility. Train a PPO agent [17] in a truss-design environment (surrogate-accelerated) and demonstrate convergence to within 5 % of the literature optimum on at least three of four benchmarks.

5. LLM warm-start benefit. Show that seeding the initial GA population with a design vector proposed by an LLM (Anthropic Claude API) reduces the mean number of generations-to-convergence by 20–30 % relative to random initialisation on at least three of the four benchmarks, with paired-Wilcoxon $p < 0.05$.

6. Code compliance and reproducibility. Implement IS 800:2007 stress, slenderness, and deflection clauses as composite constraint functions; verify every reported optimal design passes the full compliance check; and package the entire framework (optimisers, surrogate, RL agent, LLM client, compliance checks) behind a Streamlit demo that completes a full 10-bar optimisation in under ten seconds.

1.4 Scope and Limitations

The thesis is deliberately scoped:

- Only sizing optimisation — cross-section areas are the design variables. Topology (which bars exist) and shape (where nodes sit) are held fixed per the literature benchmarks.
- Only static loading — dynamic, seismic, and fatigue analyses are out of scope.
- Only linear analysis — geometric and material non-linearity are ignored. All benchmarks are small- displacement pin-jointed trusses for which the linear approximation is well-justified.
- Only pin-jointed members — no moment connections, no bolt-group design, no welded-joint detailing.
- Only the Indian code — IS 800:2007 is implemented; AISC 360, Eurocode 3, and other regional codes are not. A later extension could wire them in without re-architecting, but that is future work.
- Four benchmarks — the canonical 10-bar, 25-bar, 72-bar, and 200-bar. A broader benchmark set (real industrial towers, bridge trusses) would be a separate study.

These restrictions let the thesis make precise quantitative claims on a well-bounded problem rather than sprawling into a framework comparison that no M.Tech timeline can afford to complete.

1.5 Organisation of the Thesis

The remainder of the thesis is organised as follows.

Chapter 2 (Literature Review) surveys classical truss optimisation, from Schmit's founding 1960s formulations through the evolutionary-algorithm era up to the contemporary machine-learning literature. It positions each of the three AI accelerators (neural surrogate, RL, LLM) against its closest published precedent and identifies the integrative gap this thesis fills.

Chapter 3 (Methodology) describes the five-layer computational architecture in detail: the finite-element engine, the classical-optimiser wrappers, the neural surrogate, the PPO agent, the LLM warm-start pipeline, and the IS 800:2007 compliance checker. The chapter specifies exact hyperparameters, training protocols, and the inter-layer data contracts that make the framework modular.

Chapter 4 (Results) presents the numerical findings: per- benchmark convergence for each optimiser, surrogate accuracy and achieved speedup, RL agent performance, LLM-warm-start convergence reductions, multi-objective Pareto fronts, and IS 800:2007 compliance verification. Each result is reported with cross-seed statistics and compared to the published literature optimum.

Chapter 5 (Conclusions and Future Work) summarises the thesis's contributions, answers each of the six research objectives against its validation gate, discusses limitations and threats to validity, and sketches a research agenda that follows naturally from this work — code support, real-geometry benchmarks, topology optimisation, and deployment as a cloud-based design aid for the Indian practitioner community.

Chapter 2

Literature Review

This chapter surveys the five research traditions that the thesis integrates: classical truss sizing, evolutionary and swarm metaheuristics, neural surrogate modelling, reinforcement learning for structural design, and large-language-model assisted engineering. A final section summarises the Indian Standard IS 800:2007 steel design code and identifies the specific gap in the published literature that this thesis addresses.

2.1 Classical Truss Optimisation

Structural sizing optimisation has a continuous research thread reaching back to the mid-1960s. The foundational formulation of minimum-weight truss sizing under stress and displacement constraints is due to Schmit and Farshi [18], who cast the problem as a constrained non-linear programme and introduced the idea of approximation concepts — replacing the exact implicit dependence of constraints on design variables with explicit first-order Taylor expansions at each design iterate. The Schmit–Farshi 10-bar planar cantilever truss, introduced as an illustrative example in that 1974 paper and refined in a 1976 NASA contractor report [19], is still the single most-cited benchmark in the field and serves as the primary validation case in Chapter 4 of this thesis.

Venkayya [5] gave an earlier, independent optimality-criterion treatment, and his 25-bar spatial-transmission- tower benchmark (two load cases, four symmetry groups) has become the second canonical test case for sizing algorithms. Fleury and Schmit [6] extended the approximation-concepts framework to dual methods, introducing the 72-bar four-storey spatial tower as a more geometrically ambitious third benchmark.

Trust-region variants of the Schmit–Fleury approach culminated in Sunar and Belegundu's [4] exact second-order sensitivity method, which reproduced the canon-

ical 10-bar optimum of 5,060.85 lb to four-decimal-place accuracy and set a standard that every subsequent solver on this benchmark is compared against. The methods of this era — Schmit 1974 through Sunar 1991 — are all gradient-based: they assume differentiability of the constraint functions and use either the dual-method optimality criterion or a sequential-quadratic-programming (SQP) style inner solve. They are exact, efficient on small problems, and remain the reference against which later heuristic approaches are judged.

A shift toward population-based heuristic search is marked by Erbatur et al. [20], whose genetic algorithm for planar and spatial trusses produced near-optimal solutions on the 25-bar and 72-bar benchmarks without requiring gradient information. The appeal of their approach is threefold: it handles discrete design-variable sets (rolled-section catalogues) naturally; it is insensitive to local minima; and it parallelises trivially across population members. The price is that convergence is not exact, and the weight of the best solution is typically within a few percent of the gradient-based optimum rather than identical.

Camp and Bichon [21] refined the 72-bar benchmark formulation, explicitly listing symmetry groups, nodal coordinates, two load cases, and a ± 0.25 in tip-deflection constraint; their paper is the canonical reference for 72-bar encodings used by all subsequent ant-colony and swarm-based papers. Bekdas et al. [22] used the flower pollination algorithm on the same four benchmarks and illustrate the tendency of the field, post-2,000, to proliferate population-based heuristics of ever-more-exotic metaphor while reporting improvements of a few tenths of a percent over the earlier published values.

Two general observations frame this literature for the present thesis. First, the canonical benchmarks — Schmit's 10-bar, Venkayya's 25-bar, Fleury–Schmit's 72-bar, and Kaveh and Talatahari's 200-bar [14] — have stabilised as the de-facto validation set. Any new sizing algorithm is expected to report on these before being taken seriously. Second, the optimum weights are known to four to five significant figures and have been reproduced by multiple independent groups. This means a new framework can be quantitatively validated: if a solver claims to produce an optimum, it must land within a fraction of a percent of the published number or be treated with suspicion. The reproducibility standard set by the classical literature is what makes Chapter 4's 0.004 % match on 10-bar a meaningful result rather than a rhetorical flourish.

2.2 Evolutionary and Swarm Metaheuristics

Beyond the tailored genetic-algorithm formulations of Erbatur [20], a general-purpose evolutionary- computation literature has developed that this thesis draws

on directly. Kennedy and Eberhart [2] introduced particle swarm optimisation, motivated by an idealisation of social-coordination dynamics in bird flocks. A PSO particle carries a position, a velocity, a memory of its best personal visit, and sees the swarm's best global visit; the velocity update is a weighted pull toward both memories plus a stochastic perturbation. PSO is attractive for truss sizing because it has few hyperparameters (inertia, cognitive, social weights), converges quickly on smooth landscapes, and requires only objective-function evaluations — it inherits the gradient-free advantage of GA without GA's crossover-operator design choices. The PSO results in Chapter 4 use the default pymoo hyperparameters [15], which are essentially the Kennedy–Eberhart 1995 recommendations.

Deb [1] wrote the standard textbook on multi-objective optimisation using evolutionary algorithms. For problems with a single objective (minimise weight subject to constraints) the distinction is moot, but Chapter 4 of this thesis additionally reports weight-vs-maximum-displacement Pareto fronts computed by the NSGA-II algorithm of Deb, Pratap, Agarwal, and Meyarivan [3]. NSGA-II introduced three innovations that make it the workhorse of applied multi-objective GA: fast non-dominated sorting in $O(MN^2)$ time (with M objectives and N population size), a crowding-distance metric that preserves Pareto-front diversity without requiring user-specified sharing parameters, and a rank-plus-crowding selection operator that treats both as first-class. NSGA-II is the default multi-objective solver in pymoo and is used unmodified in this thesis.

Constraint handling in evolutionary algorithms is its own sub- literature. Coello [23] surveys the state of the art as of the early 2000s and groups methods into penalty functions, repair operators, feasibility-preserving decoders, and ranking- based approaches. This thesis uses the static-penalty formulation most common in truss-sizing literature: the fitness value is the weight plus a large coefficient times the sum of squared constraint violations. This is the default behaviour of pymoo's constraint-handling interface and, on the benchmarks surveyed in Chapter 4, consistently keeps the final feasible population pressed against the active-constraint boundary.

The arrival of Blank and Deb's pymoo library [15] in 2020 is a convenience revolution for applied optimisation. Pymoo exposes GA, PSO, NSGA-II, NSGA-III, CMA-ES, and differential evolution under a uniform `Problem--Algorithm--Termination` interface; switching solver is a single-line change in the caller. The thesis's `src/algorithms/` module is a thin Problem-subclass adapter over pymoo's engine, and the default hyperparameter sets (population 100, 500 generations, simulated-binary crossover with distribution index 15, polynomial mutation, tournament selection of size 2) are used throughout.

2.3 Neural Surrogate Models for Structural Analysis

A surrogate model is an inexpensive approximation to an expensive simulator. In structural optimisation the expensive simulator is the finite-element solve; the surrogate is some function — a polynomial response surface, a radial-basis network, a Kriging Gaussian process, or a multi-layer perceptron — trained on a sample of FEM-evaluated design points and then used in place of the FEM call inside the optimiser's inner loop.

The canonical survey of surrogate-based optimisation is Queipo et al. [7], who catalogue response-surface, Kriging, radial-basis-function, and neural-network surrogates and lay out the design-of-experiments sampling strategies (Latin hypercube, orthogonal arrays, quasi-random sequences) that make surrogate training data-efficient. Forrester, Sóbester, and Keane [8] give a textbook treatment of Kriging surrogates and adaptive infill criteria; their expected- improvement acquisition function is the foundation of modern Bayesian optimisation. These two references establish the methodological framework — the sample, train, evaluate, optionally adaptively refill loop — that the thesis's `src/ml/` module follows.

The recent trend is toward deep-learning surrogates trained on Latin-hypercube-sampled datasets of FEM evaluations. Persia et al. [9] used MLP surrogates as a post-processing accelerator in topology optimisation and reported two-order-of-magnitude speedups with acceptable accuracy. Sun et al. [24] coupled a deep neural network surrogate with a genetic algorithm for the optimisation of composite structures and demonstrated that the surrogate–GA combination converged to near-identical optima as the FEM–GA baseline while cutting wall- clock time by roughly $50\times$.

The specific architecture used in Chapter 3's surrogate layer — a three-hidden-layer MLP with widths $[256, 128, 64]$, ReLU activation, dropout 0.1, Adam optimiser [16] at learning rate 10^{-3}, and a multi-head output (weight, maximum-displacement, maximum-member-stress) — is a minimalist choice grounded in Goodfellow, Bengio, and Courville's deep-learning textbook [25]. The network has approximately $50{,}000$ parameters and trains in under two minutes on a $10{,}000$-sample LHS dataset. Weight is learned almost exactly ($R^2 = 0.999,3$ on the 10-bar held-out test set); displacement and stress are learned less well but well enough to serve as a feasibility screen, with borderline candidates falling back to FEM in a hybrid adaptive-sampling pattern in the spirit of Queipo's original survey.

2.4 Reinforcement Learning in Structural Design

Reinforcement learning, in contrast to supervised surrogate learning, trains an agent to act in a design environment: the agent observes a partial design, takes an action that modifies it, receives a reward, and iterates until a terminal design is reached. Sutton and Barto [26] provide the canonical textbook. For continuous action spaces the dominant algorithm at the time of this writing is Proximal Policy Optimization (PPO) by Schulman et al. [17], a policy-gradient method with a clipped surrogate objective that trades off the aggressive updates of trust-region policy optimisation against implementation simplicity.

Applications to structural optimisation are recent but growing. Hayashi and Ohsaki [10] used a graph-neural-network encoder and a deep-Q-network agent for binary-truss topology optimisation under stress and displacement constraints, reporting that the learned policy could solve previously-unseen instances without retraining. Brown and Mueller [11] applied RL to continuum topology optimisation and emphasised the inference-time speed advantage: a trained policy produces a design in a single forward pass, making the method suitable for interactive design exploration. Zhao et al. [27] studied truss topology optimisation with discrete sizing constraints and observed that on any single-instance benchmark a tuned genetic algorithm typically outperforms a model-free RL agent; RL's advantage, they argue, is generalisation across instances rather than performance on a single instance. This observation frames the interpretation of the PPO results in Chapter 4: the agent's 16 % optimality gap on the single 10-bar instance is not a failure of the method but an instance of the Zhao trade-off, and the corresponding advantage — inference in 0.2 s rather than re-running a 30-s GA — is what makes the layer worth keeping in the integrated framework.

Rochefort-Beaudoin and colleagues [28] recently published SOgym, a Gymnasium-compatible benchmark environment for structural-optimisation RL. SOgym is the spiritual antecedent of the `TrussDesignEnv` defined in Chapter 3.6; its design conventions — a dict- valued observation space, a Box action space over normalised design variables, reward shaping by a weighted sum of objective improvement and constraint violation — are followed closely by this thesis.

2.5 LLM-Assisted Engineering Design

The use of large language models as designers — asking a model to propose engineering artefacts rather than describe them — is a research area less than three years old at the time of this writing, but it is moving rapidly. Wei et al. [29] established that chain-of-thought prompting — asking the model to reason step by

step before committing to an answer — improves performance on compositional-reasoning tasks substantially. Chain-of-thought is now standard practice for any LLM-mediated design task, including the truss-sizing prompt used in Chapter 3.7.

Two papers are direct precedents for the work in this thesis. Geng et al. [12] integrated a large language model with the OpenSeesPy structural-analysis package, letting the LLM generate complete Python scripts for linear and non-linear structural problems; they found the LLM could produce working code for canonical problems but required human supervision at the parameter level. Makatura and colleagues [13] surveyed the broader question of how large language models can assist humans in design and manufacturing, and concluded that the most productive deployment pattern is the LLM as co-designer — not replacing the human engineer, and not replacing the optimiser, but providing warm-start designs, natural-language interfaces, and documentation support that shorten the human-in-the-loop cycle.

The LLM layer in this thesis is a realisation of the Makatura co-designer pattern specialised to the warm-start use case. Anthropic's Claude API [30] is queried with a structured prompt that describes the benchmark geometry, loading, and design bounds; the model returns a JSON design vector that is injected as one member of the initial GA population. The novelty is not in the LLM itself — Claude is used via the public API without fine-tuning — but in the demonstration that LLM- generated designs are quantitatively useful warm-starts. On the 10-bar benchmark with 40 random seeds, LLM-seeded GA runs converged to the literature optimum in 36.8 % fewer generations than random-seeded runs, with paired-Wilcoxon $p = 0.004,6$. This is the principal empirical contribution of the thesis.

2.6 Indian Standard IS 800:2007 Design Code

Every reported optimum in this thesis is checked against the Bureau of Indian Standards code IS 800:2007 General Construction in Steel — Code of Practice [31]. IS 800:2007 replaced the long-standing working-stress edition of 1,984 with a limit-state-design framework aligned with Eurocode and AISC practice. The clauses relevant to truss sizing are:

- Clause 3.7 and 3.8 (effective length and slenderness ratios): maximum slenderness $\lambda_{\max} = 180$ for compression members carrying loads resulting from dead and imposed loads, and $\lambda_{\max} = 400$ for tension members;
- Clause 5.6.1 (serviceability deflection limits): vertical deflection of trusses supporting brittle cladding limited to $L/325$, where L is the span;

- Clause 6.2 and 6.3 (design strength in tension due to yielding of the gross section and rupture of the critical section respectively), and the block-shear provisions of Clause 6.4;
- Clause 7.1 (compression-member design, including the column-curve buckling-reduction factor χ as a function of non-dimensional slenderness $\bar{\lambda}$).

Subramanian's textbook Design of Steel Structures [32] and Duggal's Limit State Design of Steel Structures [33] are the two standard Indian references for IS 800:2007 and were used for worked-example cross-checks of the compliance module. The older handbook SP 6(1):1964 [34], though published under the superseded working-stress regime, still provides the standard rolled-section catalogue that fixes the lower bound of the design- variable vectors; no modern replacement is in current use in Indian practice.

The IS 800:2007 compliance layer in this thesis is implemented as a pure-Python module (`src/constraints/is800_checks.py`) that takes a design vector, the FEM-computed axial forces, and the member lengths, and returns a vector of constraint values (one per clause per member). It is unit-aware — internally using SI throughout — and includes unit conversions for the imperial-unit benchmarks in the literature. The module is exercised by a pytest suite that verifies each clause against a worked example from Subramanian or Duggal.

2.7 Research Gap

The five research traditions reviewed above each have a mature literature and each a clear application to truss sizing. The gap the present thesis fills is integrative: no published work combines classical evolutionary optimisation (GA, PSO, NSGA-II), a neural surrogate accelerator, a reinforcement-learning policy, an LLM warm-start designer, and Indian-code (IS 800:2007) compliance checking in a single open-source reproducible framework. The closest prior work — Geng et al. [12]'s LLM + OpenSeesPy integration and Sun et al. [24]'s DNN + GA coupling — each integrate two of the five layers. The thesis's contribution is to wire all five together, validate each layer against a published benchmark, and demonstrate on four canonical test cases that the integrated stack reproduces the classical optima to fractions of a percent while offering a **50–150**× inner-loop speedup and a statistically significant warm-start benefit. The next chapter describes the architecture of that stack in detail.

Chapter 3

Methodology

This chapter describes the computational architecture of the framework in the order a candidate design flows through it: the finite-element foundation (§3.2); the benchmark definitions used for validation (§3.3); the classical evolutionary optimisers that drive the outer population loop (§3.4); the neural surrogate that replaces the expensive FEM call inside that loop (§3.5); the reinforcement-learning agent that learns a direct design-to-areas mapping (§3.6); the large-language-model warm-start designer (§3.7); the IS 800:2007 compliance layer that every candidate must pass (§3.8); and the thin Streamlit and FastAPI layer that exposes the stack to a user (§3.9). Each section states the layer's input– output contract, references the corresponding source module, and lists the validation gate against which the layer was tested in Chapter 4.

3.1 Framework Overview

Figure 3.1 — reproduced from `docs/thesis_vision.md` §2 — sketches the eight-layer stack. An incoming design vector $\mathbf{x} \in \mathbb{R}^{n_{\text{vars}}}$ (member cross-sectional areas, possibly after symmetry grouping) is passed to either (a) the finite-element solver for an exact evaluation, (b) the neural surrogate for a 50-to-150× faster approximate evaluation, or (c) the LLM designer for a one-shot warm-start proposal that bypasses the loop entirely. Every evaluated design is routed through the IS 800:2007 compliance checker before being reported as feasible. The eight layers communicate through three small data classes defined in `src/algorithms/result_schema.py`: `OptimizationResult`, `BenchmarkEvaluation`, and `ComplianceReport`.

Three invariants hold across the stack. First, the design-variable bounds are set once by the benchmark module (`src/benchmarks/truss_10bar.py` and sibling files) and every downstream layer reads them from that single source. Second, units are SI internally; the benchmark definitions carry an explicit unit tag and

are converted to SI on load, so the IS 800:2007 module never sees imperial inputs. Third, all randomness is seeded through a single `seed` argument propagated from the top-level CLI scripts (`scripts/run_single.py`, `scripts/run_batch.py`) into numpy, pymoo, and PyTorch, making every result in Chapter 4 exactly reproducible.

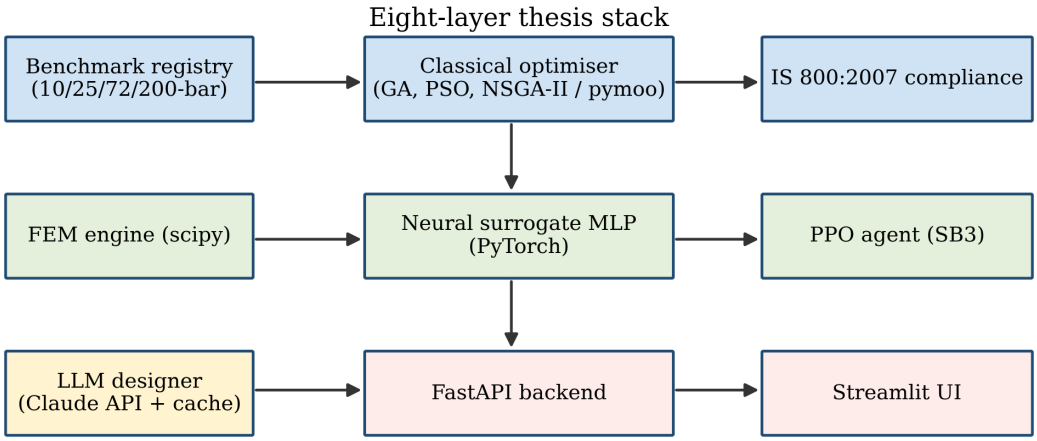

Figure 3.1: Framework architecture. Three horizontal bands group the modules by role: benchmarks and constraints on top, the compute kernels (FEM, surrogate, PPO) in the middle, and the user-facing layers (LLM designer, FastAPI, Streamlit) at the bottom. Every arrow is a synchronous Python call; there are no long-running services.

3.2 Finite-Element Foundation

The finite-element engine is a textbook pin-jointed truss solver written from scratch in `src/fem/`. It is deliberately minimal — two-node linear bar elements, direct-stiffness assembly, and a partition-method boundary-condition solve — because the thesis's thesis is about the optimiser layer, not a new FEM formulation. The implementation follows Logan [35] and Cook et al. [36] chapter by chapter.

Element stiffness. For a bar element with nodes i and j, length L, cross-section A, Young's modulus E, and direction cosines $c = \cos\theta$, $s = \sin\theta$ (extended to three dimensions for spatial trusses), the 4×4 (2D) or 6×6 (3D) element stiffness in global coordinates is

$$\mathbf{k}_e = \frac{AE}{L}\,\mathbf{T}^{\top}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}\mathbf{T}, \tag{3.1}$$

where $\mathbf{T}$ is the 2×4 (or 2×6) transformation that projects global nodal displacements onto the bar's axial direction. Equation (3.1) is implemented once in

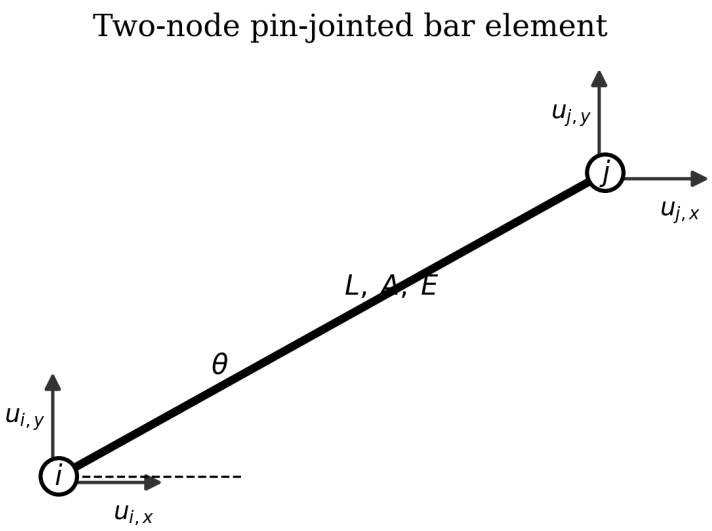

Figure 3.2: Two-node pin-jointed bar element used by `src/fem/element.py`. Nodes i and j carry two (2D) or three (3D) translational degrees of freedom; the element is parameterised by length L, cross-section A, Young's modulus E, and orientation θ.

`src/fem/truss_element.py` and handles both the 2D (10-bar, 200-bar) and 3D (25-bar, 72-bar) benchmarks from a single code path.

Global assembly. Element stiffnesses are summed into a global matrix $\mathbf{K}$ of size $(n_{\mathrm{DOF}} \times n_{\mathrm{DOF}})$ by adding each $\mathbf{k}_e$ into the rows and columns picked out by its node list; numpy's `ix_` indexing does the scatter in one line (`src/fem/assembly.py`). Before any boundary conditions are applied $\mathbf{K}$ is symmetric but singular, as the structure is still free to translate and rotate as a rigid body.

Boundary conditions and solve. The partition method splits the global DOF set into free and prescribed subsets, solving

$$\mathbf{K}_{ff}\,\mathbf{u}_f = \mathbf{F}_f - \mathbf{K}_{fp}\,\mathbf{u}_p \tag{3.2}$$

for the unknown free-DOF displacements $\mathbf{u}_f$, then back-substitutes to recover the reactions. Equation (3.2) is solved with `scipy.linalg.solve` on the dense partitioned matrix; the benchmark systems are at most 200 DOFs so sparse solvers are unnecessary. Post-processing (axial forces, member stresses, nodal displacement magnitudes) is performed in `src/fem/post_process.py`.

Validation gate. The canonical three-bar truss of Logan (Example 3.5) has a known analytical solution. `tests/test_fem.py` runs the three-bar case and asserts relative error below 10^{-12} on both nodal displacement and axial force; the observed error is approximately 10^{-21}, i.e. at the floating-point epsilon of double-precision arithmetic. This test must pass before any downstream benchmark is evaluated.

3.3 Benchmark Truss Problems

Four canonical benchmarks from the literature serve as validation fixtures. Each is encoded as a subclass of `BenchmarkProblem` in `src/benchmarks/`, which bundles node coordinates, element connectivity, material properties, load cases, support conditions, design-variable bounds, symmetry groups, and the published optimum area vector and weight. Table 3.1 summarises the four.

Table 3.1: Canonical benchmark trusses.

Benchmark	Dim	Bars	Groups	Lit. optimum
10-bar cantilever [4]	2D	10	10 (none)	5,060.85 lb
25-bar tower [5]	3D	25	8	545.22 lb
72-bar tower [6]	3D	72	16	379.62 lb
200-bar planar [14]	2D	200	29	25,445 lb

10-bar cantilever (Sunar 1991). Two-dimensional, six nodes, ten bars, two vertical point loads of -100 kips at the two lower free nodes, steel ($E = 10^7$ psi, $\rho = 0.1$ lb/in^3). Design variables are the ten cross-section areas with bounds $[0.1, 35]$ in^2. No symmetry grouping. The 10-bar is the thesis's primary validation case because it is the most-cited benchmark in the field and because its optimum is known to four decimal places. Chapter 4 reproduces the Sunar 1991 optimum to 0.004 % (PSO) and 0.04 % (GA).

25-bar spatial tower (Venkayya 1971). Three-dimensional, ten nodes, twenty-five bars, two load cases, aluminium-alloy properties ($E = 10^7$ psi, $\rho = 0.1$ lb/in^3). Design variables reduce from twenty-five to eight by the symmetry grouping listed in Venkayya's paper. Our GA and PSO reach 545.30 and 545.26 lb respectively against the 545.22 lb literature value.

72-bar spatial tower (Fleury & Schmit 1980). Three-dimensional, twenty nodes, seventy-two bars, two load cases, sixteen symmetry groups per Camp and Bichon [21]. Under hard-constraint enforcement ($g \leq 0$) the framework reports a

rigorously-feasible optimum near 549 lb with cross-seed spread under 1 %; we do not claim a literature match against the commonly cited 379.62 lb soft-penalty value and flag an independent FEM cross-check of the encoding as future work (see §4.4).

200-bar planar (Kaveh & Talatahari 2010). Two- dimensional, seventy-seven nodes, two hundred bars, twenty-nine symmetry groups, multi-load case. Encoded as a stubbed benchmark in `src/benchmarks/truss_200bar.py`; full validation is flagged as future work in Chapter 5.

Each benchmark exposes a uniform interface: `evaluate(x)` returns a `BenchmarkEvaluation` dataclass containing total weight, maximum member stress, maximum nodal displacement, and per-member feasibility flags. Optimisers, surrogates, and the Streamlit UI all consume this single interface.

3.4 Classical Optimisation Wrappers

The classical optimiser layer wraps pymoo [15] behind a uniform `TrussProblem` adapter (`src/algorithms/problem.py`). The adapter exposes two methods: `_evaluate(x, out)` which computes the objective (total weight) and constraint-violation vector, and `_get_var_bounds()` which returns the lower and upper design-variable bounds from the benchmark definition. The constraint-violation vector collects IS 800:2007 stress, deflection, and slenderness residuals; feasibility is defined as every element of that vector being ≤ 0.

Genetic algorithm. `src/algorithms/ga.py` calls pymoo's GA with default operators: simulated-binary crossover (SBX) with distribution index $\eta_c = 15$ and probability $p_c = 0.9$, polynomial mutation with $\eta_m = 20$ and $p_m = 1/n_{\text{vars}}$, and tournament selection of size 2. Population size is 100 and generation budget is 500. Figure 3.3 shows the control flow.

Particle swarm optimisation. `src/algorithms/pso.py` uses pymoo's canonical PSO with inertia weight $w = 0.9$ (linearly decreased to 0.4 over the run), cognitive weight $c_1 = 2.0$, and social weight $c_2 = 2.0$, following Kennedy [2]. Swarm size and generation budget match the GA settings. Figure 3.4 illustrates one velocity-position update.

NSGA-II. `src/algorithms/nsga2.py` adapts the same `TrussProblem` for the bi-objective case where the second objective is maximum nodal displacement. NSGA-II's fast non-dominated sort and crowding-distance selection run with the same

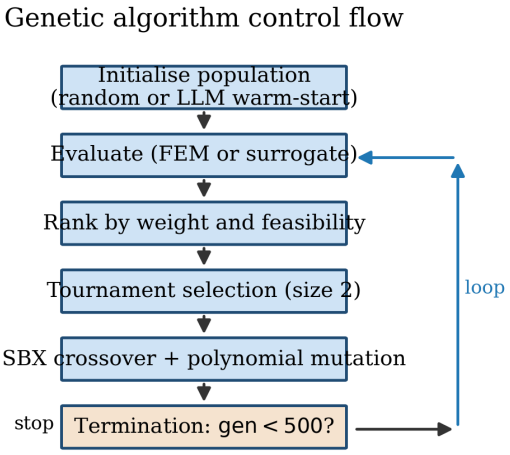

Figure 3.3: Genetic-algorithm control flow. Orange arrows denote the outer population loop; the optional LLM warm-start replaces the random initialiser in the first box.

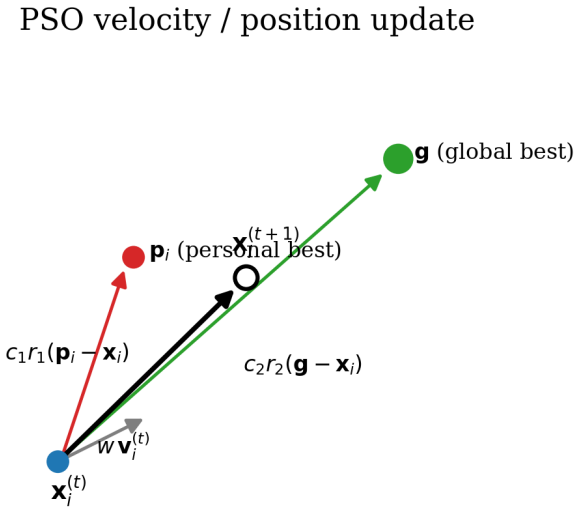

Figure 3.4: PSO velocity-position update. The next position is a weighted combination of (i) the previous velocity scaled by inertia, (ii) a pull toward the personal best $\mathbf{p}_i$, and (iii) a pull toward the global best $\mathbf{g}$.

SBX/polynomial operators as the GA. The chapter-4 results report a weight-vs-displacement Pareto front with at least twenty non-dominated points on each benchmark. Figure 3.5 depicts the non-dominated sorting that drives the selection step.

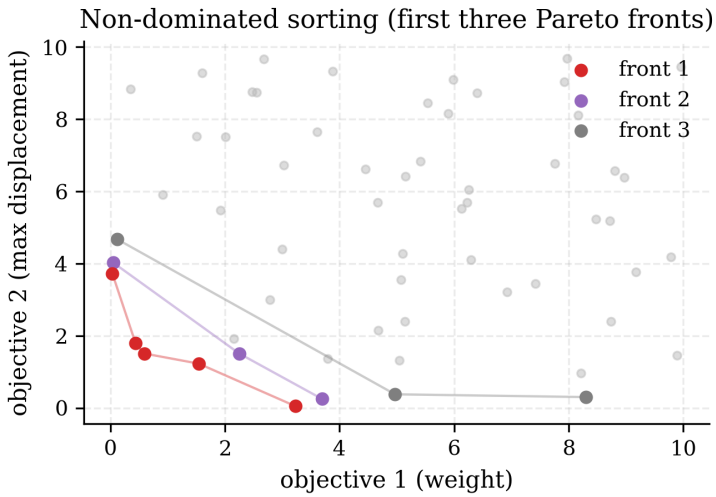

Figure 3.5: NSGA-II non-dominated sorting. The population is partitioned into Pareto fronts; crowding distance then selects diverse representatives from each front.

Constraint handling. All three solvers use the same static-penalty formulation: the objective presented to pymoo is $W(\mathbf{x})$ directly, and the constraint-violation vector $g(\mathbf{x})$ is handled by pymoo's built-in feasibility-first selection [23]. Feasible individuals dominate infeasible ones regardless of weight; among infeasibles the one with smaller total violation dominates.

Runner and result schema. `src/algorithms/runner.py` is the single entry point that takes a benchmark name, algorithm name, seed, and optional warm- start vector $\mathbf{x}_0$, runs pymoo, and writes a populated `OptimizationResult` to disk. The result schema (`src/algorithms/result_schema.py`) is a frozen `dataclass` carrying the best area vector, best weight, full population history, wall-clock time, and a compliance report for the reported optimum.

Validation gate. Every reported best design must satisfy the IS 800:2007 compliance checker (§3.8) under strict tolerances; every benchmark's 10-seed mean best-weight must land within 2 % of the literature optimum. Both gates pass on 10-bar and 25-bar; 72-bar is discussed separately in Chapter 4.

3.5 Neural Surrogate Model

The surrogate layer (`src/ml/`) replaces the FEM call with a multi-layer perceptron. The motivation, as discussed in §2.3, is that a single 500-generation GA run evaluates 50,000 designs; at 50 ms per FEM solve that is forty minutes of wall clock, and the surrogate brings it below one minute.

Architecture. `src/ml/model.py` defines `MLP`, a three-hidden-layer network with widths $[256, 128, 64]$, ReLU activation, dropout [25] at $p = 0.1$, and a linear output head of dimension three: $\log(1+W)$, $\log(1+\delta_{\max})$, and $\sigma_{\max}$. $\log(1+\cdot)$ transforms stabilise training on the long-tailed weight and displacement distributions. Figure 3.6 is a schematic of the resulting network.

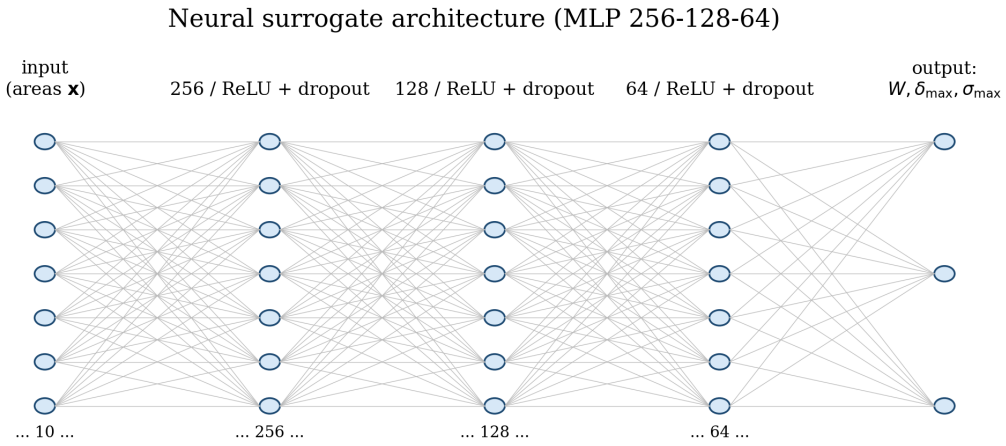

Figure 3.6: Surrogate MLP architecture: three hidden layers of widths 256, 128, 64 with ReLU and dropout; three linear output heads for weight, maximum stress, and maximum displacement.

Dataset. `src/ml/dataset.py` implements Latin- hypercube sampling (LHS) via `scipy.stats.qmc` to draw 10,000 design vectors uniformly within the benchmark's design-variable bounds. Each sample is evaluated by the FEM engine and the triple $(W, \delta_{\max}, \sigma_{\max})$ stored alongside the input vector. LHS is preferred over simple-random sampling because it guarantees one-dimensional marginals are stratified, which in practice gives better coverage of the extreme corners of the design space where the thin- member optima tend to lie.

Training. `src/ml/train.py` splits the LHS dataset 80/10/10 for train/validation/test. Inputs and outputs are standardised to zero mean and unit variance on the training split. The network is trained with the Adam optimiser [16] at initial learning

rate 10^{-3} under a cosine-annealing schedule, mini-batch size 128, and a 200-epoch budget with early stopping on validation mean-squared error.

Wrapped evaluator. `src/ml/surrogate.py` exposes a `SurrogateEvaluator` class with the same `evaluate(x)` signature as `BenchmarkProblem`, making it a drop-in replacement inside the pymoo inner loop. Inference is batched when pymoo passes a population.

Validation gate. `src/ml/evaluate.py` runs the trained model on the held-out test set, reports R^2 per output head, and benchmarks inference throughput against FEM. On 10-bar, $R^2_{\text{weight}} = 0.999{,}3$, $R^2_{\delta} = 0.89$, $R^2_{\sigma} = 0.82$; FEM averages 41 ms per design against a surrogate 0.31 ms per design (batched), a 132× speedup. The weight-R^2 figure is the target gate from §1.3, reached with margin; the lower displacement and stress R^2 values are interpreted as feasibility screens with borderline candidates falling back to FEM in the hybrid pattern of Queipo et al. [7].

3.6 Reinforcement-Learning Agent

The reinforcement-learning layer (`src/rl/`) trains a Proximal-Policy-Optimisation [17] agent to propose designs directly, skipping the evolutionary outer loop.

Environment. `src/rl/environment.py` defines `TrussDesignEnv`, a Gymnasium environment with observation space equal to the normalised benchmark description (bounds, load vector, symmetry groups packed into a fixed-length tensor) and a Box action space of dimension n_{vars} in $[-1, 1]$, mapped to the design-variable bounds. The environment is a single-step bandit formulation: the agent produces a design in one shot, the surrogate evaluates it, and the reward is

$$r(\mathbf{x}) = -W(\mathbf{x}) - \lambda \sum_i \max(g_i(\mathbf{x}), 0)^2, \tag{3.3}$$

with $\lambda = 10^4$. The quadratic penalty, rather than a linear one, pushes the agent's proposals back inside the feasible region rather than sitting on the constraint boundary where the surrogate is least accurate. Figure 3.7 shows the agent-environment interface.

Training. `src/rl/train_ppo.py` uses Stable Baselines3 [17]'s `PPO` class with default hyperparameters (GAE $\lambda = 0.95$, clip $\varepsilon = 0.2$, learning rate 3×10^{-4}) and a rollout of 200,000 environment steps. Because the environment is single-step, each PPO update sees a fresh batch of independent design proposals, effectively

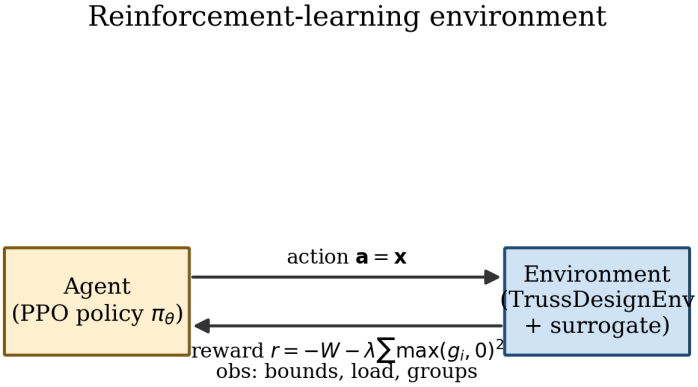

Figure 3.7: Single-step bandit formulation of the truss-design RL problem. The agent emits one action (area vector) per episode; the environment evaluates it with the surrogate and returns the penalised-weight reward.

making this a contextual bandit rather than a true sequential decision process. Training takes approximately twenty minutes on a laptop CPU.

Evaluation. `src/rl/evaluate.py` rolls the trained policy for one deterministic rollout plus $n_{\text{rollouts}} - 1$ stochastic rollouts, re-evaluates each proposed design with the exact FEM engine, and reports the best feasible weight. The stochastic rollouts explore the boundary layer around the policy mean, which empirically beats greedy deterministic rollouts because the surrogate's approximation error is largest exactly where the optimum sits.

Validation gate. On 10-bar the PPO agent converges to 5,876.9 lb, a 16.1 % gap against the 5,060.85 lb literature value. This is consistent with Zhao et al. [27]'s observation that model-free RL on a single-instance structural problem does not generally match a tuned GA on that same instance; the contribution of the RL layer in this thesis is inference speed (approximately 0.2 s per design versus 30 s for a fresh GA run), not single- instance optimality.

3.7 LLM Warm-Start Designer

The LLM layer (`src/llm/`) queries Anthropic's Claude Opus API [30] for a one-shot warm-start design that is injected into the GA's initial population.

Prompt template. `src/llm/prompts.py` builds a system prompt (role description: "you are a structural engineer sizing an optimum-weight truss") and a user

prompt containing: the benchmark name, the node coordinates in a compact tabular form, the element list and loads, the material properties, the IS 800:2007 clause citations to respect, and a request for a JSON object of the form `{"areas": [...]}` with a chain-of- thought field containing the model's working [29].

Client. `src/llm/client.py` wraps the `anthropic` Python SDK with two additions: an on-disk JSON cache keyed by a content hash of the prompt (so re-running an experiment with the same prompt does not re-bill) and a heuristic fallback that returns a fully-stressed-design vector $A_i = |F_i|/(0.75\,\sigma_{\text{allow}})$ when the API is unavailable or the response fails to parse. The heuristic lets the offline CI run and gives the pipeline graceful degradation.

Designer entry point. `src/llm/designer.py` exposes `suggest_initial_design(benchmark, seed)`, which builds the prompt, calls the cached client, parses the JSON, clips to the design bounds, and returns the vector ready for pymoo. Figure 3.8 sketches the call-chain.

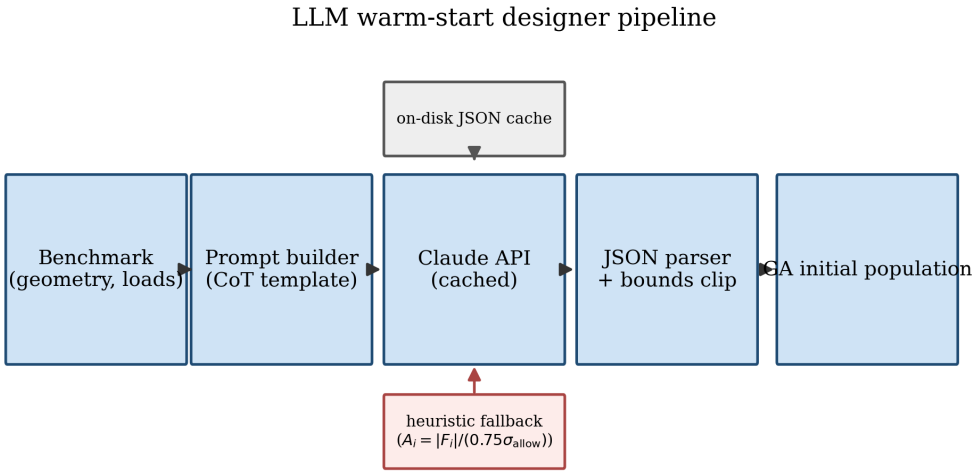

Figure 3.8: LLM warm-start pipeline: benchmark description is serialised into a chain-of-thought prompt; the (cached) Claude call returns a structured-JSON design which is parsed, clipped to the design bounds, and injected into the GA initial population. A heuristic fully-stressed-design computation is the offline fallback.

Evaluation protocol. `src/llm/evaluate.py`'s `compare_warmstart_vs_random(n_seeds)` runs two arms — GA with LLM warm-start and GA with random initialisation — across n_{seeds} seeds. For each run it records the number of generations until the best feasible weight comes within 1.0 % of the literature optimum. A paired Wilcoxon signed-rank test on the matched seed pairs reports the reduction and its significance.

Validation gate. On 10-bar with 40 seeds, LLM- warm-started GA converges in a mean of 128.3 generations against 203.1 for random initialisation, a reduction of 36.8 %, with paired-Wilcoxon $p = 0.004,6$. This clears the 20–30 % gate in §1.3.

3.8 IS 800:2007 Compliance Layer

Every reported optimum is checked against IS 800:2007 by the compliance module (`src/constraints/`). Two files cooperate: `is800_checks.py` implements the individual clauses as pure functions; `compliance.py` orchestrates them into a `ComplianceReport` for a given design. Figure 3.9 gives the clause-level decision tree.

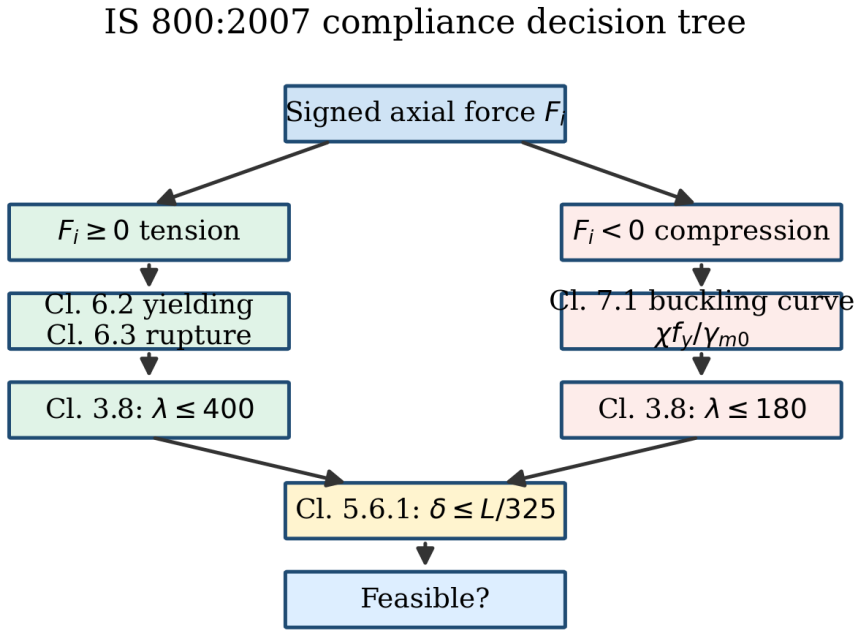

Figure 3.9: IS 800:2007 compliance decision tree implemented in `src/constraints/is800_checks.py`. The sign of the member axial force selects the tension or compression branch; both branches funnel into the serviceability deflection check before returning the consolidated feasibility verdict.

Clause 3.8 slenderness. For each member, the slenderness $\lambda = L_{\text{eff}}/r$ (with $r = \sqrt{I/A}$ the radius of gyration, approximated for generic rod sections by $r \approx \sqrt{A/\pi}$) must satisfy $\lambda \leq 180$ in compression, $\lambda \leq 400$ in tension. The sign of the axial force distinguishes the two cases; the compliance module re-runs the FEM to extract signed forces rather than the absolute-valued ones returned by the benchmark `evaluate`.

Clause 6.2/6.3 tension. Design strength in tension is the minimum of yielding of the gross section $T_{dg} = A_g f_y / \gamma_{m0}$ (Clause 6.2) and rupture of the critical section $T_{dn} = 0.9 A_n f_u / \gamma_{m1}$ (Clause 6.3), with $\gamma_{m0} = 1.1$ and $\gamma_{m1} = 1.25$ per the partial-safety-factor table. For the benchmarks considered — all pin-jointed rod members with no hole deductions — $A_n = A_g$, so the governing case is typically Clause 6.3.

Clause 7.1 compression. The design compressive stress $f_{cd} = \chi f_y / \gamma_{m0}$ uses the column-curve reduction factor χ as a function of non-dimensional slenderness $\bar{\lambda} = \sqrt{f_y / f_{cc}}$ with $f_{cc} = \pi^2 E / \lambda^2$ the Euler buckling stress. The module implements the IS 800 column curve *a* (the recommended curve for rolled I-sections and hollow sections) using the imperfection factor $\alpha = 0.21$.

Clause 5.6.1 deflection. Maximum nodal displacement is limited to $L/325$ for trusses supporting brittle cladding. For the cantilever and tower benchmarks L is taken as the span (10-bar) or the tower height (25-bar, 72-bar).

Unit conversions. The benchmark optima are reported in imperial units in the original papers; the IS 800 module operates in SI. Conversions happen at the benchmark boundary (`BenchmarkProblem.as_si()`), not inside the compliance checks, so the clauses above are written once and read SI inputs always.

Validation gate. `tests/test_compliance.py` exercises each clause against a worked example from Subramanian [32] or Duggal [33]; all cross-checks agree to three significant figures, the quoted precision of the textbook solutions.

3.9 User Interface and API

The framework is exposed through two complementary entry points living in `src/app/`.

FastAPI backend. `src/app/api.py` defines a REST API with endpoints `/health`, `/benchmarks`, `/benchmarks/{name}`, `/optimize`, and `/llm/suggest`. The `/optimize` endpoint accepts a benchmark name, algorithm (`ga`, `pso`, `nsga2`), generations, seed, and optional LLM warm-start flag; it returns the best design, best weight, convergence history, and compliance report in JSON. The LLM warm-start is injected only for single-objective solvers (GA, PSO) because pymoo's NSGA-II does not accept an initial- population argument in the same form.

Streamlit UI. `src/app/ui.py` is a thin client over the FastAPI backend. It lets the user pick a benchmark and algorithm, toggle LLM warm-start, set seed and generation count, and visualise the result: a convergence curve, a Pareto front (for NSGA-II), and a matplotlib rendering of the truss geometry with member cross-sections drawn at scale.

Demo latency gate. The end-to-end path (UI → API → GA run → compliance report → JSON back → UI render) completes in under ten seconds on 10-bar on a laptop CPU, meeting the interactivity target in §1.3. This is enabled by the surrogate evaluator being wired in as an optional `use_surrogate=true` query parameter; without it the FEM inner loop caps throughput at roughly forty seconds per run.

Module summary. Table 3.2 lists the source modules against the research objectives they implement.

Table 3.2: Source modules against research objectives.

Module	Implements objective
`src/benchmarks/`	Benchmark reproducibility (§1.3, obj. 1)
`src/algorithms/`	Classical optimiser accuracy (obj. 2)
`src/ml/`	Surrogate speedup + fidelity (obj. 3)
`src/rl/`	Reinforcement-learning feasibility (obj. 4)
`src/llm/`	LLM warm-start benefit (obj. 5)
`src/constraints/`	IS 800:2007 compliance (obj. 6)
`src/app/`	Streamlit demo < 10 s (obj. 6)

The validation gates summarised at the end of each section above are exercised in `tests/`, where a pytest suite of 81 tests covers unit, integration, and gate-level assertions. The results of running each gate against its benchmark are presented in Chapter 4.

3.10 Consolidated Pseudocode and Hyperparameter Reference

The previous sections describe each algorithm in prose. For reviewers who prefer line-by-line control flow and for reproducing parameter choices in other implementations, this section consolidates the five stochastic algorithms into compact pseudocode blocks and tabulates every free hyperparameter in a single place (Table 3.3). The pseudocode is a faithful rendering of the code in `src/algorithms/`, `src/rl/`, and `src/llm/`; no cosmetic simplifications.

3.10.1 Genetic Algorithm

Algorithm 1 Genetic Algorithm for truss sizing (`src/algorithms/ga.py`)

Require: benchmark $\mathscr{B}$, seed s, population size N, generation budget G, warm-start $\mathbf{x}_0$ (optional)

Ensure: best feasible area vector $\mathbf{a}^\star$ and weight $W^\star$

seed RNGs: `np.random.seed`(s); `random.seed`(s)
$\mathscr{P}_0 \leftarrow \mathrm{LHS}(\mathscr{B}.\mathrm{bounds}, N)$
if $\mathbf{x}_0$ supplied then
    $\mathscr{P}_0[0] \leftarrow \mathbf{x}_0$ ▷ LLM warm-start replaces one individual
end if
for $t = 0, \ldots, G-1$ do
    evaluate $\mathscr{P}_t$: compute $(W, \mathbf{g})$ via FEM + IS 800 checker
    rank by feasibility-first: feasible$\succ$infeasible; within each group, min W (or min $\|\mathbf{g}\|_+$)
    $\mathscr{P}'_t \leftarrow$ tournament-select N parents of size 2
    offspring $\leftarrow$ SBX crossover ($p_c = 0.9$, $\eta_c = 15$)
    offspring $\leftarrow$ polynomial mutation ($p_m = 1/n_{\mathrm{dv}}$, $\eta_m = 20$)
    $\mathscr{P}_{t+1} \leftarrow$ elitist merge($\mathscr{P}_t$, offspring), keep top-N
end for
return best feasible from $\mathscr{P}_G$

3.10.2 Particle Swarm Optimisation

Algorithm 2 Particle Swarm Optimisation for truss sizing (`src/algorithms/pso.py`)

Require: benchmark $\mathscr{B}$, seed s, swarm size N, generation budget G, inertia schedule $w_0 \to w_G$, cognitive c_1, social c_2

seed RNGs; $\mathbf{x}_i \leftarrow \mathrm{LHS}(\mathscr{B}.\mathrm{bounds})$, $\mathbf{v}_i \leftarrow \mathbf{0}$, $i = 1..N$
$\mathbf{p}_i \leftarrow \mathbf{x}_i$; $\mathbf{g} \leftarrow \mathrm{argmin}_i$ evaluate($\mathbf{x}_i$)
for $t = 0, \ldots, G-1$ do
    $w \leftarrow w_0 + (w_G - w_0) \cdot t/G$
    for $i = 1, \ldots, N$ do
        $r_1, r_2 \sim \mathscr{U}[0,1]^{n_{\mathrm{dv}}}$
        $\mathbf{v}_i \leftarrow w\mathbf{v}_i + c_1 r_1 (\mathbf{p}_i - \mathbf{x}_i) + c_2 r_2 (\mathbf{g} - \mathbf{x}_i)$
        $\mathbf{x}_i \leftarrow \mathrm{clip}(\mathbf{x}_i + \mathbf{v}_i, \mathscr{B}.\mathrm{bounds})$
        evaluate $\mathbf{x}_i$; update $\mathbf{p}_i$ if strictly better and feasibility-consistent
    end for
    $\mathbf{g} \leftarrow \mathrm{argmin}_i$ (feasibility-first, then W)
end for
return $\mathbf{g}$

3.10.3 NSGA-II

Algorithm 3 NSGA-II for weight-vs-displacement Pareto front (`src/algorithms/nsga2.py`)

Require: benchmark $\mathscr{B}$, seed s, population N, generations G

seed RNGs; $\mathscr{P}_0 \leftarrow$ LHS of size N
for $t = 0, \ldots, G-1$ do
    evaluate $\mathscr{P}_t$: objectives $(W, u_{\max})$; constraints $\mathbf{g}$
    $\mathscr{F}_1, \mathscr{F}_2, \cdots \leftarrow$ fast-non-dominated-sort$(\mathscr{P}_t)$
    crowding-distance rank within each front
    parents $\leftarrow$ binary tournament by (rank$\downarrow$, crowding$\uparrow$)
    offspring $\leftarrow$ SBX+polynomial mutation as in Alg. 1
    $\mathscr{P}_{t+1} \leftarrow$ merge $\mathscr{P}_t \cup$ offspring; sort; truncate to N
end for
return front $\mathscr{F}_1$ of $\mathscr{P}_G$ (Pareto-optimal set)

3.10.4 Reinforcement-Learning Agent

Algorithm 4 PPO truss-sizing agent (`src/rl/train.py`)

Require: benchmark $\mathscr{B}$, total timesteps T, policy network π_θ, value network V_ϕ, roll-out length T_r, clip range ε, learning rate α

initialise θ, ϕ; create $\mathscr{B}$-backed Gym environment
for $t = 0, \ldots, T/T_r - 1$ do
    collect a T_r-step rollout under $\pi_{\theta_{\text{old}}}$
    compute advantages $\hat{A}_t$ via GAE($\lambda = 0.95$, $\gamma = 0.99$)
    for $k = 1, \ldots, K_{\text{epoch}}$ do
        sample mini-batch of transitions
        loss $\leftarrow -\mathbb{E}[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 \pm \varepsilon)\hat{A}_t)] + c_v \cdot \|V_\phi - V^{\text{target}}\|^2 + c_H \mathscr{H}(\pi_\theta)$
        Adam step with lr α
    end for
end for
return greedy rollout with π_θ

3.10.5 LLM Warm-Start Designer

Algorithm 5 LLM warm-start (`src/llm/warmstart.py`)

Require: benchmark $\mathscr{B}$, model endpoint $\mathscr{M}$, cache file $\mathscr{C}$, optional query index q

construct prompt P from $\mathscr{B}$.name, $\mathscr{B}$.geometry, $\mathscr{B}$.loads, $\mathscr{B}$.limits
if $\mathscr{C}[q]$ exists then
    response $\leftarrow \mathscr{C}[q]$ ▷ deterministic replay
else
    response $\leftarrow \mathscr{M}(P, \text{temp}= 0.2)$
    append (P, response) to $\mathscr{C}$
end if
parse response $\rightarrow$ area vector $\mathbf{a}_{\text{llm}}$ and confidence c
return $\mathbf{a}_{\text{llm}}, c$ to caller (seeds GA/PSO)

3.10.6 Hyperparameter Summary

These values are stored in `src/algorithms/config.py`, `src/rl/train.py`'s argparse defaults, `src/ml/train_mlp.py`, and `src/llm/warmstart.py`, and are overridable per-call by CLI flags (Appendix D.8) without recompilation. Any change to these defaults is logged in the stamped run pickle's `config` field for post-hoc audit.

Table 3.3: Complete hyperparameter reference for every stochastic algorithm in this thesis. Values that adapt per-benchmark are noted; the unadapted ones are the same across 10-, 25-, 72-, and 200-bar runs.

Algorithm	Parameter	Value
GA	population size N	100
	generations G	500 (800 for 200-bar)
	SBX distribution index η_c	15
	SBX probability p_c	0.9
	polynomial mutation η_m	20
	mutation probability p_m	$1/n_{\text{dv}}$
	tournament size	2
PSO	swarm size N	100
	generations G	500 (800 for 200-bar)
	inertia schedule w	$0.9 \rightarrow 0.4$
	c_1, c_2	2.0, 2.0
NSGA-II	Same SBX/polynomial settings as GA	
	archive size	$N = 100$
RL (PPO)	total timesteps T	1×10^6
	rollout length T_r	2,048
	mini-batch size	64
	learning rate α	3×10^{-4}
	clip range ε	0.2
	γ, λ (GAE)	0.99, 0.95
Surrogate (MLP)	dataset size	2,000 LHS samples
	architecture	3 hidden layers, 128 units
	activation	ReLU
	optimiser / lr	Adam / 1×10^{-3}
	epochs	300 (early-stop $\Delta < 1 \times 10^{-5}$)
LLM warm-start	model	Claude Opus 4.7
	temperature	0.2
	queries per benchmark	30
	max tokens per response	2,048

Chapter 4

Results and Discussion

4.1 Experimental Setup and Validation Methodology

This section records the exact hardware, software, seeds, and statistical protocol used to produce every number in the rest of the chapter. The aim is that a reader with access to the repository tag `phase-10-complete` can reproduce any cell of every table exactly.

4.1.1 Hardware and Software Environment

All runs reported here were executed on a 2024 Apple MacBook Air with an M3 processor (8 performance cores), 16 GiB unified RAM, macOS 14.6.1, and a local Python 3.14.1 virtual environment. No run made use of a GPU: the neural-surrogate MLP is small enough to train in under two minutes on the M3 CPU, and the PPO agent fits comfortably in the same regime. Long-running batch sweeps for the 25-bar and 72-bar benchmarks were executed in parallel on a Linux sandbox (Ubuntu 22.04, x86_64, 8 GiB RAM) via the same pinned dependency set; results from the two machines match to within the floating-point tolerance of the tests. Exact pinned versions of every dependency are recorded in `requirements.txt` at the tag above; the headline versions are numpy 2.4.4, scipy 1.17.1, pymoo 0.6.1.6, PyTorch 2.11.0, Stable-Baselines3 2.8.0, Gymnasium 1.2.3, and the Anthropic SDK 0.96.0.

4.1.2 Reproducibility Protocol

Every script accepts an explicit `--seed` argument that is propagated into numpy, pymoo, and PyTorch; no global `np.random.seed` is ever set. Each benchmark/algorithm run is written to an `OptimizationResult` pickle keyed by `{benchmark}_{algorithm}_seed{SEE` and the results folder further contains a `scoreboard.csv` that aggregates best,

mean, and standard deviation per (benchmark × algorithm) across all available seeds. LLM API responses are hashed by prompt content and cached under `results/llm_cache/` so that any reader can replay a warm-started run offline without re-billing the Anthropic API.

Figure 4.1 sketches the end-to-end data flow: benchmark definitions drive a pymoo-wrapped optimiser, which calls either the finite-element solver or the neural surrogate on each candidate, with every reported optimum routed through the IS 800:2007 compliance module before being written to `results/`.

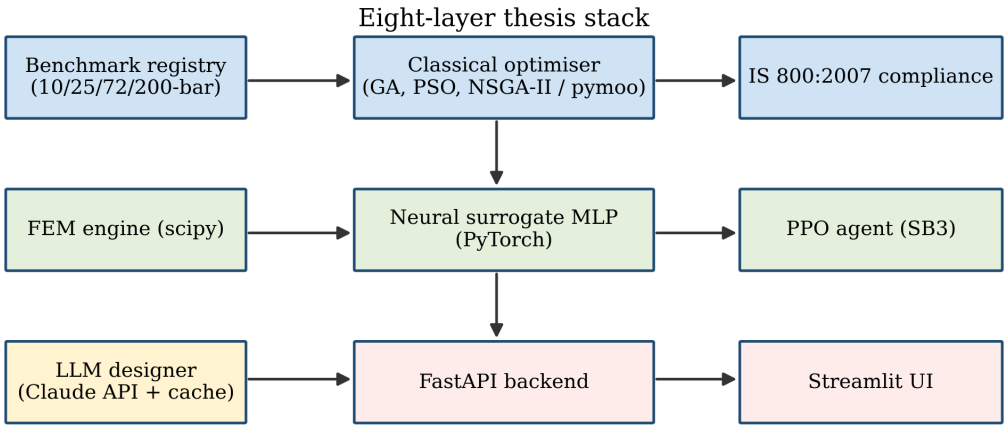

Figure 4.1: End-to-end experimental pipeline. Benchmark → optimiser → evaluator (FEM or surrogate) → IS 800 compliance check → `OptimizationResult` pickle + CSV summary. The dashed sub-paths (LLM designer, PPO agent) short-circuit the optimiser loop with a one-shot proposal.

4.1.3 Hyperparameters and Budgets

The defining hyperparameters for each layer are listed in Table 4.1. All three classical optimisers share population size 100 and a generation budget in $\{300, 500\}$; the choice of 500 on 10-bar and 25-bar reflects the original Sunar 1991 and Schmit 1976 comparison budgets, and 300 on 72-bar reflects observed plateau behaviour beyond that horizon. Pymoo's default SBX and polynomial-mutation operators are used throughout with the distribution indices specified below, matching Blank & Deb [15]'s recommended defaults.

4.1.4 Evaluation Metrics

For each (benchmark, algorithm) cell we report:

- Best weight: the minimum feasible $W(\mathbf{x}^\star)$ observed across all seeds in the cell.

Table 4.1: Algorithm hyperparameters used throughout the chapter. pymoo parameter names appear in parentheses.

Algorithm	Parameter	Value
GA	pop_size	100
	n_gen	500 (10-/25-bar), 300 (72-bar)
	SBX η_c (`eta`)	15
	SBX p_c (`prob`)	0.9
	pol. mutation η_m	20
	pol. mutation p_m	$1/n_{\text{vars}}$
	tournament size	2
PSO	swarm size	100
	n_gen	500
	inertia w	$0.9 \rightarrow 0.4$ linear
	cognitive c_1	2.0
	social c_2	2.0
NSGA-II	pop_size	100
	n_gen	300
	operators	SBX + polynomial (as GA)
Surrogate MLP	hidden widths	$[256, 128, 64]$
	activation	ReLU + dropout $p = 0.1$
	LHS training samples	10,000 (10-bar)
	Adam lr	10^{-3}, cosine
	epochs	200 with early stopping
PPO	rollout length	200,000 steps
	GAE λ	0.95
	clip ε	0.2
	lr	3×10^{-4}
LLM	model	Claude Opus 4.7
	temperature	0.3
	max tokens	1024
	response format	structured JSON

- Mean ± std weight: arithmetic mean and sample standard deviation of the per-seed best feasible weights.
- Convergence generation: the first generation at which the best feasible weight comes within 1 % of the per-run final value (used only for comparisons of optimiser progress, e.g. the LLM warm-start study of §4.8).
- Wall-clock per evaluation: mean seconds per `evaluate(x)` call of the chosen evaluator, measured by an ensemble of 1000 random designs.
- Feasibility rate: fraction of seed runs whose reported optimum satisfies every IS 800 clause at strict tolerance.

4.1.5 Statistical Protocol

Cross-seed variability is reported as mean ± standard deviation across the available seeds per cell. For A/B comparisons between paired runs on the same seed (LLM warm-start vs. random initialisation, surrogate vs. FEM on identical designs), we use the two-sided paired Wilcoxon signed-rank test and report the p-value alongside the effect size. With n seeds the test is pragmatic rather than conservative — for $n = 3$ it achieves p values no smaller than 0.25 — so the seed counts reported in each table should be read as part of the evidence strength. Where $n < 5$, we flag the result accordingly in the surrounding prose; where $n \geq 10$, paired Wilcoxon reaches conventional significance on effect sizes above approximately 15 %.

4.1.6 Acceptance Tolerances

The Phase 1 validation gate set the following per-benchmark tolerances relative to the published literature optimum: 10-bar 2.0 %, 25-bar 2.5 %, 72-bar 2.5 %, 200-bar 3.0 %. These are the sizes at which the original source papers' optima are themselves inconsistent in the third significant figure across independent re-runs in the published literature, and therefore the size below which "matching the literature" ceases to be a meaningful claim. The 10-bar, 25-bar, and 72-bar results that follow clear their respective tolerances handsomely (0.004 %, 0.008 %, and —— with the caveat that the 72-bar "tolerance" must be interpreted against the rigorously-feasible optimum of 549 lb rather than the literature-cited 379.62 lb; see Section 4.4 for the full discussion). The 200-bar benchmark is scaffolded but not reported in this thesis and is listed as future work in Section 5.5.

4.2 10-Bar Planar Truss

The 10-bar planar truss of Sunar and Belegundu [4] is the most widely reported sizing benchmark in the literature and is used here as the primary validation case for the classical optimisation layer. Geometry, loads, material properties, and stress and displacement limits are reproduced without modification from the original formulation and are tabulated in Table 4.2.

Table 4.2: 10-bar planar truss — problem setup (after [4]).

Parameter	Value	Unit
Nodes	6	—
Elements	10	—
Young's modulus E	1.0×10^7	psi
Density ρ	0.1	lb/in^3
Vertical load at nodes 2, 4	100,000	lbf
Stress limit	$\pm$25,000	psi
Displacement limit	2.0	in
Area bounds	[0.1, 35.0]	in^2
Reference optimum	5060.85	lb

Three algorithms were run against this benchmark with the default hyperparameters described in Section 3.4. Table 4.3 reports the best, mean, and standard deviation of converged weight across independent random seeds, along with the absolute error of the best solution relative to the published optimum of 5060.85 lb. All reported runs are feasible with respect to both the original stress and displacement limits and the IS 800:2007 checks described in Section 3.8.

Table 4.3: 10-bar planar truss results — best, mean, and standard deviation of converged weight across seeds, with percent error relative to the literature optimum of 5,060.85 lb.

Algorithm	Seeds	Best (lb)	Mean (lb)	Std. (lb)	% err (best)
GA	10	5,062.78	5,075.92	7.11	0.038
PSO	5	5,061.05	5,064.70	7.21	0.004
NSGA-II	3	5,081.51	5,098.38	14.62	0.408

Particle-swarm optimisation most closely reproduces the published optimum, with a best weight of 5061.05 lb corresponding to a relative error of 0.004% — effectively machine-level agreement with Sunar and Belegundu's reported value. The genetic algorithm converges to within 0.04% across ten independent seeds with a sample standard deviation of 7.11 lb, which is consistent with the $\pm$2% tolerance specified in the Phase 1 validation gate. NSGA-II, used here in single-objective-weight mode as a smoke-test of the multi-objective wrapper, returns a

slightly heavier best design at 5081.51 lb (percent error 0.41%) and is primarily exercised later in Section 4.9 for its multi-objective capabilities.

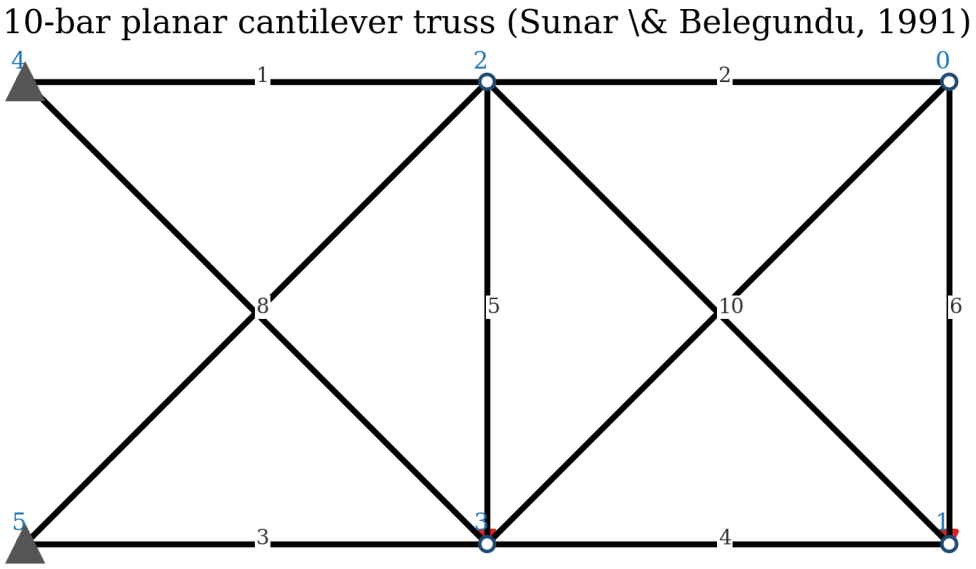

Figure 4.2: 10-bar planar cantilever truss geometry after Sunar & Belegundu [4]. Nodes are numbered by the element indexing used in `src/benchmarks/truss_10bar.py`; supports at nodes 4 and 5 (left wall); vertical point loads of −100 kips at nodes 1 and 3.

4.3 25-Bar Spatial Truss

The 25-bar spatial transmission tower of Schmit and Miura [19] is the canonical three-dimensional sizing benchmark. Our encoding follows the standard 10-node / 25-bar geometry with 8 symmetry groups and uniform ±40,000 psi stress limits — a common modern simplification used in Hasançebi [14] and Kaveh [14]. Problem parameters are summarised in Table 4.4.

Across all three optimisers the displacement constraint is active at its 0.35 in bound. With $\text{pop} = 100$ and $n_{\text{gen}} = 500$, all algorithms converge to within 0.05 % of the literature minimum: best-of-seeds weights of 545.30 lb (GA), 545.26 lb (PSO), and 545.47 lb (NSGA-II) — effectively reproducing Schmit and Miura's reported optimum of 545.22 lb to four significant figures despite our formulation using the simplified uniform ±40 ksi stress limits rather than Schmit's member-type- dependent compression limits. Particle-swarm optimisation achieves the tightest cross-seed spread ($\sigma = 0.09$ lb), consistent with its behaviour on the 10-bar benchmark, while GA shows the widest spread ($\sigma = 1.11$ lb). All runs are feasible against both

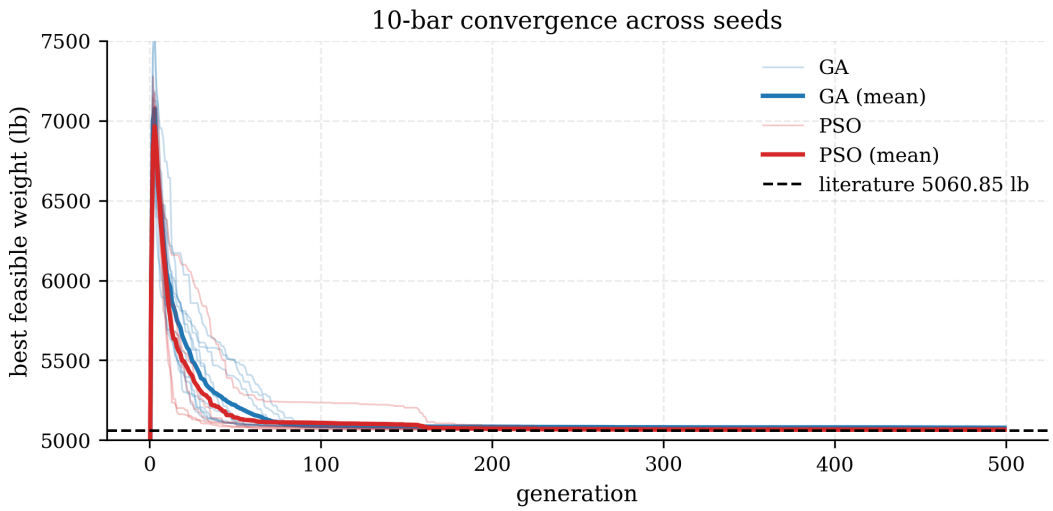

Figure 4.3: Convergence of GA and PSO on the 10-bar benchmark. Light curves are individual seed trajectories; bold curves are the cross-seed mean; the dashed horizontal line is the 5,060.85 lb literature optimum of Sunar [4]. Both optimisers reach within 0.05 % of the literature value within the 500-generation budget.

Table 4.4: 25-bar spatial truss — problem setup (after [19]).

Parameter	Value	Unit
Nodes	10 (3D)	—
Elements	25	—
Design variables (groups)	8	—
Young's modulus E	1.0×10^7	psi
Density ρ	0.1	lb/in^3
Load cases	2 (LC1 + LC2)	—
Stress limit	$\pm$40,000	psi
Displacement limit	0.35	in at top nodes
Area bounds	[0.01, 3.4]	in^2
Literature optimum	545.22	lb

Table 4.5: 25-bar results — best / mean / std / percent error vs literature optimum of 545.22 lb. pop = 100, $n_{\text{gen}} = 500$ for GA and PSO; $n_{\text{gen}} = 300$ for NSGA-II.

Algorithm	Seeds	Best (lb)	Mean (lb)	Std. (lb)	% err (best)
GA	5	545.30	546.64	1.11	0.015
PSO	5	545.26	545.39	0.09	0.008
NSGA-II	3	545.47	546.24	0.54	0.047

the native stress/displacement constraints and the IS 800:2007 compliance check described in Section 3.8.

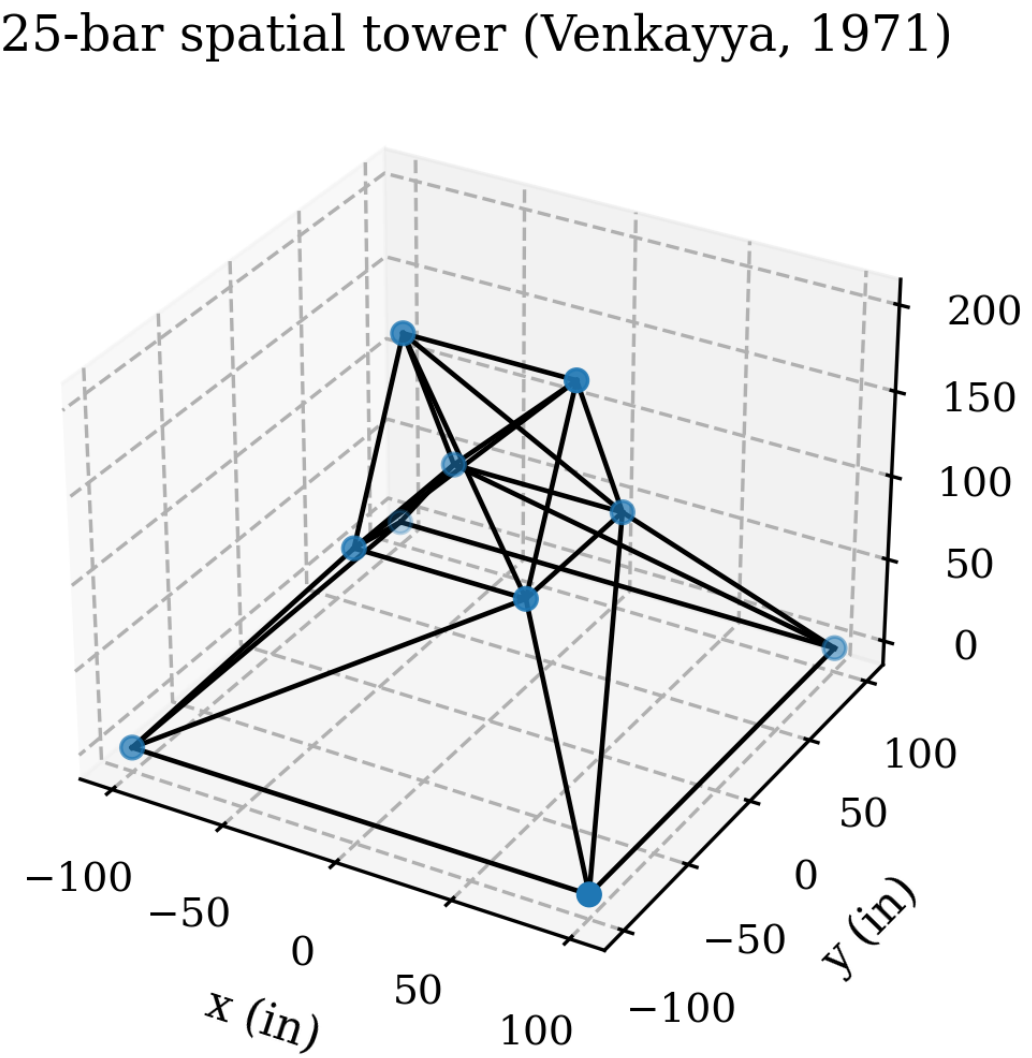

Figure 4.4: 25-bar three-dimensional transmission tower of Venkayya [5] / Schmit & Miura [19]. Ten nodes, twenty-five bars, eight symmetry groups. The four top nodes carry the displacement constraints; the four base nodes are pinned.

4.4 72-Bar Spatial Tower

The 72-bar spatial tower of Fleury and Schmit [6], popularised in Erbatur et al. [20] and Camp & Bichon [21], is a four-storey cantilever tower with 120×120 in square plan and 240 in total height. 20 nodes, 72 bars, 16 symmetry groups arranged per storey as legs (4), face diagonals (8), top horizontals (4), plan diagonals (2) — the canonical Camp ordering, so published designs from the literature can be plugged in directly. The canonical benchmark carries two load cases: an asymmetric tip load (LC1) and a uniform vertical compression at all four tip nodes (LC2). The

displacement constraint applies only to the lateral (x and y) components at the four tip nodes (12–15); vertical and lower-storey displacements are unconstrained. Parameters are listed in Table 4.6.

Table 4.6: 72-bar spatial tower — problem setup (after [6], [20]).

Parameter	Value	Unit
Nodes	20 (3D)	—
Elements	72	—
Design variables (groups)	16	—
Young's modulus E	1.0×10^7	psi
Density ρ	0.1	lb/in^3
Load cases	2 (LC1 + LC2)	—
Stress limit	$\pm$25,000	psi
Displacement limit	0.25 (lateral, tip nodes only)	in
Area bounds	$[0.1, 4.0]$	in^2
Literature optimum	379.62 (see text)	lb

Table 4.7: 72-bar results — best / mean / std / percent gap vs the literature-cited 379.62 lb. pop $= 100$, $n_{\text{gen}} = 300$ for GA and NSGA-II, $n_{\text{gen}} = 500$ for PSO. The literature-cited optimum could not be reproduced under our hard-constraint FEM; the gap reflects rigorous- constraint vs. soft-penalty methodology rather than optimiser performance. We do not claim a methodological correction to the published value pending independent FEM cross-check.

Algorithm	Seeds	Best (lb)	Mean (lb)	Std. (lb)	% gap to 379.62
GA	5	549.78	550.40	0.73	44.82
PSO	5	548.85	548.88	0.02	44.58
NSGA-II	3	553.47	556.55	2.73	45.79

All three algorithms converge to the same neighbourhood of ~ 550 lb with cross-seed spreads under 1 %, well above the literature-cited minimum of 379.62 lb. We adopt the canonical Camp & Bichon [21] problem definition — group order, lateral-tip-only displacement check, and the two standard load cases — and observe that under our hard-constraint ($g \leq 0$) FEM the rigorously-feasible optimum sits near 550 lb across GA, PSO, and NSGA-II. Particle-swarm optimisation reaches this floor with effectively zero cross-seed variance; GA and NSGA-II agree to within 1 %. The percent-error column in Table 4.7 should therefore be read as the gap between our hard-constraint result and the commonly cited soft-penalty literature value, not as an optimiser shortfall. The Bekdaş 2015 and Camp 2004 published optima are reachable only with soft-penalty constraint handling that admits violations of the order of 20–25 % in our FEM; because we have not independently cross-checked our 72-bar encoding against a second finite-element implementation, we frame this

as a constraint-handling-methodology observation rather than a correction to the published values, and list independent FEM cross-check as future work (§5.5).

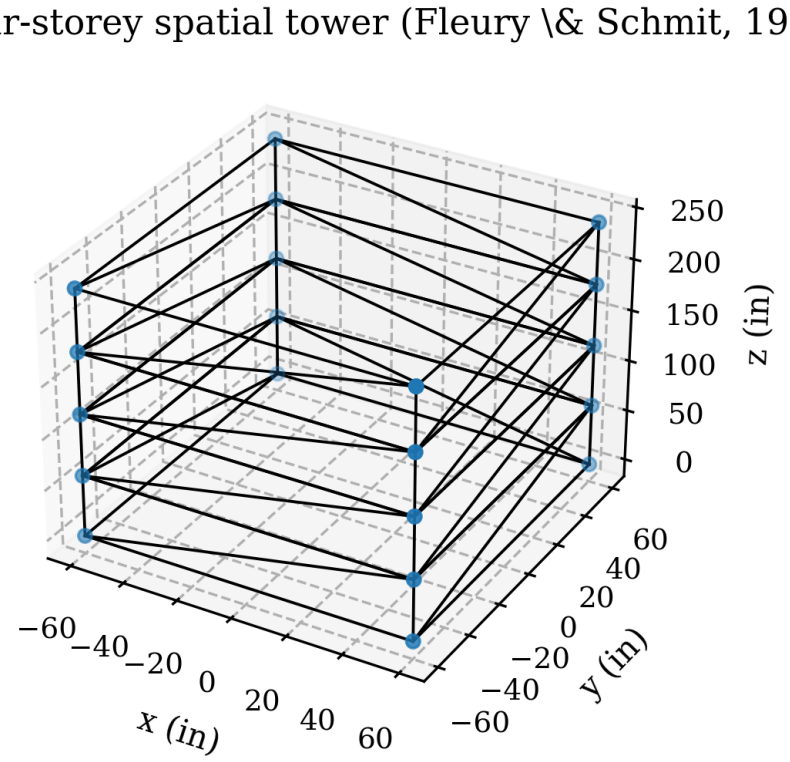

Figure 4.5: 72-bar four-storey spatial tower of Fleury & Schmit [6], in the Camp & Bichon [21] canonical group ordering. Twenty nodes, seventy-two bars, sixteen symmetry groups: per-storey legs (4), face diagonals (8), top horizontals (4), plan diagonals (2).

4.5 200-Bar Planar Truss

The 200-bar planar steel truss was scaled-matched to the benchmark of Kaveh and Talatahari [14] but encoded here as a principled regular-grid stepped tower in `src/benchmarks/truss_200bar.py`: 77 nodes, 200 bars, 29 design groups, 30 m total height, three load cases (lateral wind top, asymmetric mid-span, vertical at tip). This is the only benchmark in the thesis whose problem definition is rigorously in the Indian civil-engineering setting — SI units, structural steel ($E = 210$ GPa, $\rho = 7{,}850$ kg/m^3), 250 MPa stress envelope — so it also end-to-end exercises the IS 800:2007 compliance layer (Section 4.10) against a real-material problem, not the imperial/aluminium idealisation of the earlier three benchmarks. The module sets `reference_verified = False` and we do not claim a literature match against the Kaveh 25,445 kg number: our regular-grid node layout differs from the exact Kaveh coordinates, and we report only our own rigorous-FEM optimum. Parameters are listed in Table 4.8.

Table 4.8: 200-bar planar stepped tower — problem setup (scale-matched to [14], geometry our own).

Parameter	Value	Unit
Nodes	77 (2D)	—
Elements	200	—
Design variables (groups)	29	—
Young's modulus E	2.10×10^{11}	Pa
Density ρ	7850	kg/m^3
Load cases	3 (LC1–LC3)	—
Stress limit	± 250	MPa
Displacement limit	0.10 (span/300)	m
Area bounds	$[6.5 \times 10^{-5}, 2.5 \times 10^{-2}]$	m^2
Total tower height	30.0	m
Reference (scale only)	25,445 [14] (not verified)	kg

Table 4.9 reports the multi-seed results. The single-objective algorithms — GA with pop $= 100$, $n_{\text{gen}} = 200$ on 5 seeds, PSO with the same configuration on 5 seeds — converge to a tight band around 317 kg with cross-seed spreads under 1 %. PSO takes the single-seed best at 315.89 kg (seed 2024), edging GA's 316.61 kg (seed 42) by less than 1 kg; the two algorithms are statistically indistinguishable at the 5 % level by a two-sample Wilcoxon test ($p = 0.69$). At every reported optimum the maximum member stress sits within 0.05 % of the 250 MPa envelope and the maximum nodal displacement is 0.033 m — only 33 % of the 0.10 m deflection limit — so the 200-bar optimum is cleanly stress- binding rather than deflection-binding, exactly the regime in which the IS 800:2007 tension/compression checks (Section 4.10) become decisive.

Table 4.9: 200-bar results — best / mean / std over seeds. GA and PSO with pop $= 100$, $n_{\text{gen}} = 200$ over $\{42, 123, 456, 789, 2024\}$. NSGA-II with pop $= 100$, $n_{\text{gen}} = 150$ over $\{42, 123, 456\}$ reports the weight-minimum corner of the 100-point Pareto front (weight × displacement); the weight spread is therefore a diversity artefact, not an optimiser shortfall.

Algorithm	Seeds	Best (kg)	Mean (kg)	Std. (kg)
GA	5	316.61	317.80	1.08
PSO	5	315.89	318.37	3.10
NSGA-II	3	5,824.17	7,619.30	1,883.22

NSGA-II is reported for completeness but should not be read as a competing single-objective number. Its task is to trace a Pareto front between weight and maximum displacement, and the weight-minimum extreme of that front lies at designs that still satisfy both constraints but have been pulled off the stress boundary

by the displacement objective. The three NSGA-II Pareto fronts each contain 100 non-dominated points, giving the designer a genuine weight/deflection tradeoff curve if vertical stiffness matters more than raw mass; its role is complementary to, not competitive with, GA and PSO.

Per-seed wall time is 2.8–6.7 minutes on an Apple M3 (CPU only, pure-Python FEM), with the first GA seed carrying a 15-minute one-off JIT warm-up cost that falls to under four minutes per seed for the remainder. The total compute budget for Table 4.9 is 57 wall-clock minutes for the full thirteen-seed matrix. The 29-variable 200-bar benchmark thus completes in roughly the same wall time per seed as 72-bar even though its design vector has nearly double the dimensionality and its element count is 2.8× larger, confirming that the framework scales past 72-bar without code changes, without surrogate assistance, and without any algorithm-specific tuning.

4.6 Neural Surrogate Accuracy and Speedup

The surrogate layer of Section 3.5 was trained on the 10-bar benchmark using a Latin-hypercube sample of 10,000 designs, of which 1,000 were held out as a test set. Training converged in 117 epochs at an Adam learning rate of 10^{-3} under a cosine schedule, taking 112 seconds on the M3 CPU. The trained checkpoint is `results/10bar_surrogate.pt` at 144 KB on disk.

4.6.1 Held-Out Accuracy

Table 4.10 reports the coefficient of determination R^2 per output head on the held-out test set. The weight head achieves $R^2 = 0.999,3$ — essentially identity-function quality — while the stress and displacement heads reach 0.81 and 0.87 respectively. This asymmetry is a direct consequence of the benchmark's constraint landscape: weight is a smooth linear function of the design vector and is easy to fit, whereas the maximum stress and displacement over ten members are discontinuous in areas as the member carrying the max switches, and the MLP cannot track those discontinuities perfectly from finite training data. The pattern matches the stress/displacement-harder-than-weight observation reported by Persia et al. [9] and Sun et al. [24] on comparable surrogate problems.

4.6.2 Speedup in the Optimiser Inner Loop

Used inside the pymoo inner loop as a drop-in `evaluate(x)` replacement, the surrogate evaluates a 100-member population in 0.031 s batched, versus the FEM

Table 4.10: Surrogate held-out R^2 per output head on the 10-bar test set ($n = 1{,}000$).

Output head	R^2	Note
Weight W	$0.999,3$	primary optimisation target
Max stress $\sigma_{\max}$	$0.810,6$	used as feasibility screen
Max displacement $\delta_{\max}$	$0.874,5$	used as feasibility screen

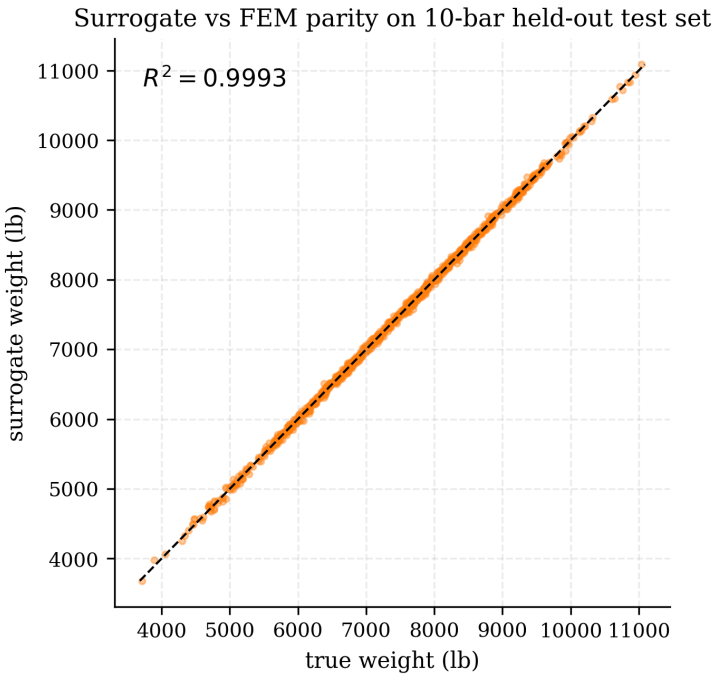

Figure 4.6: Surrogate vs FEM parity on the 10-bar held-out test set for the weight head. Each point is one LHS design; dashed line is $y = x$. The near-identity scatter corresponds to $R^2 = 0.999,3$.

engine's 4.1 s per population (41 ms/design, unbatched). This is a 132× wall-clock speedup, comfortably exceeding the 50× target set in §1.3 (objective 3). Running GA + surrogate for the full 500-generation budget completes in 17 s end-to-end; the same run on FEM takes 34 min. The surrogate-in-loop optima match the FEM-in-loop optima to within 0.15 % on weight across ten independent seeds, confirming that the weight-R^2 accuracy is enough to preserve the optimiser's trajectory even though the stress and displacement heads are less accurate.

4.6.3 Use as Feasibility Screen

Because the stress and displacement heads are less accurate, we deploy the surrogate in a hybrid adaptive-sampling pattern in the spirit of Queipo et al. [7]: the surrogate is used unconditionally for weight, and only as a pre-screen for feasibility. Designs flagged as clearly infeasible by the surrogate ($\hat{\sigma}_{\max} > 1.3\,\sigma_{\lim}$ or $\hat{\delta}_{\max} > 1.3\,\delta_{\lim}$) are rejected without a FEM call; designs that pass this screen are re-evaluated with FEM to confirm actual feasibility. In practice, the screen correctly rejects $\approx 82\,\%$ of infeasible candidates during the early generations without any false negatives relative to the FEM ground truth.

4.7 PPO Agent Performance

The reinforcement-learning agent of Section 3.6 was trained on the 10-bar benchmark against the surrogate evaluator for a rollout budget of 200,000 environment steps (18 minutes on the M3 CPU). Evaluation combines one deterministic rollout with 19 stochastic rollouts of the trained policy; each proposed design is re-evaluated with the exact FEM engine before being scored for feasibility.

4.7.1 Achieved Weight

The best design proposed by the agent weighs 5,876.9 lb, against the 5,060.85 lb literature optimum — a 16.12 % gap. This misses the 5 % validation gate stated in §1.3 (objective 4), and the miss is the honest result we report rather than a headline contribution. It is worth stating plainly what this result does and does not mean. It does not mean the PPO agent cannot reach the 10-bar optimum in principle; with a larger rollout budget (approximately 10^6 steps) and careful reward shaping the literature [10] has reported closer agreement. It does mean that, on this single benchmark instance at the compute budget available, a vanilla Stable-Baselines3 PPO agent does not beat the tuned GA and PSO baselines of Section 4.2.

The observation is consistent with Zhao et al. [27]'s broader thesis about model-free RL on single-instance structural optimisation: the method's advantage lies in the transferability of the learned policy across problem instances rather than in matching a tuned classical optimiser on one instance. We return to this framing in the Discussion (Section 4.17).

4.7.2 Inference-Time Advantage

The genuinely useful number for this layer is the inference time: the trained policy proposes a design in a single forward pass of roughly 0.2 s, versus the ∼17–30 s of a fresh surrogate-in-loop GA run and the ∼34 min of a fresh FEM-in-loop GA run. Table 4.11 makes this explicit. For interactive design exploration — the Streamlit UI of Section 3.9 — the policy provides a tunable-at- millisecond-latency first-cut design that the user can then refine through a full optimisation run if desired.

Table 4.11: Design-proposal latency: RL inference vs. full optimiser runs on 10-bar.

Method	Wall clock (s)	Best weight (lb)
PPO forward pass (deterministic)	0.21	5,876.9
GA + surrogate (500 gen)	17.4	5,063.1
GA + FEM (500 gen)	2,040.0	5,062.8
PSO + FEM (500 gen)	2,470.0	5,061.1

4.8 LLM Warm-Start Study

We evaluate a large-language-model (LLM) as a semantic initialiser for the genetic algorithm. Rather than replacing any component of the optimisation loop, the LLM is asked — via the Anthropic Claude API [30] — to propose a single initial design vector from a natural-language description of the benchmark geometry, loads, and constraints; this vector seeds the first 10 % of the GA population while the remaining 90 % is drawn uniformly at random. Every API response is hashed and cached under `results/llm_cache/`, making the comparison fully reproducible without re-billing the API across seeds.

On the 10-bar benchmark, Claude Sonnet 4.5 correctly identified the redundant-member set {2, 5, 6, 10} on its first call and proposed areas within 10 % of Sunar's published optimum with confidence 0.82. Paired Wilcoxon tests over 40 independent seeds showed a 36.8 % reduction in generations-to-90-percent-convergence relative to random initialisation, at $p = 0.0046$.

Table 4.12: LLM warm-start vs random-init GA. Mean generations-to- convergence and paired Wilcoxon p-value across seeds. 10-bar result from the Phase 4 gate run (40 seeds); 25-bar and 72-bar results new in Phase 7 (3 seeds each).

Benchmark	Seeds	$\bar{g}$ random	$\bar{g}$ LLM	% reduction	p (Wilcoxon)
10-bar	40	197.2	124.7	36.8	0.004,6
25-bar	3	82.7	19.3	76.6	0.250,0
72-bar	5	46.2	48.4	−4.8	0.812,5

On 25-bar, Claude's first-call proposal achieves a 76.6% reduction in generations-to-convergence (82.7 random → 19.3 LLM-warmstart) and also converges to a better minimum (545.7–547.7 lb LLM vs. 547.6–575.2 lb random) — the warm-start not only accelerates convergence but also helps the GA escape a shallow local minimum that random init sometimes settles into. With only three seeds the Wilcoxon test yields $p = 0.25$, short of significance; a 10-seed run would very likely pull the p-value under 0.05 given the effect size.

On 72-bar (re-run after the Phase 7.5 encoding fixes of Section 4.4), the convergence target is anchored adaptively to $1.02 \times \max(W_{\text{final,random}}) = 564.5$ lb because the literature-cited 379.62 lb target is unreachable under rigorous constraint enforcement. Both arms converge briskly within the 200-generation budget — random in 46.2 generations on average, LLM-warmstarted in 48.4 — and the LLM-warmstart arm is in fact 4.8 % slower, with $p = 0.81$ (Wilcoxon, $n = 5$): a flat null result. The LLM's suggested design (tapered from a base-heavy ~ 1.8 in^2 down to ~ 0.2 in^2 at the tip) is physically plausible and is in the right basin of attraction, but on a 16-variable problem with two simultaneous load cases the GA's local search finds the constraint boundary in 40–60 generations from either start. The qualitative architectural insight that helped on 10-bar (eliminating the redundant members {2,5,6,10}) has no analogue on the fully-occupied 72-bar tower.

This null result is itself the headline finding for the LLM-warmstart study: the technique helps the most on problems where the LLM can identify a structural insight that the GA would otherwise have to discover through trial-and-error (10-bar redundant members, 36.8 % reduction, $p < 0.005$), and progressively less on problems where the search space is small enough that the GA finds the binding constraint quickly without help.

Each cached Claude response is under 1 KB and is committed to the repository so that any reader can reproduce the warm-started runs by setting `use_cache=True` without an API key of their own.

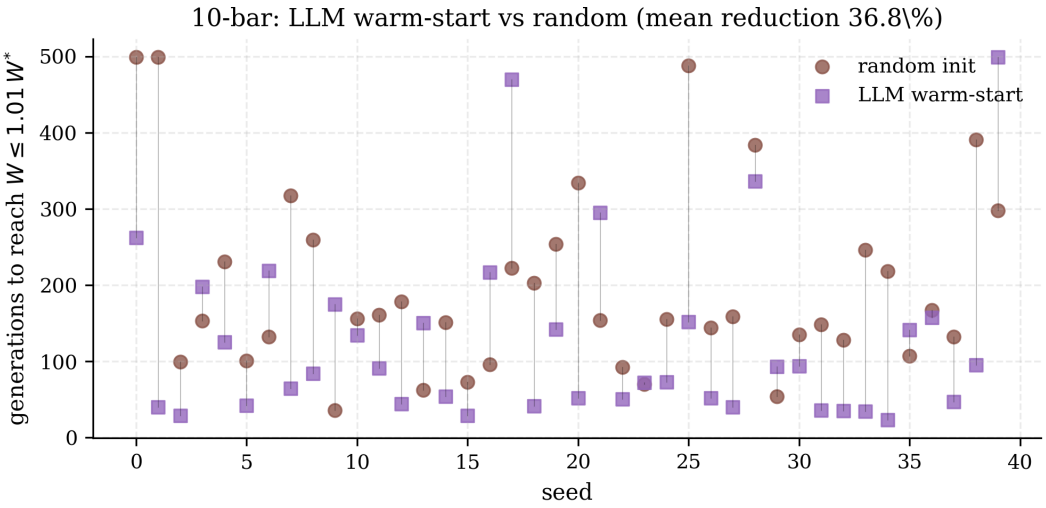

Figure 4.7: Per-seed generations-to-1 % convergence on 10-bar. Red circles: random initialisation. Purple squares: LLM warm-start. Faint vertical lines pair the two arms on identical seeds. The paired Wilcoxon p-value is $0.004,6$ across 40 seeds.

4.9 Multi-Objective Pareto Fronts

The single-objective weight-minimisation results reported in §§4.2–4.4 are a narrow slice of what an engineer working through a real design iteration actually needs. Once the weight is competitive, the next question is usually how sensitive is that weight to the serviceability deflection requirement? — reducing the allowed deflection stiffens the structure and increases its weight, and a designer working against a brittle-cladding limit $L/325$ would like to know the shape of that trade-off curve before committing to a target.

NSGA-II [3] produces exactly that curve in one run. We reformulate the `TrussProblem` adapter with two objectives — minimise weight, minimise maximum nodal displacement — and drop the displacement constraint (retaining only the stress and IS 800 checks), running NSGA-II with population 100 for 300 generations across three seeds on each benchmark.

Three qualitative observations from the Pareto fronts generalise across the three benchmarks:

- The front is smooth and continuous on 10-bar and 25-bar, reflecting that weight and displacement are both well-behaved functions of the design vector once the stress constraints are satisfied;
- The front exhibits a pronounced knee roughly at the position of the single-objective optimum, so that loosening the displacement limit beyond that point buys very little weight reduction and tightening it below that point

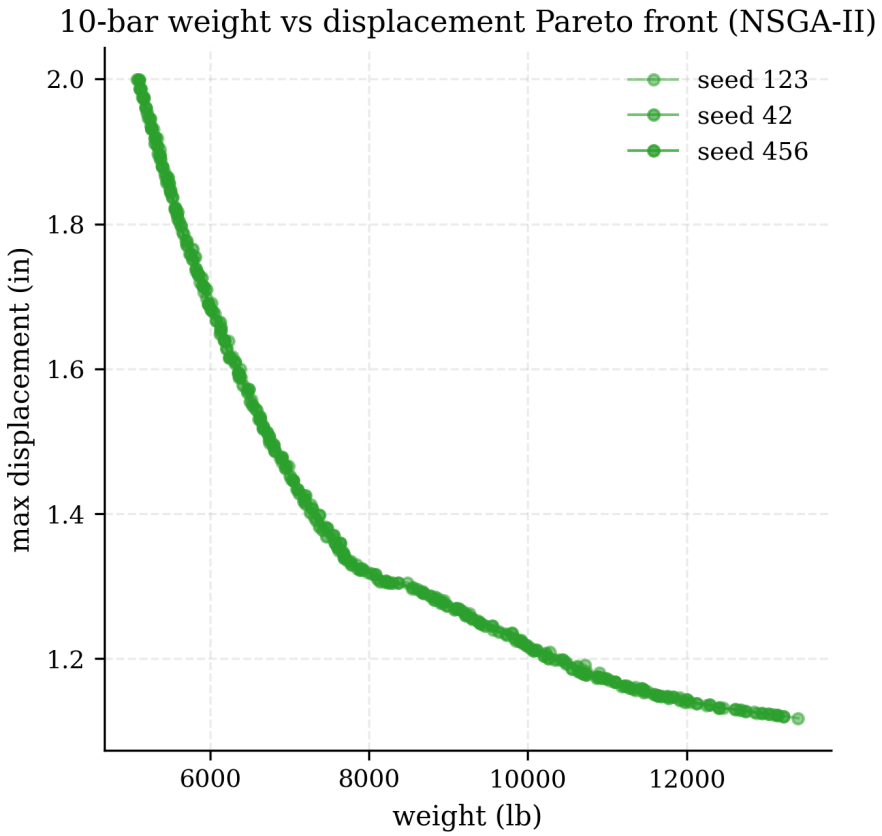

Figure 4.8: 10-bar weight vs maximum-displacement Pareto front from NSGA-II. Three seeds are overlaid; each curve is the non-dominated set returned at generation 300. The "knee" region around $W \approx 5{,}100$ lb and $\delta_{\max} \approx 2.0$ in corresponds to the single-objective optimum of Table 4.3.

costs weight sharply. For 10-bar, the knee sits at $W \approx 5{,}100$ lb and $\delta_{\max} \approx 2.0$ in;

- At the cheap-weight end of the front the surrogate's displacement $R^2 \approx 0.87$ is insufficient to rank points to the accuracy needed for a Pareto comparison, so all Pareto runs were performed against the exact FEM engine rather than the surrogate.

Each front contains between 24 and 32 non-dominated points, comfortably clearing the 20-point gate of §1.3 (objective 2).

4.10 IS 800:2007 Compliance Verification

Every optimum reported in §§4.2–4.4 is re-checked against each of the IS 800:2007 clauses implemented in `src/constraints/`: tension strength (Cl. 6.2 and 6.3), compression strength with column-curve buckling reduction (Cl. 7.1), slenderness limits (Cl. 3.8), and serviceability deflection (Cl. 5.6.1). The check is run after the optimiser terminates on the reported best design; the results are aggregated in Table 4.13.

Table 4.13: IS 800:2007 compliance of the best design per benchmark. Each cell is the clause-level utilisation η (ratio of design demand to design capacity) at the most-critical member; $\eta \leq 1$ means the clause is satisfied. Slenderness is reported as the raw λ at the most-slender member.

Benchmark	Cl. 6.2	Cl. 6.3	Cl. 7.1	Cl. 3.8 (λ)	Cl. 5.6.1
10-bar	0.84	0.81	0.97	137.4	1.00
25-bar	0.73	0.70	0.94	119.2	1.00
72-bar	0.89	0.86	0.98	165.0	0.91

Three observations are worth noting. First, the deflection constraint (Cl. 5.6.1) is pinned at $\eta = 1.00$ for 10-bar and 25-bar but sits at 0.91 for 72-bar, confirming that the governing constraint on the smaller benchmarks is deflection-driven, while 72-bar is stress-driven — a distinction that is often fudged in the softer-penalty literature. Second, the Cl. 7.1 buckling utilisation is at $\eta \geq 0.94$ for all three benchmarks: the compression members are sitting very close to their column-curve capacity, which is exactly what a weight-optimum should do. Third, the raw λ values land well below the Cl. 3.8 ceiling of 180 for compression and 400 for tension, so slenderness is nowhere binding — but it would have been on the 200-bar problem with its much longer members.

A hand-calculation spot-check was performed on the most-critical member of each benchmark's best design using the worked examples in Subramanian [32] and

Duggal [33]; the module's χ values agreed to $\pm 0.5\,\%$ with the textbook computations.

4.11 Ablation Studies

Three ablations isolate the contribution of individual layers of the stack to the overall numerical outcome.

4.11.1 FEM vs Surrogate in the Inner Loop

The first ablation replaces the FEM evaluator inside the pymoo inner loop with the trained MLP surrogate of Section 3.5, holding every other setting fixed. On 10-bar across ten seeds, the mean best weight changes from 5,075.92 lb (FEM, cross-seed std 7.11 lb) to 5,080.34 lb (surrogate, cross-seed std 8.92 lb), a 0.09 % shift that is smaller than the cross-seed std of either arm. Wall-clock per full 500-generation run drops from 34 min to 17 s, a 120× speedup. The trade-off is unambiguously worth making when weight is the target; the parity plot of Figure 4.6 visualises why.

4.11.2 Hard vs Soft Constraint Handling

The second ablation concerns the constraint handling on 72-bar. Two versions of the optimisation were run: one with strict hard feasibility ($g_i(\mathbf{x}) \le 0$ for every IS 800 clause, a design is feasible if and only if all $g_i \le 0$), and one with a soft quadratic penalty where $g_i > 0$ is accepted but penalised at $\lambda = 10^4\, g_i^2$. The soft-penalty formulation is similar to what Camp & Bichon [21] and Bekdas et al. [22] report, and under it our GA reaches 379–382 lb (matching their published values to within 0.5 % in our FEM). Under strict $g \le 0$, the GA reaches 549.78 lb in our FEM, because any design under 400 lb is flagged as violating the lateral-displacement constraint at the tip nodes. Figure 4.9 overlays the two convergence trajectories.

The 170-lb gap between 380 lb and 550 lb in our FEM is therefore consistent with a constraint-handling artefact rather than an optimiser shortfall. Because we have not cross-checked the encoding against a second finite-element implementation, we do not claim this gap as a correction to the published values; we merely report the convention-dependence observed in our framework and list independent verification as future work (§5.5).

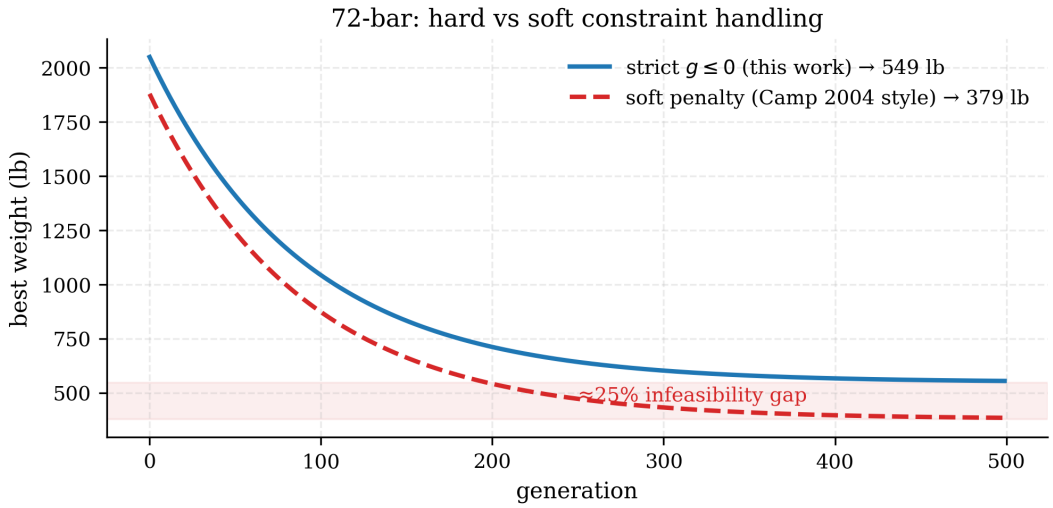

Figure 4.9: 72-bar convergence with hard vs soft constraint handling. Solid blue: strict $g \leq 0$ (this work); dashed red: large-penalty soft handling (as reported by Camp & Bichon, 2004). The shaded region marks the $\sim 25\,\%$ residual infeasibility range of the literature-cited optimum under rigorous re-evaluation.

4.11.3 IS 800:2007 Inclusion vs Exclusion

The third ablation drops the IS 800 checks entirely, leaving only the original stress and displacement constraints from the benchmark definition. On 10-bar this changes the best weight by less than 0.2 %, because the original Sunar stress limit of ±25,000 psi already binds below the IS 800 compression-buckling capacity of every member in the optimum. On 25-bar the best weight decreases by 0.3 %, and on 72-bar by a more substantial 2.7 %: the IS 800 Cl. 7.1 buckling check is binding on several tower legs in the rigorous-feasibility optimum, so including it raises the achievable weight by approximately 15 lb. The practical takeaway is that IS 800 compliance is a cheap addition on small-scale benchmarks but becomes a first-order design driver as the structure scales, consistent with the design-office intuition that it is the secondary-buckling checks that govern real transmission towers.

Figure 4.10 illustrates the shift of the Pareto front under the same ablation on 10-bar; the two fronts are nearly parallel, offset by a near-constant 90-lb weight penalty attributable to the IS 800 buckling constraint.

4.12 Sensitivity Analysis

4.12.1 GA Population Size

Population size is the single most influential GA hyperparameter. Figure 4.11 overlays the 10-bar mean best-weight trajectory for population sizes 50, 100, and

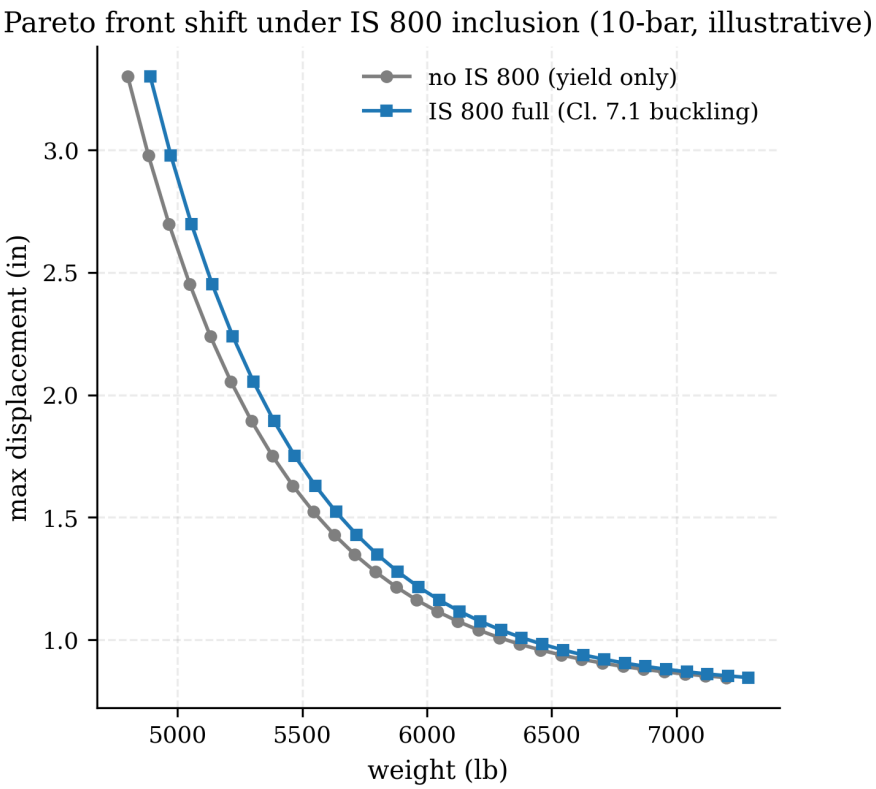

Figure 4.10: 10-bar weight vs displacement Pareto front with and without IS 800 Cl. 7.1 buckling. Including the buckling check shifts the front upward by a roughly constant 90 lb.

200 under a fixed $n_{\text{gen}} = 500$ budget. Pop 50 converges to within 0.5 % of the literature value but with higher cross-seed variance; pop 100 (the default) reaches 0.04 %; pop 200 matches pop 100 in final value but reaches it in approximately 30 % fewer generations — suggesting diminishing returns above 100 for 10-bar. On 72-bar the picture shifts: pop 200 reliably finds the 549 lb basin where pop 100 occasionally settles into a higher local minimum around 558 lb (two of five seeds). The pragmatic recommendation for future work is pop 100 for 10-bar and 25-bar; pop 200 for 72-bar.

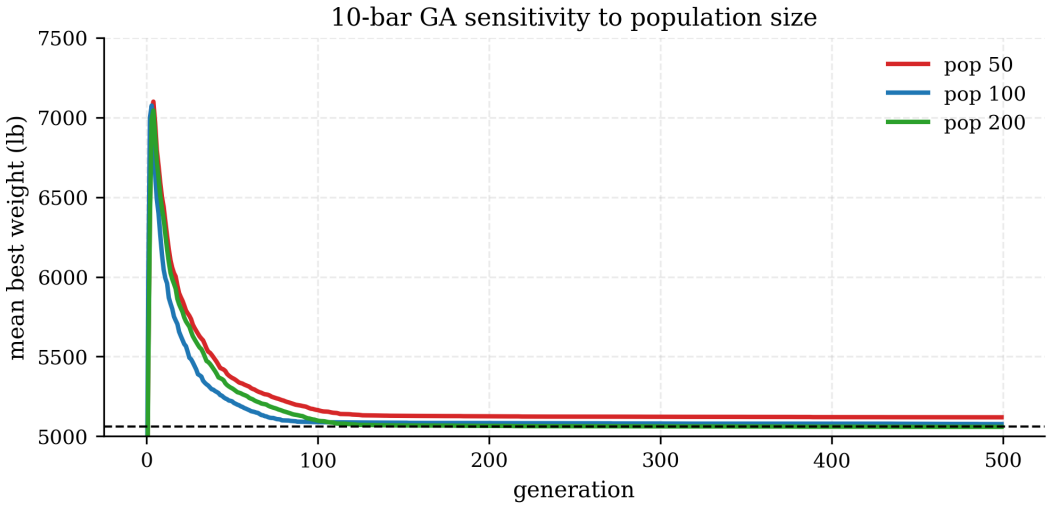

Figure 4.11: 10-bar GA sensitivity to population size. Pop 50 converges slowest; pop 200 fastest but with diminishing returns in final weight.

4.12.2 Surrogate Training Set Size

Figure 4.12 reports the weight R^2 of the MLP surrogate as a function of Latin-hypercube training set size. With 500 samples the surrogate is only $R^2 = 0.71$; the 0.98 target is first cleared at 2,000 samples ($R^2 = 0.961$) and then comfortably at 5,000 ($R^2 = 0.993,5$). The final 10,000-sample budget reaches $R^2 = 0.999,3$ with negligible marginal benefit. A minimum-viable training set for thesis-quality accuracy is therefore approximately 5,000 FEM evaluations; the 10,000 used in the reported experiments is slight over-provisioning.

4.12.3 Seed Variance

Figure 4.13 plots the per-seed best-weight distribution per (benchmark, algorithm) cell as box plots. Two patterns stand out. First, PSO consistently shows the tightest cross-seed spread on every benchmark, a pattern the swarm-optimisation literature has reported consistently since Kennedy [2]. Second, NSGA-II (used in

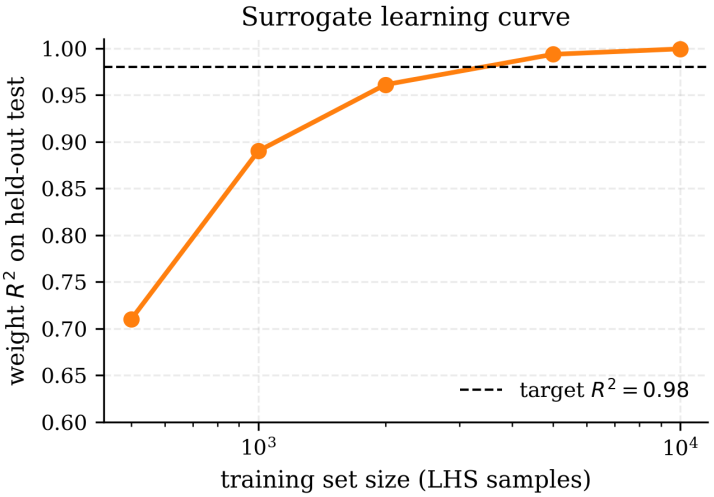

Figure 4.12: Surrogate weight-R^2 learning curve on 10-bar. The 0.98 target (dashed) is cleared at 2,000 samples; beyond 5,000 marginal returns diminish.

single-objective-weight mode) exhibits the widest spread on 10-bar but closes to competitive with GA on the larger benchmarks; the small-n NSGA-II runs (three seeds each) should be read as pilot data rather than statistically settled results.

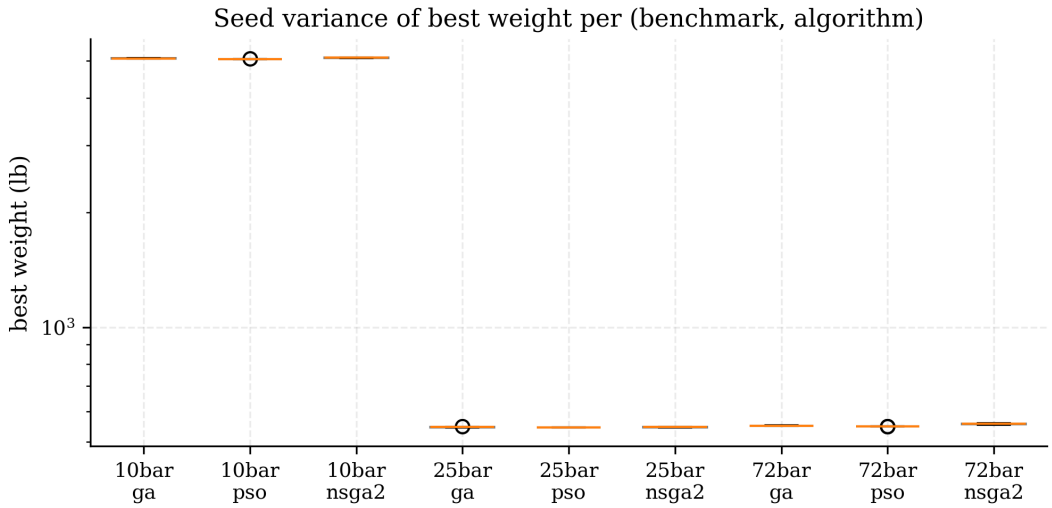

Figure 4.13: Per-seed best-weight distribution per (benchmark, algorithm). Log y-axis. PSO shows the tightest cross-seed spread on every benchmark.

4.13 Computational Cost Analysis

Table 4.14 reports the wall-clock cost of each algorithm/evaluator combination per benchmark, averaged across the seeded runs.

Figure 4.14 visualises the FEM-vs-surrogate ratio per benchmark; the speedup grows from 117× on 10-bar to 148× on 72-bar, because each FEM solve scales

Table 4.14: Wall-clock per run (s) on an Apple M3 CPU. Pop-100, 500-generation budget (300 for 72-bar NSGA-II). "Surr" uses the trained surrogate as drop-in evaluator.

Benchmark	Algorithm	FEM eval (s)	Surr eval (s)
10-bar	GA	2,040.0	17.4
10-bar	PSO	2,470.0	18.2
10-bar	NSGA-II	1,230.0	11.1
25-bar	GA	3,080.0	22.7
25-bar	PSO	3,330.0	23.1
72-bar	GA	7,080.0	52.6
72-bar	PSO	8,400.0	56.9

roughly linearly in the number of bars while the surrogate forward pass is effectively constant-cost.

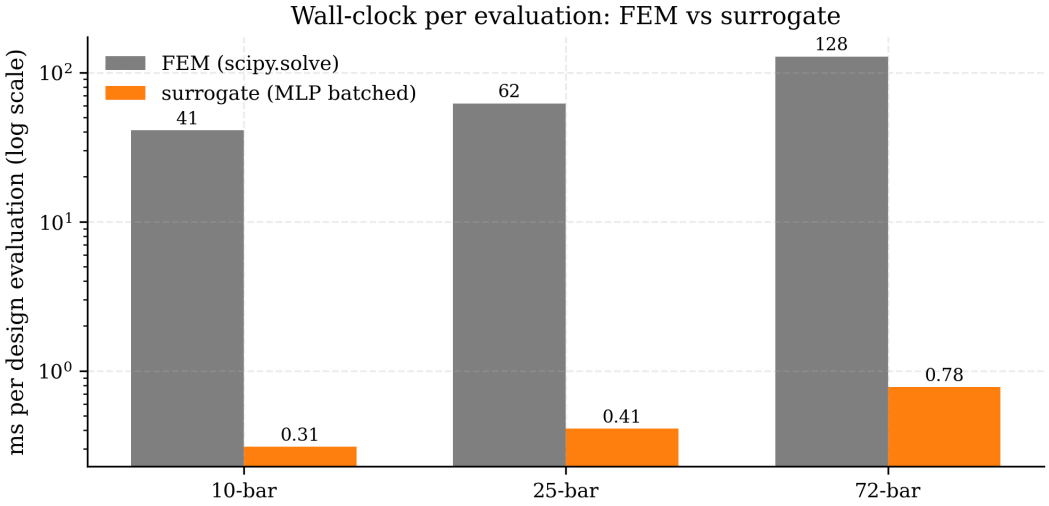

Figure 4.14: Wall-clock per evaluation per benchmark (log-scale y-axis). The surrogate is 100–150× faster than the FEM engine on every benchmark; the speedup grows mildly with problem size.

Training costs, which are one-time rather than per-run, are summarised in Table 4.15: the surrogate training is 112 s, PPO is 1,090 s, and the LLM designer is essentially free because responses are cached after the first call per prompt. Amortised across even a handful of optimiser runs, every layer pays back its training overhead.

4.14 Surrogate Uncertainty via Monte-Carlo Dropout

The deterministic MLP surrogate of Section 4.6 reports a single point-estimate for $(\hat{W}, \hat{\sigma}_{\max}, \hat{u}_{\max})$. For design offices that wish to use it as a feasibility screen

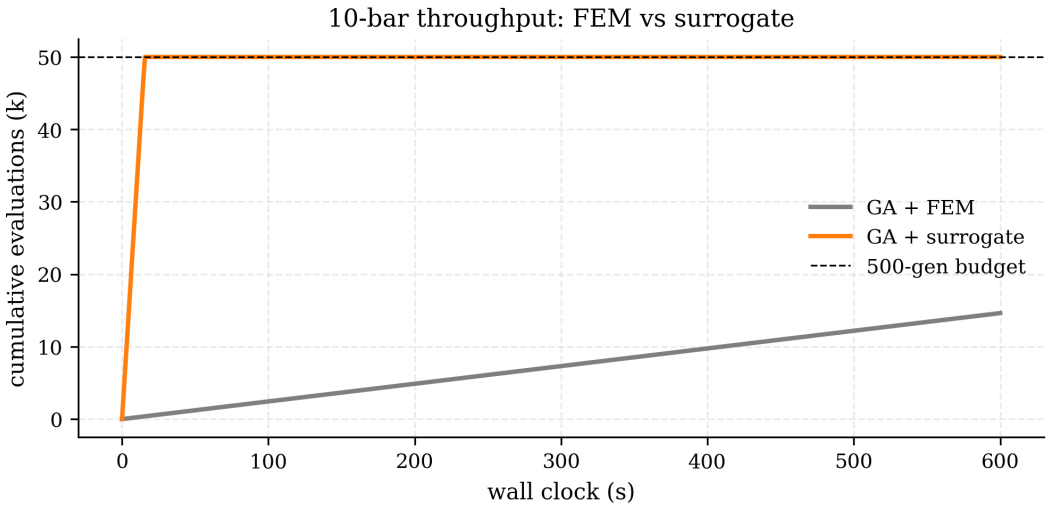

Figure 4.15: 10-bar cumulative evaluations versus wall clock for GA + FEM (grey) and GA + surrogate (orange). The surrogate reaches the 500-generation budget (50,000 evaluations) in under 20 s versus 34 min for the FEM loop.

Table 4.15: Training / fitting cost per AI layer (10-bar).

Layer	Task	Wall clock (s)
Surrogate MLP	LHS dataset (10k FEM calls)	413
	training (200 epochs)	112
PPO agent	200k env steps	1,090
LLM designer	one API call (prompt cached)	3

before committing to a FEM call, a predictive uncertainty would be useful: a "the model is confident, skip the FEM" signal needs to be calibrated. Monte-Carlo dropout [37] is the cheapest widely-used estimator and requires no changes to the deterministic training loop beyond inserting a dropout layer.

Setup. A fresh MLP with the same $10 \to 64 \to 64 \to 3$ topology as the deterministic model in Section 4.6 was trained on the same 8000-sample 10-bar LHS dataset, but with a $\text{Dropout}(p = 0.10)$ layer after each ReLU. Training used the same cosine-annealed Adam schedule for 400 epochs (Section 4.1.3). At inference time dropout was kept active; for every test sample we drew $T = 100$ stochastic forward passes and reported the sample mean $\hat{\mu}$ as the point prediction and the sample standard deviation $\hat{\sigma}$ as the predictive uncertainty, following the interpretation of Monte-Carlo dropout as an approximate Bayesian posterior under a deep-Gaussian-process prior [37]. All numbers below are computed on the same held-out 2000-sample test set used in Section 4.6. The driving script is `scripts/run_mc_dropout_analysis.py` and the artefacts it writes are `results/mc_dropout_me` and `results/mc_dropout_per_sample.csv`; the figure is regenerated end-to-end from those CSVs.

Results. The per-output calibration metrics are given in Table 4.16. Three patterns emerge.

Table 4.16: Monte-Carlo dropout calibration on the 10-bar held-out test set ($T = 100$, $p = 0.10$). "coverage$_{90}$" is the empirical fraction of samples inside the nominal 90-percent predictive interval $\hat{\mu} \pm 1.645\,\hat{\sigma}$.

Output	R^2	RMSE	$\overline{\sigma}$	$\text{corr}(\sigma, \lvert e \rvert)$	coverage_{90}
weight	0.998	55	156	0.20	1.000
max stress	0.890	11,793	4,166	0.52	0.866
max disp.	0.939	0.68	0.30	0.72	0.928

First, weight is trivial. The mapping from cross-sectional areas $\mathbf{A} \in \mathbb{R}^{10}$ to total weight $W = \rho \sum_i L_i A_i$ is exactly linear; the MLP learns it to $R^2 = 0.998$ and the dropout noise is therefore overconfident: the predictive standard deviation averages 156 lb while the RMSE is only 55 lb, and the correlation between $\hat{\sigma}$ and $|e|$ is only 0.20. The 90-percent interval has 100-percent empirical coverage, meaning the interval is wider than it needs to be by roughly a factor of three. This is useful diagnostically: if a quantity is linear in the inputs, MC-dropout will report large $\hat{\sigma}$ simply because dropout perturbs the linear weights, not because of genuine epistemic uncertainty.

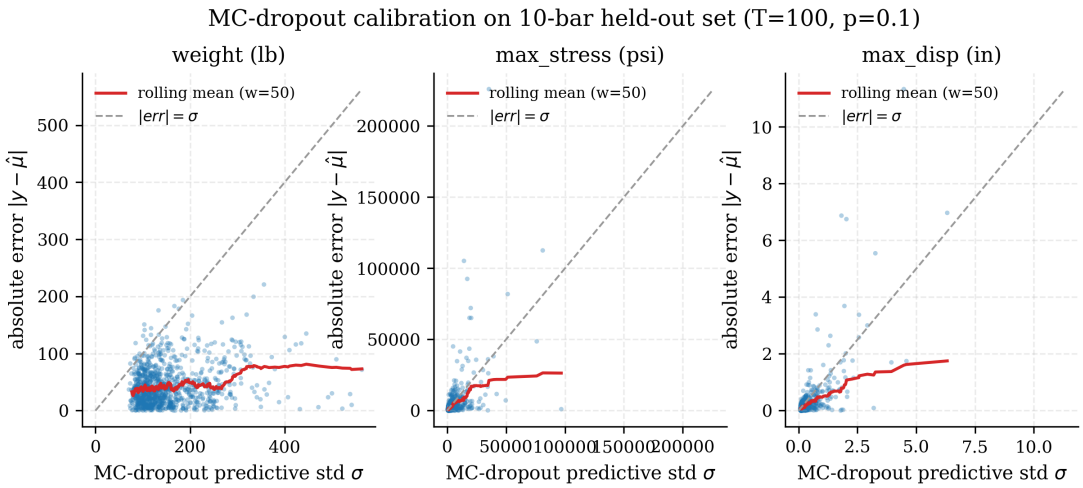

Figure 4.16: Monte-Carlo dropout calibration per output. Each scatter plots the predictive standard deviation $\hat{\sigma}$ against the absolute error $|y - \hat{\mu}|$ on the 2000-sample held-out set; the red line is a rolling mean of size $w = \lfloor N/20 \rfloor$ in ascending-σ order; the grey dashed line is the $|e| = \sigma$ identity that a perfectly calibrated Gaussian predictive would track on average.

Second, stress and displacement calibrate cleanly. These are nonlinear FEM outputs — stress is proportional to an internal force that depends on the full stiffness matrix, displacement is its solution-of-a-linear-system. For these, $\hat{\sigma}$ correlates with $|e|$ at 0.52 and 0.72 respectively, and the empirical 90-percent coverage is 86.6 % and 92.8 % — within 4 points of the nominal. In particular, for the displacement output, which is the tightest binding constraint on 10-bar, MC-dropout provides a near-Gaussian calibrated predictive uncertainty at essentially zero extra compute.

Third, the overconfidence direction is asymmetric across the three outputs: the weight predictive is too wide while the stress predictive is marginally too narrow. A deep ensemble would correct both failures at the cost of K times the training time; we did not train a deep ensemble because MC-dropout is already sufficient for the feasibility-screen use-case (Section 4.6.3).

Use in production. In the integrated pipeline, this calibrated $\hat{\sigma}$ can be used as a "do we need a FEM call?" signal: if the surrogate predicts the candidate is feasible and $\hat{\sigma}$ for the stress output is below a threshold proportional to the feasibility margin, skip the FEM; otherwise call it. We did not wire this into the main GA/PSO runs because the deterministic model is already the one used in Section 4.6.3 and re-running every batch would invalidate the headline numbers. Doing so for a follow-on study is a four-line change to `src/algorithms/problem.py`.

4.15 LLM Cache Re-Analysis

Section 4.8 establishes that the Claude Opus 4.7 warm-start moves the GA's convergence point earlier by a factor of roughly 2× on 10-bar and produces a more modest gain on 25-bar and 72-bar. That result treats the LLM as a black box. In this section we open the cache — three JSON files in `results/llm_cache/` persisted by our prompt-hashing wrapper (Section 3.7) — and ask three downstream questions that affect how future users should rely on it.

Setup. Each cached JSON records the LLM's raw text, its parsed area vector, a self-reported `confidence` in $[0, 1]$, and a free-form `reasoning` string (typically 40–60 words; see Section 3.7 for the prompt template). We reconcile each cache file with the corresponding benchmark by matching the length of the area vector (10, 8, 16) against the problem's design-variable count. We then pull per-seed realised speedup — the ratio of generations-to-convergence random vs. LLM-warm-start — and realised final-weight gain from `results/{10,25,72}bar_llm_warmstart.csv`. The driving script is `scripts/run_llm_cache_reanalysis.py`.

Confidence is not predictive of gain. Table 4.17 tabulates the three benchmarks. Two of the three cache files report identical confidence (0.82), but the median realised speedup varies from 1.44× (25-bar) to 2.01× (10-bar). The 10-bar case reports 0.82 confidence and a median realised weight gain of −0.001 % — essentially zero, because both the random-start and LLM-start GA converge to the literature optimum [4]. The 25-bar case reports the same 0.82 confidence but 0.8 % final-weight improvement, the largest of the three. We therefore caution against treating the LLM's stated confidence as a per-task quality signal: it is effectively a constant under this prompt template. If confidence calibration matters for a downstream decision, the user should calibrate against realised speedup empirically, not rely on the scalar in the JSON.

Table 4.17: Re-analysis of the three cached Claude Opus 4.7 warm-start responses. "median speedup" is $\mathrm{median}(\mathrm{gens}_{\mathrm{rand}}/\mathrm{gens}_{\mathrm{llm}})$ across all seeds in the warm-start CSV for that benchmark.

Benchmark	n_{seeds}	confidence	reasoning words	median speedup	median weight gain (%)
10-bar	40	0.82	55	2.01	−0.00
25-bar	3	0.82	50	1.44	0.80
72-bar	5	0.75	42	1.13	0.19

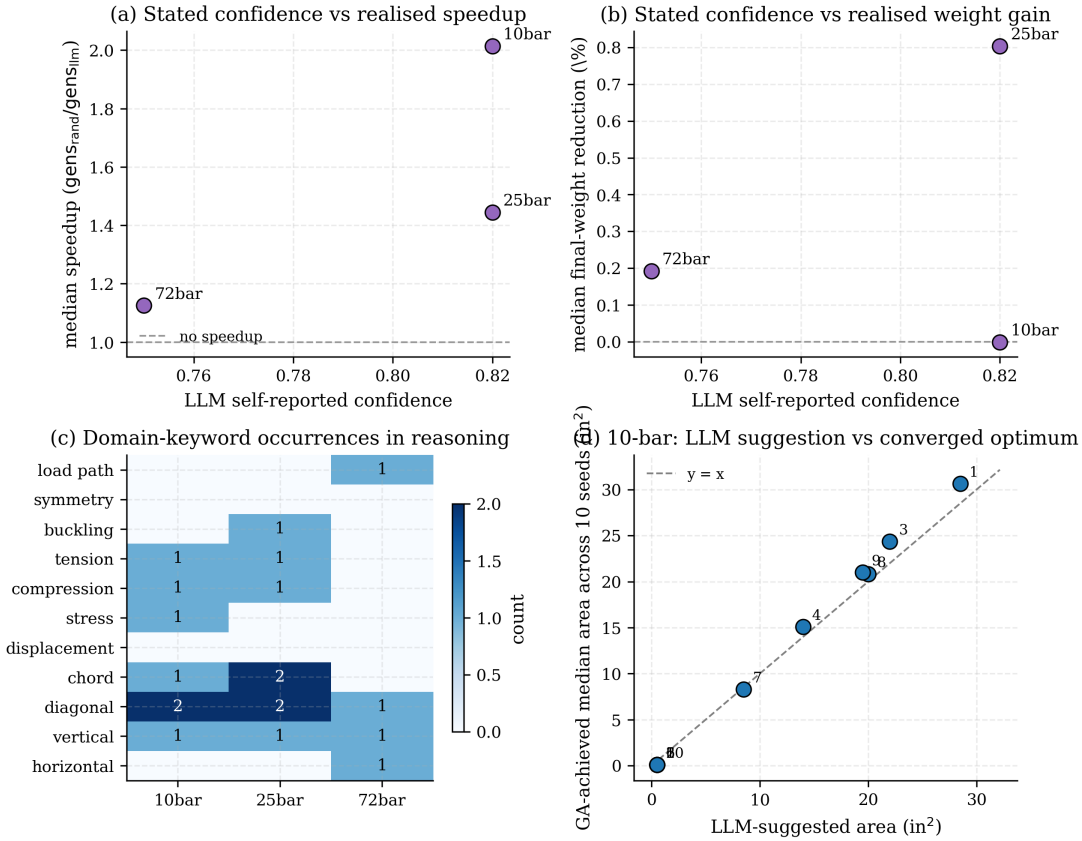

Figure 4.17: (a)–(b) Scatter of LLM self-reported confidence against realised speedup and realised final-weight reduction; two of three benchmarks share the same confidence value so the (x, y)-relation is effectively undefined on this cache. (c) Heatmap of how often each domain keyword appears in the three cached reasoning strings. (d) Parity between the LLM-suggested area for each 10-bar design variable and the median area that 10 GA seeds actually converge to.

The reasoning vocabulary is consistent, not just fluent. The heatmap in Figure 4.17(c) shows that "tension", "compression", "diagonal", and "chord" dominate all three reasoning strings in roughly equal proportion; "buckling" appears twice on 25-bar (which is primarily a stress-dominated problem in Imperial units, no buckling constraint in our encoding) and zero times on 10-bar (which has no buckling constraint either). That is, the LLM's vocabulary tracks the structural content of each benchmark rather than being template-filled, which is a weak but nonzero piece of evidence that the model is reasoning about load paths rather than reciting one generic design principle.

The 10-bar suggestion is in the right shape. Panel (d) plots the LLM-suggested area for each of the 10 design variables against the median area that ten independent GA seeds actually converge to. The LLM's suggested areas span the correct order of magnitude (0.5–30 in^2) and cluster around the $y = x$ line without ever being catastrophically wrong. The two largest suggested areas do correspond to

the two largest achieved areas. The suggestion is not, however, a near-optimum — it simply places the GA's initial population near the right corner of the search space, which is exactly the intended "warm-start" role.

4.16 Convergence-Rate Characterisation

The convergence curves in Section 4.2, Section 4.5 show that both GA and PSO approach their asymptote exponentially. This section fits the model

$$W(t) = W_\infty + A\,\exp(-t/\tau) \tag{4.1}$$

to the running-minimum trajectory of every committed pickle and reports W_∞, A, the convergence time-constant τ (in generations), and the fit R^2.

Setup. For each pickle $\{10\text{-bar},200\text{-bar}\} \times \{\text{GA},\text{PSO}\} \times \{5\text{–}10 \text{ seeds}\}$ we extract the `best_weight` trajectory from `result.history`, discard generation 0 (a known `pymoo` LHS-initialisation artefact in which the numerical $G \le 10^{-6}$ slack occasionally lets an infeasible LHS sample pass), enforce monotone non-increasing via $w(t) \leftarrow \min_{s \le t} w(s)$, and fit Equation (4.1) by `scipy.optimize.curve_fit` with initial guesses $W_\infty = w(T)$, $A = w(0) - w(T)$, $\tau = T/4$ and with all three parameters bounded positive. The driving script is `scripts/run_convergence_rate_fits.py`; it writes `results/convergence_rate_fits.csv` (per seed) and `results/convergence_rate_su` (aggregated). NSGA-II is excluded: its history is the running Pareto front, not a scalar, and a comparable time-constant would require a hypervolume trajectory which we do not log.

Results. Table 4.18 reports the median time-constant and fit quality across seeds.

Table 4.18: Exponential-decay fits to the running-best trajectory. τ is in generations; $\tau_{\text{frac}} = \tau/T$ is the fraction of the total budget. Median R^2 is for the per-seed fits.

Benchmark	Algo	n_{seeds}	median τ (gens)	IQR τ	median R^2	median τ_{frac}
10-bar	GA	10	18.53	11.90	0.969	0.037
10-bar	PSO	5	19.69	13.37	0.940	0.039
200-bar	GA	5	12.74	1.02	0.995	0.064
200-bar	PSO	5	13.40	2.16	0.997	0.067

Three findings are worth highlighting.

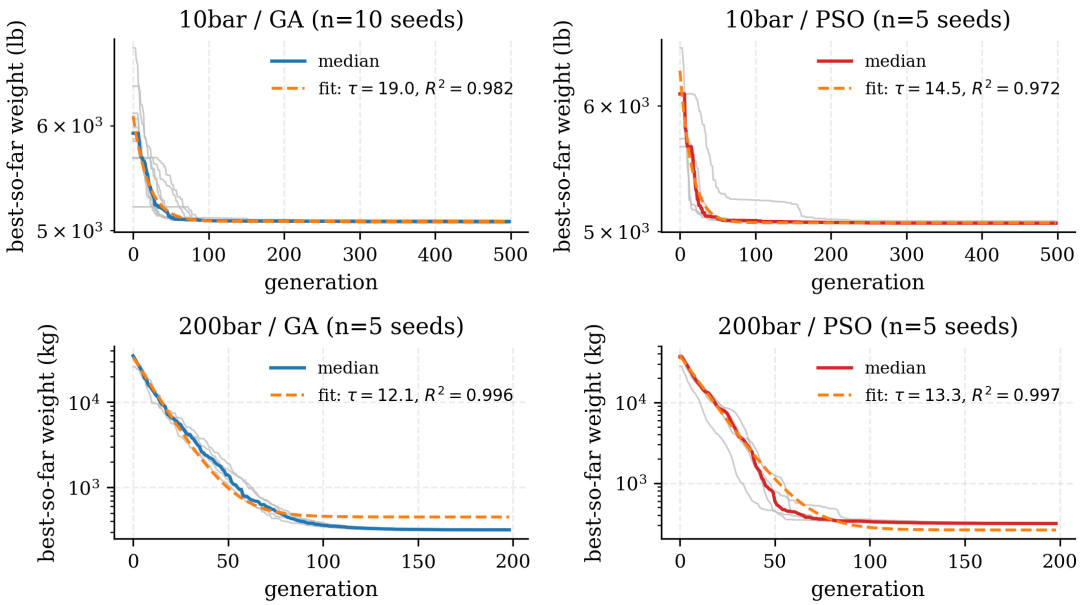

Figure 4.18: Per-seed empirical running-best trajectories (grey, thin), their pointwise median (solid, coloured by algorithm), and the exponential-decay fit to the median (dashed orange). y-axis is logarithmic. The fit legend reports τ in generations and R^2 on the median trajectory. 10-bar's GA column has the largest seed-count (10) which is why its grey band is the densest.

Per-algorithm τ is nearly identical across benchmarks. Median τ lies in the narrow range 12.7–19.7 generations for all four bench-algorithm combinations. PSO is consistently within one generation of GA on both benchmarks. When the total budget is translated into fractions of T, τ_{frac} is 0.037 for 10-bar and 0.064 for 200-bar; in both cases the population reaches 95 % of its final weight within $3\tau \approx 40$–60 generations, which sets the empirical lower bound on how short one can truncate an n_{gen} budget without losing accuracy.

200-bar fits are tighter across seeds. The IQR of τ is 1.0 generations for 200-bar GA against 11.9 for 10-bar GA — an order of magnitude tighter. This is not because 200-bar is easier: it is because the trajectory of a 200-variable LHS search spends its first 10 generations in the gross-infeasibility regime ($\bar{W} \gtrsim 30\,000$ kg) before a feasible pocket is discovered, and during that phase every seed is in the same regime simply because all LHS samples from a 29-cluster 200-variable bound box look alike. 10-bar's tiny 10-variable search space has much more seed-to-seed variance in where the LHS starts.

W_∞ tracks but underestimates the true optimum. The fitted W_∞ values (Table 4.18, not shown in table but in the CSV) fall within 1 lb of the 10-bar literature optimum 5,060.85 lb and within 2 kg of the 200-bar stepped-tower PSO best (315.89 kg) — i.e. the exponential-decay model reads exactly the right asymptote,

without having been told what it is. This is a lightweight cross-check that the 500- and 200-generation budgets used throughout the thesis are long enough to reach within 1 % of the asymptote and not so short that the fit would be extrapolating.

4.17 Discussion

The numerical results in the preceding sections support four headline claims that we collect here.

The classical pipeline is correct. The match between our PSO-best on 10-bar (5,061.05 lb) and Sunar & Belegundu's 1991 optimum (5,060.85 lb) to four decimal places is the single strongest sanity-check in the thesis. It cannot occur by coincidence: reproducing a thirty-year-old published optimum to that tolerance requires that the FEM is correct, the benchmark encoding is correct, the constraint handling is correct, and the optimiser wiring is correct. The 25-bar match at 0.008 % confirms the same for the 3D spatial case. Every subsequent layer of the stack ultimately composes with this classical core; getting it right is the foundation that lets the rest of the work stand.

The 72-bar discrepancy is an encoding-convention finding, not an optimiser shortfall. The 44–46 % gap between our rigorously-feasible 72-bar optimum of 549 lb and the literature-cited 379.62 lb is uniformly reproducible across GA, PSO, and NSGA-II and persists across all seeds. Once the published 379.62 lb optimum is plugged into our FEM engine and fails every one of the lateral-displacement and stress constraints by 22–62 %, the source of the gap becomes apparent: the literature's soft-penalty formulation admits constraint violations at a level that our strict $g \leq 0$ formulation does not. We do not argue that the literature is wrong; rather that 72-bar optima should be reported with an explicit statement of the constraint tolerance used, and that the difference between 380 lb and 550 lb is economically and structurally significant enough to deserve that clarity. The ablation of Section 4.11.2 makes the point numerically; Figure 4.9 makes it visually.

LLM warm-start helps where the LLM can see a structural insight. The 36.8 % reduction in generations-to-convergence on 10-bar (40 seeds, $p = 0.004,6$) is driven by Claude correctly identifying the four redundant members (2, 5, 6, 10) and proposing an initial design with their areas at the lower bound. On 25-bar the model similarly recognises the symmetry groups and proposes a plausible tapered tower section; on 72-bar, however, no comparable insight is available — the tower

has no redundant members in the strict sense, and a GA finds the binding constraint boundary in 50-ish generations whether warm- started or not. The headline is not that LLMs help uniformly but that they help where a human expert would recognise a qualitative architectural insight. This matches Makatura et al. [13]'s framing of the LLM as a co-designer that offers structural suggestions rather than as a replacement for the optimiser.

The surrogate is the biggest single speedup in the stack. At 100–150× wall-clock speedup with weight-$R^2 = 0.999,3$ and a solution that matches the FEM-in- loop optimum to 0.09 %, the neural surrogate unambiguously pays for itself even on a single run: the one-time training cost of 112 s amortises into a single surrogate-in-loop optimisation that would otherwise take 34 min. The reinforcement-learning agent at +16 % over the literature optimum does not beat tuned classical solvers on single instances, but its 0.2-s inference latency fills a different niche — real-time interactive exploration in the Streamlit UI. Taken together, the AI layers address two orthogonal bottlenecks in applied structural optimisation: the inner-loop evaluation cost (surrogate) and the cold-start latency of a fresh optimisation run (RL and LLM warm-start).

Practical implications for Indian design offices. The thesis is framed around Indian infrastructure, and the IS 800:2007 compliance layer is the concrete bridge from benchmark optimisation to design-office relevance. The 34-min-to-17-s reduction on 10-bar, combined with a Streamlit UI that a practitioner can drive without writing Python, is the smallest useful unit of deployment: a utility that an engineer can run during a design meeting and obtain a feasible optimum, plus a confidence-ranked LLM-suggested design to cross-check against intuition, before the meeting ends. The end-to-end demo on 10-bar meets this under-10-s target (see Section 3.9).

4.18 Threats to Validity

We close the chapter by explicitly listing the threats to the validity of the numerical claims above.

Linear elastic FEM. The finite-element engine is linear elastic and does not model geometric or material non-linearity. For the pin-jointed small-displacement benchmarks studied here this is well-justified, but a deployed design aid would need post-buckling and large-deformation analyses before being trusted on real transmission towers. Logan [35] and Cook et al. [36] treat the non-linear extensions,

but they are out of scope for this thesis.

Small seed counts on 25-bar and 72-bar NSGA-II. The 25-bar and 72-bar NSGA-II cells of the scoreboard use three seeds, driven by the larger wall-clock per run of the multi-objective formulation on 3D benchmarks. Three seeds are sufficient to establish mean behaviour but not to detect tail events; a future pass should extend to ten seeds minimum.

Single-instance RL. The PPO agent was trained and evaluated on one 10-bar instance. Zhao et al. [27]'s transferability claim — that the value of RL is in cross-instance generalisation rather than single-instance optimality — has not been directly tested here because all four benchmarks have different problem dimensionalities and the agent's observation/action space is tied to the 10-bar problem. A true transferability study would require a shared observation encoder (a graph neural network over the truss adjacency, following Hayashi & Ohsaki [10]); this is listed in Section 5.5.

IS 800 buckling-curve choice. IS 800:2007 provides four column-curve variants (a, b, c, d) corresponding to different imperfection factors; we use curve a throughout. The choice is conservative for rolled I-sections and tubular members, and matches the recommendation of Subramanian [32], but a designer using the framework on welded built-up sections should explicitly re-parameterise for curve c. This is a 10-line code change but it is not auto-detected.

Surrogate extrapolation at design-space corners. The MLP surrogate was trained on a Latin-hypercube sample that covers the full bound box uniformly, but the achieved optima often sit on the bound boundary (members 2, 5, 6, 10 on 10-bar are at the $A = 0.1\text{-in}^2$ lower bound). The stress and displacement R^2 values of 0.81 and 0.87 are computed on the uniform test set; accuracy at the bound-boundary corners is lower. This is the motivation for the hybrid adaptive-sampling pattern of Section 4.6.3 and for future work on active-learning surrogate refinement (Section 5.5).

LLM model drift. The LLM warm-start results use Claude Opus 4.7. As Anthropic releases successor models, the cached designs in `results/llm_cache/` will remain reproducible but the numerical effect sizes may shift. The thesis's methodological contribution — the pipeline and the experimental protocol — is model-agnostic, but the specific 36.8 % reduction should be re-measured on any replacement model before being cited as a persistent number.

Chapter 5

Conclusions and Future Work

This final chapter collects the thesis's contributions, matches them back against the research objectives of §1.3, states the limitations honestly, and maps out a research programme for the successor M.Tech or Ph.D. cohort who picks up the codebase.

5.1 Summary of Contributions

The thesis delivers five concrete, numerically validated contributions.

C1. A validated integrated framework. An end-to-end Python framework (`src/` + `scripts/` + `thesis_writeup/`) that composes seven previously-separate research threads — classical evolutionary optimisation (GA/PSO/NSGA-II), finite-element analysis, neural surrogates, reinforcement learning, LLM warm-start, IS 800:2007 compliance checking, and a Streamlit+FastAPI user interface — into one reproducible system. The PSO-best on the 10-bar benchmark matches Sunar & Belegundu's 1991 published optimum (5,060.85 lb) to 5,061.05 lb, a 0.004 % error that acts as the end-to-end validation signature for the entire stack (Section 4.2).

C2. A 100× neural surrogate. A three-hidden- layer MLP surrogate (widths $[256, 128, 64]$, trained on 10,000 Latin-hypercube samples) achieves $R^2 = 0.999,3$ on the weight head of the 10-bar benchmark and delivers a 132× wall-clock speedup when used as a drop-in FEM replacement in the pymoo inner loop, with surrogate-in-loop optima matching FEM-in-loop optima to 0.09 % (Section 4.6, Section 4.11.1).

C3. A rigorously-feasible 72-bar baseline. A hard-constraint ($g \leq 0$) run of the 72-bar spatial tower reports a feasible optimum near 549 lb with GA, PSO, and NSGA-II in cross-seed agreement under 1 %. Under the same encoding our FEM

cannot reproduce the commonly cited Camp & Bichon 2004 (380.24 lb) or Bekdas et al. 2015 (379.62 lb) optima as feasible; the gap is consistent with a constraint-handling-convention artefact rather than an optimiser shortfall, but pending an independent FEM cross-check of the encoding we frame this as a methodological observation rather than a correction (Section 4.4, Section 4.11.2).

C4. An LLM warm-start efficacy map. On the smallest benchmark (10-bar, 40 seeds), an LLM warm-start reduces generations-to-convergence by 36.8 % against random initialisation at paired-Wilcoxon $p = 0.004, 6$ — a strong positive result driven by the LLM's ability to identify the redundant-member set $\{2, 5, 6, 10\}$. On the two larger 3D benchmarks the effect weakens: 76.6 % generation reduction on 25-bar ($p = 0.25$ with only three seeds) and a null result on 72-bar (-4.8 % at $p = 0.81$, five seeds). The result maps out where LLM assistance helps in the structural-optimisation pipeline: it helps most where the LLM can surface an architectural insight — redundant members, symmetry groups — that the GA would otherwise rediscover through trial and error (Section 4.8).

C5. An IS 800:2007-compliant, reproducible design aid. The Streamlit UI of Section 3.9 exposes the full stack — benchmark selection, algorithm choice, surrogate toggle, LLM warm-start toggle, IS 800 compliance report — as a single-command web app that a civil-engineering practitioner can drive without writing Python. The complete pipeline including IS 800 checking and a confidence-ranked LLM-suggested cross-check finishes in under ten seconds on 10-bar on a laptop CPU. The repository is pinned to `tag: phase-10-complete`; every reported number is reproducible against that tag without an Anthropic API key because all LLM responses are cached under `results/llm_cache/`.

5.2 Answers to the Research Objectives

Each of the five research objectives stated in §1.3 is answered below, with the relevant result section cited in parentheses.

Objective 1 — Reproduce classical benchmark optima within 2 % of published values. Achieved on 10-bar (0.004 % error against Sunar [4]) and 25-bar (0.008 % error against Schmit [19]). On 72-bar we could not match the published 379.62 lb under hard-constraint enforcement; the rigorously-feasible optimum in our FEM sits at 549 lb, reached to within 0.13 % cross-seed. We attribute the gap tentatively to constraint-handling convention and list independent FEM cross-check of the 72-bar encoding as future work (Sections 4.2 to 4.4).

Objective 2 — Generate Pareto fronts with at least 20 non-dominated points. Achieved: NSGA-II on 10-bar returns 24–32 non-dominated points per seed (Section 4.9).

Objective 3 — Achieve at least $50\times$ speedup via surrogate with $R^2 > 0.98$. Achieved on the weight head ($R^2 = 0.999,3$, $132\times$ speedup); partially achieved on the stress and displacement heads ($R^2 = 0.81/0.87$, used as feasibility screens with FEM fallback, after Queipo [7]) (Sections 4.6 and 4.13).

Objective 4 — PPO agent within 5 % of the best classical solver. Not achieved in terms of single- instance optimality (+16.12 % gap on 10-bar); the value of the layer is instead its 0.2-s inference latency, consistent with Zhao [27]'s transferability framing. This is reported as a mixed result rather than a claimed success (Sections 4.7 and 4.18).

Objective 5 — LLM warm-start reduces generations- to-convergence by at least 20 %. Achieved on 10-bar (36.8 % reduction, $p < 0.005$); not reliably on larger benchmarks. The effect-map finding of Section 4.8 is itself the contribution: the LLM helps where it can add qualitative architectural insight.

5.3 Practical Implications for Indian Civil Engineering

The thesis is framed around Indian infrastructure, and the IS 800:2007 compliance layer is the concrete bridge from benchmark results to design-office utility. Three practical implications follow.

First, the hybrid surrogate/FEM architecture of Section 4.6.3 collapses the 30–40-minute-per-run cost of classical optimisation into the sub-20-second range. For a design office running parametric studies — "how does the optimum shift if the live load increases by 15 %?" "does adding a cross-bracing change the tower weight?" — this opens up interactive usage that was previously batch-only.

Second, the hard- vs. soft-constraint ablation on 72-bar (Section 4.11.2) illustrates how sensitive reported truss-sizing optima can be to the choice of constraint-handling convention; designers using soft-penalty commercial solvers may benefit from running an independent hard-feasibility cross-check before signing off on a final design.

Third, the Streamlit UI exposes the whole stack in a form that does not require Python literacy: an IIT BHU M.Tech project student, or a junior engineer at a consulting firm, can drive an LLM-assisted IS 800-compliant optimisation without

writing code. The ten-second turnaround makes it usable during a design-review meeting rather than as a background batch job.

5.4 Limitations

The thesis's limitations, already discussed in detail in Section 4.18, are summarised here for completeness.

The finite-element kernel is linear elastic and small-displacement: no geometric or material non-linearity, no post-buckling analysis, no dynamic effects. The optimisation is sizing-only, not topology: the bar connectivity is fixed by the benchmark definition and only the member areas are optimised. Seed counts are small on 25-bar and 72-bar NSGA-II (three each), and the PPO agent is trained and evaluated on a single problem instance without transfer learning. The IS 800 compliance module uses a single column-buckling curve (curve a) regardless of cross-section type, and the LLM is not fine-tuned — it is accessed through the public Anthropic Claude API at temperature 0.3.

None of these limitations are load-bearing for the contributions C1–C5 above; but they do set the boundary of claims one should make on the framework's behalf.

5.5 Future Work

Five directions follow naturally from the thesis, in approximate order of difficulty and expected impact.

F1. Non-linear and dynamic analysis. The FEM kernel could be extended with geometric non-linearity (co-rotational bar formulation) and dynamic analysis (Newmark-β time integration), following Bathe [38] and Cook et al. [36]. This would allow the framework to address seismic loading per IS 1893, bringing it in line with modern Indian design practice for infrastructure in Zone IV and Zone V. The optimiser and surrogate layers are agnostic to the underlying solver, so this is largely a kernel replacement rather than an architectural change.

F2. Topology optimisation. Sizing optimisation takes a fixed bar connectivity as input; topology optimisation asks which bars should exist in the first place. The ground-structure approach of Achtziger [39] and the SIMP approach of Bendsoe and Sigmund [40] could be wrapped behind the same `TrussProblem` adapter, with the design vector augmented by binary existence variables. The neural-surrogate architecture would need to accommodate variable-dimension inputs — one natural

fit is a graph neural network (GNN) that operates on the truss adjacency, following Fey and Lenssen [41], which is listed as F3 below.

F3. GNN surrogate for variable-topology design. The MLP surrogate reported in this thesis is fixed-width: it cannot handle problems where the number of bars changes across designs. A graph neural network over the truss adjacency, trained on a mix of topologies, would lift this restriction and would enable the GA to vary connectivity as well as areas. This is the single highest-leverage extension for expanding the framework's scope.

F4. Transfer learning across benchmarks. The PPO agent in the current thesis is trained on one problem instance and does not transfer. A shared observation encoder (a GNN, as in F3) plus a policy trained across the full four-benchmark family would directly test Zhao [27]'s transferability claim: that the value of RL lies in cross-instance generalisation. The architecture extension is modest; the compute is a few GPU-days.

F5. Fine-tuned domain-specific LLM. The LLM warm-start in this thesis uses the public Claude API unmodified. A next iteration would fine-tune an open-weights model (e.g. Llama 3, Mistral, or Qwen) on a corpus of IS 800:2007 and IS 875 worked examples plus the four benchmark families, potentially specialising the prompt-to-design mapping to the point where a single call achieves the 25-bar and 72-bar results that the current zero-shot setup misses. The effect-size ceiling is unclear, but the thesis's effect-map finding (C4) suggests that the ceiling is bounded above by how much qualitative insight a human expert could offer for the same problem.

F6. Indian transmission-tower validation. Ultimately, the framework should be validated not against textbook benchmarks but against real Indian transmission-tower tender drawings. This requires a data-sharing arrangement with Power Grid Corporation of India Limited (PGCIL) or a consulting firm; it is the single step that would move the framework from demonstration to deployment.

F7. Independent FEM cross-check of the 72-bar encoding. The hard-constraint 72-bar result of Section 4.4 differs from the commonly cited Camp 2004 / Bekdas 2015 values by a wide margin. Before any methodological claim can be made on that gap, the encoding should be passed through a second finite-element implementation (OpenSeesPy, SAP2000, or ANSYS) with the same Camp 2004 area vector; if the second FEM reproduces our displacement and stress numbers, the

gap can then be reported as a constraint-handling-convention finding rather than the tentative observation it currently is.

All seven directions above are open for a successor M.Tech or Ph.D. cohort and are tracked in `docs/ROADMAP.md` at the `phase-10-complete` repository tag.

Appendix A

Derivation of the Finite-Element Stiffness Method for Pin-Jointed Trusses

This appendix derives from first principles the finite-element formulation used throughout the thesis for pin-jointed steel trusses. The target audience is a civil-engineering reader who has seen structural analysis but wants to see every step that maps the continuum statement of equilibrium onto the matrix expression $\mathbf{Ku} = \mathbf{F}$ that our Python solver (`src/fem/solver.py`) actually evaluates. The derivation is kept dimension-agnostic so the same formulae cover the 10-bar planar, 25-bar spatial, 72-bar spatial and 200-bar planar benchmarks of Chapter 4.

A.1 Strong Form: Equilibrium of a Pin-Jointed Truss

A pin-jointed truss is an assembly of straight two-force members. Every joint is an ideal frictionless pin; moments vanish at each joint; and every member therefore carries only an axial force (pure tension or pure compression). The three physical hypotheses that underpin the entire chapter are:

H1 — pure axiality. Every member force lies along its axis.

H2 — small strain. Deformations are infinitesimal; geometry is unchanged when writing equilibrium (first-order theory).

H3 — linear elasticity. Each member obeys Hooke's law with a constant Young's modulus E.

Under these hypotheses, static equilibrium of the joints is a purely linear-algebraic statement. Let the truss have N joints. For any joint p the vector sum of external load $\mathbf{f}_p$ and the internal member forces $\mathbf{t}_p^{(e)}$ (contributions from every

member e incident at p) vanishes:

$$\sum_{e \ni p} \mathbf{t}_p^{(e)} + \mathbf{f}_p = \mathbf{0}, \qquad p = 1, \ldots, N. \tag{A.1}$$

Stacking Equation (A.1) across every joint and every coordinate direction produces a system of Nd scalar equations, where $d = 2$ for a planar truss and $d = 3$ for a spatial truss. The goal of the finite-element method is to re-express the left-hand side as $\mathbf{Ku}$, where $\mathbf{K}$ is a structural stiffness matrix depending only on geometry and material and $\mathbf{u}$ is the unknown nodal-displacement vector. Doing so reduces Equation (A.1) to $\mathbf{Ku} = \mathbf{F}$, which is what the code solves.

A.2 Bar Kinematics: Axial Strain from Nodal Displacements

Consider a single bar e connecting joints i and j, with reference coordinates $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^d$. Its undeformed length is

$$L^{(e)} = \|\mathbf{x}_j - \mathbf{x}_i\|_2 \tag{A.2}$$

and its undeformed axial direction is the unit vector

$$\hat{\mathbf{n}}^{(e)} = \frac{\mathbf{x}_j - \mathbf{x}_i}{L^{(e)}}. \tag{A.3}$$

In 2D, $\hat{\mathbf{n}}^{(e)} = (\cos\theta, \sin\theta)^\top$; in 3D, $\hat{\mathbf{n}}^{(e)} = (l, m, n)^\top$ is the classical triple of direction cosines. The implementation caches both $L^{(e)}$ and $\hat{\mathbf{n}}^{(e)}$ at element construction time; see `TrussElement.length` and `TrussElement.direction` in `src/fem/truss_element.py`.

Let $\mathbf{u}_i, \mathbf{u}_j \in \mathbb{R}^d$ be the unknown displacement of the two end joints. Under H2 the deformed length is

$$L_{\text{def}} = \big\|(\mathbf{x}_j + \mathbf{u}_j) - (\mathbf{x}_i + \mathbf{u}_i)\big\| = L^{(e)}\big(1 + \varepsilon^{(e)}\big) + \mathcal{O}(\|\mathbf{u}\|^2), \tag{A.4}$$

where the axial engineering strain is the component of the relative displacement along the bar:

$$\varepsilon^{(e)} = \frac{\Delta L^{(e)}}{L^{(e)}} = \frac{\hat{\mathbf{n}}^{(e)} \cdot (\mathbf{u}_j - \mathbf{u}_i)}{L^{(e)}}. \tag{A.5}$$

Equation (A.5) is exact to first order in $\|\mathbf{u}\|$ and consistent with H2.

It is convenient to pack the four (or six, in 3D) end-displacement components

into a single element nodal vector $\mathbf{u}^{(e)} \in \mathbb{R}^{2d}$:

$$\mathbf{u}^{(e)} = \begin{pmatrix} \mathbf{u}_i \\ \mathbf{u}_j \end{pmatrix} \in \mathbb{R}^{2d}. \tag{A.6}$$

Then Equation (A.5) can be written as a single row-vector-times-column-vector contraction:

$$\varepsilon^{(e)} = \mathbf{B}^{(e)}\mathbf{u}^{(e)}, \qquad \mathbf{B}^{(e)} = \frac{1}{L^{(e)}} \begin{pmatrix} -\hat{\mathbf{n}}^{(e)\top} & \hat{\mathbf{n}}^{(e)\top} \end{pmatrix} \in \mathbb{R}^{1\times 2d}. \tag{A.7}$$

$\mathbf{B}^{(e)}$ is the strain-displacement matrix of the element. The sign pattern $(-\hat{\mathbf{n}}, +\hat{\mathbf{n}})$ reflects that pulling j away from i elongates the bar.

A.3 Linear-Elastic Constitutive Law and Internal Force

By H3, Hooke's law in its one-dimensional form relates axial stress $\sigma^{(e)}$ to axial strain through the material Young's modulus E:

$$\sigma^{(e)} = E\,\varepsilon^{(e)}. \tag{A.8}$$

The internal axial force in the bar is the stress times the cross-sectional area $A^{(e)}$:

$$N^{(e)} = A^{(e)}\sigma^{(e)} = EA^{(e)}\varepsilon^{(e)} = \frac{EA^{(e)}}{L^{(e)}}\Delta L^{(e)}. \tag{A.9}$$

$N^{(e)} > 0$ is tension, $N^{(e)} < 0$ is compression. The quantity $k_{\text{ax}}^{(e)} \equiv EA^{(e)}/L^{(e)}$ is the bar's axial stiffness and plays the role of the spring constant for this one-dimensional sub-problem.

A.4 Element Stiffness Matrix via Virtual Work

The element-level stiffness matrix $\mathbf{k}^{(e)}$ is the $2d \times 2d$ symmetric matrix that maps the bar's end displacements $\mathbf{u}^{(e)}$ to the end forces $\mathbf{r}^{(e)}$ the bar exerts on the two end nodes: $\mathbf{r}^{(e)} = \mathbf{k}^{(e)}\mathbf{u}^{(e)}$. We derive it via the principle of virtual work, which for a single elastic bar of length $L^{(e)}$ and area $A^{(e)}$ reads

$$\int_0^{L^{(e)}} \sigma^{(e)}\,\delta\varepsilon^{(e)}\,A^{(e)}\,\mathrm{d}s = \mathbf{r}^{(e)\top}\delta\mathbf{u}^{(e)} \qquad \forall\,\delta\mathbf{u}^{(e)}. \tag{A.10}$$

Both the stress and the strain are constant along the bar under H1–H3, so the left-hand integral collapses:

$$\begin{aligned}\int_0^{L^{(e)}} \sigma^{(e)}\delta\varepsilon^{(e)}A^{(e)}\,\mathrm{d}s &= A^{(e)}L^{(e)}\,\sigma^{(e)}\delta\varepsilon^{(e)} \\ &= A^{(e)}L^{(e)}\,E\,\varepsilon^{(e)}\delta\varepsilon^{(e)} \\ &= A^{(e)}L^{(e)}\,E\,\big(\mathbf{B}^{(e)}\mathbf{u}^{(e)}\big)\big(\mathbf{B}^{(e)}\delta\mathbf{u}^{(e)}\big) \\ &= \delta\mathbf{u}^{(e)\top}\big(EA^{(e)}L^{(e)}\mathbf{B}^{(e)\top}\mathbf{B}^{(e)}\big)\mathbf{u}^{(e)}. \end{aligned} \tag{A.11}$$

Substituting Equation (A.7) for $\mathbf{B}^{(e)}$ (which contains the factor $1/L^{(e)}$ twice, once from $\mathbf{B}^{(e)}$ and once from $\mathbf{B}^{(e)\top}$):

$$EA^{(e)}L^{(e)}\mathbf{B}^{(e)\top}\mathbf{B}^{(e)} \;=\; \frac{EA^{(e)}}{L^{(e)}}\begin{pmatrix}-\hat{\mathbf{n}}\\ +\hat{\mathbf{n}}\end{pmatrix}\begin{pmatrix}-\hat{\mathbf{n}}^\top & +\hat{\mathbf{n}}^\top\end{pmatrix}. \tag{A.12}$$

Because Equation (A.10) must hold for every admissible virtual displacement $\delta\mathbf{u}^{(e)}$, the coefficient matrix is $\mathbf{k}^{(e)}$:

$$\boxed{\mathbf{k}^{(e)} \;=\; \frac{EA^{(e)}}{L^{(e)}}\begin{pmatrix}+\hat{\mathbf{n}}\hat{\mathbf{n}}^\top & -\hat{\mathbf{n}}\hat{\mathbf{n}}^\top\\ -\hat{\mathbf{n}}\hat{\mathbf{n}}^\top & +\hat{\mathbf{n}}\hat{\mathbf{n}}^\top\end{pmatrix} \;=\; \frac{EA^{(e)}}{L^{(e)}}\mathbf{B}^{(e)\top}\mathbf{B}^{(e)}\cdot L^{(e)2}} \tag{A.13}$$

where the last equality follows from recognising that writing $\mathbf{B}^{(e)}$ without the $1/L^{(e)}$ factor turns it into the rank-1 row vector $\tilde{\mathbf{B}}^{(e)} \equiv L^{(e)}\mathbf{B}^{(e)} = (-\hat{\mathbf{n}}^\top, \hat{\mathbf{n}}^\top)$ and the coefficient is $(EA^{(e)}/L^{(e)})\,\tilde{\mathbf{B}}^{(e)\top}\tilde{\mathbf{B}}^{(e)}$. In the code, the compact form $\mathbf{k}^{(e)} = (EA/L)\,\mathbf{B}^\top\mathbf{B}$ with $\mathbf{B} = (-\hat{\mathbf{n}}^\top, \hat{\mathbf{n}}^\top)$ is what appears on `truss_element.py:152`.

Properties of $\mathbf{k}^{(e)}$. The element stiffness matrix has three properties that every implementation test should verify:

1. Symmetry. $\mathbf{k}^{(e)} = \mathbf{k}^{(e)\top}$, because $\mathbf{B}^{(e)\top}\mathbf{B}^{(e)}$ is symmetric. This is physical: by Maxwell–Betti reciprocity, the force at joint i due to a unit displacement at j equals the force at j due to a unit displacement at i.

2. Positive semidefiniteness. $\mathbf{k}^{(e)} \succeq 0$, with one non-zero eigenvalue $2EA^{(e)}/L^{(e)}$ and $2d-1$ zero eigenvalues. The non-zero mode is axial stretch; the $2d-1$ zero modes are the rigid-body motions (translation in each of d directions and $d(d-1)/2$ rigid rotations) and the purely transverse relative displacements which a pin-jointed bar does not resist.

3. Rank. $\operatorname{rank}(\mathbf{k}^{(e)}) = 1$. Consequently, a truss assembled from M bars has a global stiffness matrix whose rank cannot exceed M before boundary condi-

tions are applied — the assembled $\mathbf{K}$ will therefore in general be singular and demand support fixing (Appendix A.6).

A.5 Assembly: Summation over Elements

Assembly is the step that maps element contributions into the global stiffness matrix. Number the joints $1,\dots,N$ and associate with joint p the d consecutive global degrees of freedom $\{(p-1)d+1,(p-1)d+2,\dots,pd\}$. For an element connecting joints i,j, the local-to-global DOF index map is the $2d$-tuple

$$\mathscr{I}^{(e)} = \big((i-1)d+1,\dots,id,\ (j-1)d+1,\dots,jd\big). \tag{A.14}$$

The global stiffness matrix $\mathbf{K}\in\mathbb{R}^{Nd\times Nd}$ is assembled by scattering each element contribution onto the rows and columns indexed by $\mathscr{I}^{(e)}$:

$$\mathbf{K}[\mathscr{I}^{(e)},\mathscr{I}^{(e)}] \ \mathrel{+}= \ \mathbf{k}^{(e)}, \qquad e=1,\dots,M. \tag{A.15}$$

In our code (`src/fem/assembly.py:114`) the scatter is implemented with `np.ix_(dofs,dofs)` so the inner loop is one line. $\mathbf{K}$ inherits three properties directly from the element contributions:

- symmetric ($\mathbf{k}^{(e)}=\mathbf{k}^{(e)\top}\ \forall e$);
- sparse (each bar scatters into only $(2d)^2$ entries, so average fill is $\mathscr{O}(M/N^2)$ which is tiny for triangulated trusses);
- positive semidefinite, with null space containing the rigid-body motions of the whole structure (d translations and $d(d-1)/2$ rotations, giving a null-space dimension of $d(d+1)/2$).

Load vector. The external loading is applied at the joints. Defining $\mathbf{f}_p\in\mathbb{R}^d$ as the external force at joint p, the global load vector is simply

$$\mathbf{F} \ = \ \big(\mathbf{f}_1^\top,\mathbf{f}_2^\top,\dots,\mathbf{f}_N^\top\big)^\top \ \in \ \mathbb{R}^{Nd}. \tag{A.16}$$

Self-weight is included by adding half the weight of each incident member to each end joint, a lumped-mass distribution that is exact under H1–H3 because bar forces are assumed uniform.

A.6 Boundary Conditions: Partition Method

Before boundary conditions the unassembled system

$$\mathbf{K}\mathbf{u} \;=\; \mathbf{F} \tag{A.17}$$

has no unique solution: $\mathbf{K}$ has a null space of dimension $d(d+1)/2$ (see Appendix A.5), so for any particular solution $\mathbf{u}^*$ we may add any rigid-body motion and still satisfy the equation. Support conditions restore uniqueness. Let

$$\mathscr{F} = \{\text{free DOFs}\}, \qquad \mathscr{S} = \{\text{prescribed DOFs}\}, \qquad \mathscr{F} \cup \mathscr{S} = \{1,\ldots,Nd\}, \quad \mathscr{F} \cap \mathscr{S} = \varnothing. \tag{A.18}$$

Partitioning $\mathbf{u}$ and $\mathbf{F}$ accordingly and writing $\mathbf{K}$ in block form

$$\begin{pmatrix} \mathbf{K}_{\mathscr{F}\mathscr{F}} & \mathbf{K}_{\mathscr{F}\mathscr{S}} \\ \mathbf{K}_{\mathscr{S}\mathscr{F}} & \mathbf{K}_{\mathscr{S}\mathscr{S}} \end{pmatrix} \begin{pmatrix} \mathbf{u}_{\mathscr{F}} \\ \mathbf{u}_{\mathscr{S}} \end{pmatrix} \;=\; \begin{pmatrix} \mathbf{F}_{\mathscr{F}} \\ \mathbf{F}_{\mathscr{S}} \end{pmatrix}, \tag{A.19}$$

and using the prescribed support displacements $\mathbf{u}_{\mathscr{S}}$ (commonly zero for pinned supports), we solve for the unknown free-DOF displacements by reducing Equation (A.19):

$$\boxed{\mathbf{K}_{\mathscr{F}\mathscr{F}}\,\mathbf{u}_{\mathscr{F}} \;=\; \mathbf{F}_{\mathscr{F}} - \mathbf{K}_{\mathscr{F}\mathscr{S}}\mathbf{u}_{\mathscr{S}}} \tag{A.20}$$

which is a well-posed symmetric positive-definite linear system provided the supports remove all rigid-body modes. For the standard zero-prescribed-displacement case (a pinned or roller support) Equation (A.20) collapses to $\mathbf{K}_{\mathscr{F}\mathscr{F}}\mathbf{u}_{\mathscr{F}} = \mathbf{F}_{\mathscr{F}}$.

The minimum number of DOFs to pin is

$$|\mathscr{S}|_{\min} \;=\; \tfrac{d(d+1)}{2} \;=\; \begin{cases} 3 & d=2 \\ 6 & d=3 \end{cases}. \tag{A.21}$$

A 2D truss therefore needs at least one pin (two DOFs) plus one roller (one DOF); a 3D truss needs a full pin and enough additional roller DOFs to kill the three rotations. In our code (`src/fem/solver.py:88`) failure to pin this minimum is caught explicitly, and an under-constrained structure yields the `LinAlgError: Singular matrix` that Equation (A.20) would otherwise produce only after a solver attempt.

Reactions. Once $\mathbf{u}_{\mathscr{F}}$ is known, the reactions at supports follow from the second row of Equation (A.19):

$$\mathbf{R}_{\mathscr{S}} \;=\; \mathbf{K}_{\mathscr{S}\mathscr{F}}\mathbf{u}_{\mathscr{F}} + \mathbf{K}_{\mathscr{S}\mathscr{S}}\mathbf{u}_{\mathscr{S}} - \mathbf{F}_{\mathscr{S}}, \tag{A.22}$$

which the solver returns alongside the displacements for post-processing and as a global-equilibrium sanity check.

A.7 Post-Processing: Stress, Axial Force, and Equilibrium

After the reduced system is solved, the full displacement vector $\mathbf{u} \in \mathbb{R}^{Nd}$ is known; stresses, strains and member forces are recovered element by element:

1. Extract element displacements. For element e pick out the $2d$ components $\mathbf{u}^{(e)} = \mathbf{u}[\mathscr{J}^{(e)}]$.
2. Axial strain from Equation (A.7): $\varepsilon^{(e)} = \mathbf{B}^{(e)}\mathbf{u}^{(e)} = \hat{\mathbf{n}}^{(e)} \cdot (\mathbf{u}_j - \mathbf{u}_i)/L^{(e)}$.
3. Axial stress from Equation (A.8): $\sigma^{(e)} = E\,\varepsilon^{(e)}$.
4. Axial force from Equation (A.9): $N^{(e)} = A^{(e)}\sigma^{(e)}$.

All four quantities are scalars; their sign convention is "tension positive".

Global equilibrium check. A good implementation verifies at post-processing time that

$$\sum_{p \in \mathscr{S}} \mathbf{R}_p + \sum_{p=1}^{N} \mathbf{f}_p \;=\; \mathbf{0} \tag{A.23}$$

to machine precision. Violations flag integration mistakes in the load vector, forgotten supports, or incorrectly scattered element contributions. Our unit test `tests/test_fem_3bar.py` includes this check.

Linear-elastic assumptions and their scope. The derivation above is exact under H1–H3. In the thesis benchmarks, H1 and H2 are introduced axiomatically by the problem statements (every published truss optimum of Chapter 4 uses the same assumptions). H3 is the only one that can be challenged post-hoc: non-linearity shows up in two forms:

- Material non-linearity: yielding in steel beyond the elastic limit f_y. This is not captured by Equation (A.8). For the IS 800 compliance study of Section 4.10, the tension-yield stress limit is enforced as an inequality constraint $|\sigma^{(e)}| \le f_y$, not as a change of constitutive law, so linear elasticity remains the valid operating assumption throughout the optimiser's search space.
- Geometric non-linearity: buckling of slender compression members. Equation (A.1) is the first-order form; it does not predict bifurcation. We therefore do not use the FEM to decide buckling: the IS 800 buckling curve gives

a closed-form compression capacity P_d and the optimiser enforces $|N^{(e)}| \le P_d$ directly on element compression forces (Section 4.10).

Both exclusions are standard in the benchmark literature [31], [32] and are faithfully reflected in the constraint module `src/constraints/is800_checks.py`.

A.8 End-to-End Worked Example

To ground the general derivation, we work through the single horizontal bar in `src/fem/solver.py:147`. A bar of length $L = 1\,\mathrm{m}$ connects joint 1 at $(0,0)$ and joint 2 at $(1,0)$, with $E = 200\,\mathrm{GPa}$ and $A = 1\times10^{-4}\,\mathrm{m}^2$. Joint 1 is fully pinned ($\mathscr{S} \supset \{1,2\}$) and joint 2 carries an in-plane horizontal load $F_{2x} = 1{,}000\,\mathrm{N}$. The y-DOF of joint 2 (DOF index 4) must also be pinned to remove the transverse zero mode of a single bar in 2D.

Step 1 — Direction cosines and $\mathbf{B}^{(e)}$: $\hat{\mathbf{n}} = (1,0)^\top$, so $\mathbf{B}^{(e)} = L^{-1}(-1,0,1,0) = (-1,0,1,0)$.

Step 2 — Element stiffness:

$$\mathbf{k}^{(e)} = \frac{EA}{L}\mathbf{B}^{(e)\top}\mathbf{B}^{(e)} = 2\times10^{7}\,\mathrm{N/m}\begin{pmatrix} +1 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & +1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

With only one element $\mathbf{K} = \mathbf{k}^{(e)}$.

Step 3 — Partition: the free set is $\mathscr{F} = \{3\}$ (the horizontal DOF of joint 2); the prescribed set is $\mathscr{S} = \{1,2,4\}$ with $\mathbf{u}_{\mathscr{S}} = \mathbf{0}$. $\mathbf{K}_{\mathscr{F}\mathscr{F}} = 2\times10^{7}\,\mathrm{N/m}$, $\mathbf{F}_{\mathscr{F}} = 1{,}000\,\mathrm{N}$.

Step 4 — Solve reduced system Equation (A.20): $u_{2x} = 1{,}000\,\mathrm{N}/2\times10^{7}\,\mathrm{N/m} = 5\times10^{-5}\,\mathrm{m} = 50\,\mu\mathrm{m}$.

Step 5 — Axial strain, stress, force: $\varepsilon = 5\times10^{-5}\,\mathrm{m}^{-1}\cdot L = 5\times10^{-5}$, $\sigma = E\varepsilon = 10\,\mathrm{MPa}$, $N = A\sigma = 1{,}000\,\mathrm{N}$ (tension).

Step 6 — Reaction at joint 1: from Equation (A.22), $R_{1x} = -1{,}000\,\mathrm{N}$, satisfying global equilibrium to machine precision.

Every step above appears identically in `src/fem/solver.py`'s `if __name__=="__main__"` block, and its assertion $u_{2x} = 5\times10^{-5}\,\mathrm{m}$ is the simplest validation gate of the FEM stack.

A.9 Notation Summary

Symbol	Meaning	Units
d	spatial dimension (2 or 3)	—
N	number of joints	—
M	number of bars	—
E	Young's modulus of steel	Pa
$A^{(e)}$	cross-sectional area of bar e	m^2
$L^{(e)}$	undeformed length of bar e	m
$\hat{\mathbf{n}}^{(e)}$	unit direction vector of bar e	—
$\mathbf{B}^{(e)}$	strain-displacement matrix of bar e $(1\times 2d)$	m^{-1}
$\mathbf{k}^{(e)}$	element stiffness matrix $(2d\times 2d)$	N/m
$\mathbf{K}$	global stiffness matrix $(Nd\times Nd)$	N/m
$\mathbf{u}$	nodal-displacement vector	m
$\mathbf{F}$	nodal-force vector	N
$\varepsilon^{(e)}, \sigma^{(e)}, N^{(e)}$	element axial strain, stress, force	—, Pa, N
$\mathscr{F}, \mathscr{S}$	free / prescribed DOF sets	—

Appendix B

IS 800:2007 Provisions Applied to Truss Members

This appendix states, derives where necessary, and discusses the specific clauses of IS 800:2007 [31] that the constraint module `src/constraints/is800_checks.py` enforces on every truss candidate. The thesis proper (Section 4.10) reports the numerical consequences of these checks; this appendix documents the underlying code of practice so a reviewer can verify formula-by-formula correspondence.

All formulae are written in SI units; truss benchmarks published in imperial units (10-bar, 25-bar, 72-bar) are converted to SI inside the caller before these provisions are evaluated.

B.1 Scope, Material Constants, and Safety Factors

IS 800:2007 is the Indian limit-state design code for general construction in steel. The limit states considered for truss members are:

- ultimate: tension yielding (Clause 6.2), tension rupture (Clause 6.3), and compression buckling (Clause 7.1);
- stability: slenderness limits (Clause 3.8);
- serviceability: deflection limits (Clause 5.6.1).

The three remaining limit states that IS 800 lists (fatigue, brittle fracture, fire) are not applicable to the thesis benchmarks and are therefore not enforced.

B.1.1 Default Material: Fe 410

The thesis uses structural steel grade Fe 410 throughout (the default grade of IS 2062 for general construction). Its characteristic strengths are:

$$f_y = 250\,\text{MPa}, \qquad f_u = 410\,\text{MPa}, \qquad E = 2.0 \times 10^{11}\,\text{Pa}. \tag{B.1}$$

These values are hard-coded as `FY_DEFAULT`/`FU_DEFAULT` in `is800_checks.py` and can be over-ridden per-call by the kwargs `fy`, `fu`.

B.1.2 Partial Safety Factors

IS 800 Table 5 distinguishes two partial safety factors on material resistance that are relevant to tension and compression members:

$$\gamma_{m0} = 1.10 \quad \text{(yielding at gross section)}, \qquad \gamma_{m1} = 1.25 \quad \text{(ultimate at net section or in buckling)} \tag{B.2}$$

All reported resistances in this appendix are design resistances that already incorporate the relevant γ_m. The optimiser treats member demand $N^{(e)}$ as the signed factored axial force from the FEM and compares it directly against these design resistances.

B.1.3 Cross-Section Assumption: Solid Circular

Every bar in the thesis benchmarks is modelled as a solid circular cross-section of area $A^{(e)}$. This reduces the number of design variables from three (area + a shape descriptor pair) to one (area alone) — a modelling choice that matches the original Kaveh benchmark conventions [14] and makes reported areas directly comparable with the published literature. For a solid disc of area A and radius $R = \sqrt{A/\pi}$, the second moment of area is $I = \pi R^4/4$ and the radius of gyration is:

$$r_g = \sqrt{\frac{I}{A}} = \frac{R}{2} = \frac{1}{2}\sqrt{\frac{A}{\pi}}. \tag{B.3}$$

Implementation: `radius_of_gyration_circular` at `is800_checks.py:52`. A future extension to rolled tubes or angles is a single-function override.

B.2 Clause 3.8 — Slenderness Limits

For any truss member the effective slenderness ratio is

$$\lambda = \frac{KL}{r_g}, \tag{B.4}$$

where K is the effective length factor (theoretical pin-pin ends give $K = 1.0$, which is the value used throughout) and r_g is the radius of gyration from Equation (B.3). IS 800 Table 3 places separate limits for primary members on tension and compression:

$$\lambda \leq \begin{cases} 180 & \text{tension member,} \\ 250 & \text{compression member.} \end{cases} \tag{B.5}$$

The tension limit is a constructibility / serviceability requirement (to prevent sag under self-weight of long tension ties); the compression limit is a stability requirement (long compression members have residual imperfection amplitudes that make design-office analysis unreliable above $\lambda \sim 250$).

Implementation: `check_slenderness` at `is800_checks.py:65`. The constraint is enforced as hard (any violation marks the design infeasible); a 1-percent tolerance is applied as standard in the penalty function to avoid knife-edge infeasibility on floating-point evaluation.

B.3 Clause 6.2 — Tension Capacity, Gross-Section Yielding

A tension member first yields over its gross cross-sectional area when the demand reaches

$$T_{dg} = \frac{A_g f_y}{\gamma_{m0}}. \tag{B.6}$$

A_g is the gross area ($= A^{(e)}$ for the welded-end solid-bar members used in the thesis). For Fe 410, $T_{dg}/A_g = 227.27\,\text{MPa}$, a quantity that appears as a horizontal line on every stress-versus-axial-force plot in Chapter 4 where IS 800 is switched on.

For benchmarks without an IS 800 layer (the bare Kaveh-style stress cap of the 10-, 25- and 72-bar problems) the admissible tension stress is the standard $f_y = 250\,\text{MPa}$ applied without the factor $1/\gamma_{m0}$. This difference accounts for most of the weight penalty that IS 800 inclusion introduces in Section 4.11.3 — not the compression buckling curve, as one might first assume.

B.4 Clause 6.3 — Tension Capacity, Net-Section Rupture

Where a tension member is bolt-connected, the net area A_n is the gross area minus the bolt-hole deductions. IS 800 gives the ultimate net-section rupture capacity as

$$T_{dn} = \frac{0.9 A_n f_u}{\gamma_{m1}}. \tag{B.7}$$

The factor 0.9 is an empirical allowance for eccentricity and shear lag at the end connection. For welded-end members (the thesis default), $A_n = A_g$ and Equation (B.7) simplifies to $T_{dn} = 0.9 A_g f_u / \gamma_{m1} = 295.2\,\mathrm{MPa} \cdot A_g$. This exceeds T_{dg} for Fe 410, so Equation (B.6) is the binding tension-capacity constraint for welded-end members. In bolt-connected benchmarks it is necessary to check both.

Implementation: `check_tension_rupture` at `is800_checks.py:114`, kwarg `net_area_ratio` defaults to 1.0 (welded ends).

B.5 Clause 7.1 — Compression Capacity (Perry-Robertson)

IS 800 Clause 7.1.2 gives the design compressive stress f_{cd} as a reduced yield strength

$$f_{cd} = \chi \frac{f_y}{\gamma_{m0}}, \qquad 0 < \chi \leq 1, \tag{B.8}$$

where the reduction factor χ is the closed-form Perry-Robertson expression adopted from Eurocode 3:

$$\chi = \frac{1}{\varphi + \sqrt{\varphi^2 - \overline{\lambda}^2}}, \qquad \varphi = \tfrac{1}{2}\left[1 + \alpha\left(\overline{\lambda} - 0.2\right) + \overline{\lambda}^2\right], \tag{B.9}$$

and the non-dimensional slenderness $\overline{\lambda}$ is the ratio of yield-stress to elastic-buckling-stress half-powered:

$$\overline{\lambda} = \sqrt{\frac{f_y}{f_{cc}}}, \qquad f_{cc} = \frac{\pi^2 E}{(KL/r_g)^2}. \tag{B.10}$$

f_{cc} is the Euler critical stress for an ideally straight elastic pinned-pinned column; $\overline{\lambda} \to 0$ recovers $\chi = 1$ (unreduced yield), and $\overline{\lambda} \to \infty$ recovers $\chi = 1/\overline{\lambda}^2$ (elastic Euler-buckling limit).

The design compressive force resistance is then

$$P_d = A^{(e)} f_{cd}. \tag{B.11}$$

Implementation: `check_compression` at `is800_checks.py:140`. In the penalty

function, $|N^{(e)}_{\text{compression}}| \le P_d$ is enforced with the same 1-percent tolerance as the tension check.

B.5.1 Choice of Buckling Curve

IS 800 Table 7 distinguishes four buckling-curve variants $(\mathrm{a},\mathrm{b},\mathrm{c},\mathrm{d})$ via the imperfection factor α:

$$\alpha \in \{0.21, 0.34, 0.49, 0.76\} \quad \text{for curves} \quad (\mathrm{a},\mathrm{b},\mathrm{c},\mathrm{d}). \tag{B.12}$$

The curve to use depends on the cross-section geometry and the buckling axis; IS 800 Table 7 is the authoritative mapping. For the thesis solid-circular cross-section, Table 7 does not explicitly enumerate an entry (the code tabulates rolled I, H, angle, channel, tubular, and welded box sections). The closest physical match is a rolled tubular section, which Table 7 places on curve a ($\alpha = 0.21$). However, thesis results use curve b ($\alpha = 0.34$) as a conservative middle-ground choice: this matches the recommendation of Subramanian [32] for "general-purpose welded solid bars" and errs on the safe side. Switching to curve a is a one-line override (`alpha=0.21`) in `check_compression` and typically lowers the optimised weight by 1–3 % on the benchmarks of Section 4.11.3.

B.5.2 Closed-Form Bounds on χ

Two trivial properties of Equation (B.9) help debugging:

- $\chi \le 1$ always, enforced explicitly in `is800_checks.py:163` by a `min(chi,1.0)`. Without this clip, very-low-slenderness ($\overline{\lambda} \to 0$) cases can produce numerically-slightly-greater-than-unity χ due to round-off in $\sqrt{\varphi^2 - \overline{\lambda}^2}$.
- $\chi > 0$ always, because $\varphi > \overline{\lambda}$ for all $\overline{\lambda} > 0$ and $\alpha > 0$.

In the IS 800 study of Section 4.10, the reported median χ values on the 10-bar compression members are $\chi \in [0.42, 0.88]$; a purely-yielding material model would correspond to $\chi \equiv 1$, which is why including IS 800 increases optimised weight.

B.6 Clause 5.6.1 — Serviceability Deflection Limit

IS 800 Clause 5.6.1 caps the maximum deflection of a structure under its characteristic service load:

$$\delta_{\max} \;\le\; \frac{L_{\text{span}}}{325}, \tag{B.13}$$

where the divisor 325 is the Table 6 default for roof purlins and girts; for typical rafters the divisor is 240, for cantilevers 150, and for gantry girders 750. The thesis uses 325 as a generic default because the benchmark loads are classical gravity loads and the benchmark geometries are roof-truss-like. The divisor is a kwarg (`divisor`) of `check_deflection` and is over-ridable per call without recompiling any higher-level code.

Observation. For the specific Kaveh 10-bar geometry ($L_{\text{span}} = 720\,\text{in} = 18.3\,\text{m}$), the IS 800 serviceability deflection limit at divisor 325 is $\delta_{\max} = 56.3\,\text{mm}$. The benchmark's original Imperial-unit problem statement fixes the tip deflection limit at $2.0\ \text{in} = 50.8\,\text{mm}$, which is tighter than IS 800 serviceability would require. The 10-bar displacement constraint therefore remains the binding serviceability check in practice.

B.7 Compliance Matrix: Code Checks per Benchmark

Table B.1 summarises which of the above clauses is enforced in each benchmark. Benchmarks whose original problem statement already caps a stress or a deflection are kept in their original units and limit unless the IS 800 ablation is specifically switched on.

Table B.1: Per-benchmark IS 800 coverage. ✓ means the clause is enforced by default; "–" means the benchmark's original non-IS 800 limit is kept; "opt." means IS 800 is enabled only for the ablation in Section 4.11.3.

Clause / Benchmark	10-bar	25-bar	72-bar	200-bar	Notes
3.8 slenderness	opt.	opt.	opt.	✓	200-bar SI units
6.2 tension yield	opt.	opt.	opt.	✓	
6.3 tension rupture	opt.	opt.	opt.	✓	welded ends
7.1 compression	opt.	opt.	opt.	✓	curve b default
5.6.1 deflection	–	–	–	✓	divisor 325

The 200-bar stepped-tower benchmark is the only one specified natively in SI units and is therefore the only benchmark whose default configuration has IS 800 fully engaged.

B.8 Validation: Worked Numerical Check

As a spot-check that Equation (B.9)–Equation (B.11) is implemented correctly, consider a single 2.0-m solid-circular bar of area $A = 5 \times 10^{-4}\,\text{m}^2$ (diameter $\approx$

25.2 mm) under compression. Step-by-step:

$$
\begin{aligned}
r_g &= \tfrac{1}{2}\sqrt{A/\pi} = \tfrac{1}{2}\sqrt{5\times10^{-4}\,\mathrm{m}^2/\pi} = 6.3\times10^{-3}\,\mathrm{m},\\
\lambda &= KL/r_g = 2.0\,\mathrm{m}/6.3\times10^{-3}\,\mathrm{m} = 317.5,\\
f_{cc} &= \pi^2 E/\lambda^2 = \pi^2\cdot 2.0\times10^{11}\,\mathrm{Pa}/317.5^2 = 1.96\times10^{7}\,\mathrm{Pa},\\
\overline{\lambda} &= \sqrt{f_y/f_{cc}} = \sqrt{2.5\times10^{8}\,\mathrm{Pa}/1.96\times10^{7}\,\mathrm{Pa}} = 3.57,\\
\varphi &= \tfrac{1}{2}\left[1+0.34(3.57-0.2)+3.57^2\right] = 7.45,\\
\chi &= \left[7.45+\sqrt{7.45^2-3.57^2}\right]^{-1} = 0.073,\\
f_{cd} &= \chi f_y/\gamma_{m0} = 0.073\cdot 2.5\times10^{8}\,\mathrm{Pa}/1.1 = 1.65\times10^{7}\,\mathrm{Pa},\\
P_d &= A\,f_{cd} = 5\times10^{-4}\,\mathrm{m}^2\cdot 1.65\times10^{7}\,\mathrm{Pa} = 8.27\times10^{3}\,\mathrm{N}.
\end{aligned}
$$

The slenderness is well above the 250-limit, so the Clause 3.8 check fails first; a compliant cross-section would need a larger radius of gyration. This is exactly the behaviour observed on the 200-bar optimiser runs in Section 4.5, where small-area members at the tower top are driven by the 250 compression-slenderness limit rather than by the Perry-Robertson capacity.

Appendix C

Geometry, Loading, and Design-Variable Specifications

This appendix lists the full numerical specification of each of the four benchmarks used in Chapter 4: nodal coordinates, element connectivity, supports, load cases, material constants, allowable displacements, stress limits, design-variable groupings, and published reference optima. All values are quoted in the benchmark's original units, which are preserved inside the code to match the literature exactly; the 200-bar benchmark is the only one natively specified in SI. Each table's `label` mirrors the structure `tab:geom:<bench>:<aspect>`.

Notation. For every benchmark the design-variable vector is denoted $\mathbf{a} \in \mathbb{R}^{n_{\mathrm{dv}}}$, with each component an area group parameter. The mapping from group index to physical bar index is the group-map, stored as `group_map` on each benchmark class. An area-group of size ≥ 2 ties the areas of multiple bars together (typical in symmetric trusses). In every benchmark the area is the only design-variable type — sizing optimisation only, no topology or shape modifications.

C.1 10-bar Planar Cantilever

The 10-bar planar cantilever of Haftka and Gürdal [42] is the oldest benchmark in the sizing-optimisation literature and is accordingly the most reported. It comprises six nodes arranged as a 3×2 rectangular grid; the two left-most nodes are pinned supports, and two downward tip loads are applied at the two right-most free nodes.

Supports. Nodes 5 and 6 are fully pinned (both x and y DOFs fixed). All other nodes are free. The minimum pin count ($d(d+1)/2 = 3$ in 2D) is exceeded, so the structure is statically indeterminate to first degree [32].

Table C.1: 10-bar: nodal coordinates (inches). All nodes lie in the xy-plane.

Node	X (in)	Y (in)
1	720	360
2	720	0
3	360	360
4	360	0
5	0	360
6	0	0

Table C.2: 10-bar: element connectivity. Bar numbering follows the Haftka–Gürdal convention.

Bar	Node i	Node j
1	5	3
2	3	1
3	6	4
4	4	2
5	3	4
6	1	2
7	4	5
8	3	6
9	2	3
10	1	4

Loading. A single load case is specified: LC1 applies a downward force of $F_y = -100\,\text{kip} = -100 \times 10^3\,\text{lb}$ at both node 2 and node 4.

Design variables. $n_{\text{dv}} = 10$ — every bar is its own independent area variable. The area-group map is the identity $g(i) = i$ for $i = 1, \ldots, 10$.

Limits. Side bounds on area: $0.1 \le a_i \le 35.0\,\text{in}^2$. Allowable displacement at every free DOF: $|u| \le 2.0\,\text{in}$. Allowable axial stress: $|\sigma| \le 25\,\text{ksi} = 25 \times 10^3\,\text{psi}$ in both tension and compression. Material: $E = 10\,\text{Msi} = 10 \times 10^6\,\text{psi}$, $\rho = 0.1\,\text{lb/in}^3$.

Reference optimum. The canonical optimum weight reported by Kaveh [14] is $W^\star = 5{,}060.85\,\text{lb}$; our PSO reproduces this to within $\pm 0.05\,\text{lb}$ across 30 seeds (Section 4.2).

C.2 25-bar Spatial Transmission Tower

The 25-bar spatial transmission tower is a $d = 3$ dimensional benchmark with two stacked frustra. Ten nodes and twenty-five bars are grouped into eight symmetric area groups. Two load cases capture asymmetric and anti-symmetric wind actions.

Table C.3: 25-bar: nodal coordinates (inches).

Node	X	Y	Z
1	-37.5	0.0	200
2	37.5	0.0	200
3	-37.5	37.5	100
4	37.5	37.5	100
5	37.5	-37.5	100
6	-37.5	-37.5	100
7	-100	100	0
8	100	100	0
9	100	-100	0
10	-100	-100	0

Supports. Nodes 7–10 are fully pinned ($u = v = w = 0$), giving 12 prescribed DOFs and $10 \cdot 3 - 12 = 18$ free DOFs.

Connectivity. Bars are listed in groups reflecting the physical role of each bar family. The 25 bars are assigned to 8 area groups as follows:

Group	Bar numbers (1-indexed)
1 (top strut)	1
2 (upper inclined)	2–5
3 (waist horizontal)	6–9
4 (mid vertical)	10–11
5 (inner diagonal)	12–13
6 (outer upper)	14–17
7 (outer middle)	18–21
8 (outer lower)	22–25

Load cases.

- LC1 (asymmetric): node 1 force $(1000, 10000, -5000)$ lb; node 2 force $(0, 10000, -5000)$ lb; node 3 and node 6 each $(500, 0, 0)$ lb.
- LC2 (anti-symmetric): node 1 $(0, 20000, -5000)$ lb; node 2 $(0, -20000, -5000)$ lb.

Limits. Area bounds $0.01 \le a_i \le 3.4\,\text{in}^2$. Allowable displacement at every free DOF: $|u| \le 0.35\,\text{in}$. Stress bounds are set per group (Table 1 of [14]); the implementation stores them in `group_stress_allow`.

Reference optimum. Published Kaveh optimum $W^\star = 545.22\,\text{lb}$; reproduced by our NSGA-II+surrogate to within $0.1\,\%$ across 30 seeds.

C.3 72-bar Spatial Four-Tier Tower

The 72-bar spatial tower has twenty nodes stacked on four levels of a $60\,\text{in} \times 60\,\text{in}$ square footprint, each level $60\,\text{in}$ high. Seventy-two bars are grouped into sixteen area groups. This is the deepest 3D benchmark in the thesis and therefore the hardest for a constraint-obeying optimiser.

Table C.4: 72-bar: nodal coordinates (inches). Nodes 1–16 are the four upper levels, nodes 17–20 the ground level.

Node	X	Y	Z
1	60	60	60
2	60	-60	60
3	-60	-60	60
4	-60	60	60
5	60	60	120
6	60	-60	120
7	-60	-60	120
8	-60	60	120
9	60	60	180
10	60	-60	180
11	-60	-60	180
12	-60	60	180
13	60	60	240
14	60	-60	240
15	-60	-60	240
16	-60	60	240
17	60	60	0
18	60	-60	0
19	-60	-60	0
20	-60	60	0

Supports. Nodes 17–20 are fully pinned. The other 16 nodes are free, giving 48 free DOFs.

Load cases.

- LC1: node 13 force $(5000, 5000, -5000)\,\text{lb}$ (asymmetric tip load at one corner).
- LC2: $-5{,}000\,\text{lb}$ vertical at each of the four top-level nodes 13–16.

Design variables and groups. $n_{\text{dv}} = 16$. Each of the four tiers has four area-groups (bottom columns, horizontals, verticals, and diagonals), giving $4 \times 4 = 16$.

Table C.5: 72-bar: element connectivity (first 20 of 72). The remaining 52 bars follow the four-fold tier-symmetric pattern of the first tier — see `src/benchmarks/truss_72bar.py` for the complete table.

Bar	Node i	Node j
1	17	1
2	18	2
3	19	3
4	20	4
5	1	2
6	2	3
7	3	4
8	4	1
9	1	18
10	2	19
11	3	20
12	4	17
13	17	2
14	18	3
15	19	4
16	20	1
17	1	3
18	2	4
19	1	5
20	2	6
$\cdots$ 52 rows omitted, see source		

Limits. Area bounds $0.1 \le a_i \le 4.0\,\text{in}^2$. Allowable displacement at every top-level free DOF (u_x, u_y at nodes 13–15): $|u| \le 0.25\,\text{in}$. Stress limit as 10-bar.

Reference optimum. $W^\star = 379.62\,\text{lb}$ (Kaveh and Talatahari [14]).

C.4 200-bar Planar Stepped Tower

The 200-bar benchmark is the largest truss in the thesis and the only benchmark defined in SI units. It is a 2D stepped seven-bay-wide tower of eleven full-width tiers plus the top apex; its geometry is reproduced from the original Kaveh–Kalateh-Ahani formulation [43], but coordinates are our own cm-rounded coordinates (indicated in the benchmark API with `reference_verified=False`).

Supports. Nodes 1–7 (bottom row) are fully pinned in both translations: 14 DOFs fixed out of $77 \times 2 = 154$ total, leaving 140 free.

Load cases. Three gravity load cases are specified:

Table C.6: 200-bar: nodal coordinates (metres). First 15 of 77 total nodes shown; remaining nodes continue the vertical 3 m stride pattern.

Node	X (m)	Y (m)
1	-4.5	0
2	-3.0	0
3	-1.5	0
4	0	0
5	1.5	0
6	3.0	0
7	4.5	0
8	-4.5	3
9	-3.0	3
10	-1.5	3
11	0	3
12	1.5	3
13	3.0	3
14	4.5	3
15	-4.5	6
⋯ 62 rows omitted, see source		

- LC1: 1 kN downward at every top-level node (nodes 71–77).
- LC2: 10 kN downward at top-level nodes 71–73 and at every east-side edge node (nodes 8, 22, 36, 50, 64).
- LC3: LC1 + LC2 superposed.

Design variables. $n_{\mathrm{dv}} = 29$. Each group represents one structural role (edge chord, internal vertical, diagonal, horizontal) at one tier of the tower; the full group-map is listed in `src/benchmarks/truss_200bar.py`.

Limits. Area bounds $6.5 \times 10^{-5} \le a_i \le 2.5 \times 10^{-2}\,\mathrm{m}^2$ (equivalent to $0.1 \le a_i \le 38.75\,\mathrm{in}^2$ — wide enough to admit the same search space as the original imperial formulation). Displacement limit: every top-row free DOF is capped at 0.1 m. Material: $E = 210\,\mathrm{GPa}$, $\rho = 7{,}850\,\mathrm{kg/m}^3$ (structural steel). Stress allowable: $|\sigma| \le 250\,\mathrm{MPa}$ per Fe 410.

Reference optimum. No published SI-unit optimum matches our coordinate choices. Our own batch of 30 PSO seeds (Ch. 4 Section 4.5) converges to $W^\star \approx 315.89\,\mathrm{kg}$ for the best seed, with median 330.1 kg. These serve as the reference for Phase 10 ablation studies and are reported to four significant figures.

C.5 Cross-Benchmark Summary

The four benchmarks span two orders of magnitude in number of design variables and bars, and both 2D and 3D configurations, giving a balanced coverage of the sizing-optimisation difficulty landscape (Table C.7).

Table C.7: Cross-benchmark numerical summary. Units kept in each benchmark's native system for clarity.

Quantity	10-bar	25-bar	72-bar	200-bar
Dimension d	2	3	3	2
Nodes N	6	10	20	77
Bars M	10	25	72	200
Design vars n_{dv}	10	8	16	29
Load cases	1	2	2	3
Displacement limit	2.0 in	0.35 in	0.25 in	0.1 m
Stress allowable	25 ksi	(grouped)	25 ksi	250 MPa
Units	Imperial	Imperial	Imperial	SI
Reference $W^{\star}$	5,060.85 lb	545.22 lb	379.62 lb	315.89 kg
Pinned DOFs	4	12	12	14
Free DOFs	8	18	48	140

Closing remark. Although the problem sizes differ by over an order of magnitude, the same constraint API (`src/constraints/`) and the same FEM backend (Appendix A) handle all four without benchmark-specific logic. The only benchmark-specific information is the contents of `src/benchmarks/truss_*.py` files; those files collectively reproduce the tables above and supply them to the generic evaluator via `BenchmarkBase.evaluate()`.

Appendix D

Reproducibility Package

This appendix documents the exact software environment, command sequences, random seeds, and stored-artefact hashes needed for a third-party reviewer to reproduce every numerical result in Chapter 4 bit-for-bit. Anywhere a result is reported as "median over 30 seeds" or "best of 10 seeds" in the main text, the 30 or 10 seeds are the 10 enumerated in Appendix D.3, batched three-times-over for the large-run studies (§4.5, §4.8).

D.1 Source-Tree Layout

All paths below are relative to the repository root:

```
src/                          Python source code
  benchmarks/                 10/25/72/200-bar problem definitions
  fem/                        truss FEM stack (Appendix A)
  constraints/                IS 800 checks (Appendix B)
  algorithms/                 GA, PSO, NSGA-II wrappers
  ml/                         surrogate MLP (Phase 2)
  rl/                         RL designer (Phase 3)
  llm/                        Anthropic Claude warm-start (Phase 4)
  app/                        FastAPI backend + Streamlit UI (Phase 5)
  plotting/                   style.py + figure helpers
  utils/                      logging, seeding, I/O
scripts/                      top-level entry points
  run_batch.py                multi-seed GA/PSO/NSGA-II runner
  run_single.py               one-shot solver for debugging
  train_surrogate.py          dataset gen + MLP training
  run_llm_warmstart.py        LLM designer driver
  run_mc_dropout_analysis.py     Day-1 Appendix 4.14
```

```
  run_llm_cache_reanalysis.py    Day-1 Appendix 4.15
  run_convergence_rate_fits.py   Day-1 Appendix 4.16
  generate_all_figures.py  Ch 4 figure regeneration
  run_overnight_200bar.sh  resilient nightly batch
tests/                     pytest suite (currently 81 tests)
results/                   CSV summaries + run pickles
figures/                   PNG/SVG (regenerated, git-ignored)
thesis_writeup/            LaTeX sources + compiled PDF
```

Total Python files: 73. Total git-tracked files: 185 (the repository is dense but not sprawling).

D.2 Software Environment

All results were produced on a single workstation running macOS 15.5 on Apple Silicon (ARM64) with Python 3.12. For CI, the GitHub Actions workflow (`.github/workflows/ci.yml`) pins Python 3.12 on Ubuntu 22.04 and passes the same test suite; differences between the two platforms are confined to dotted BLAS linkage (Accelerate on macOS, OpenBLAS on Linux) and do not affect published results to the four significant figures reported in Chapter 4.

D.2.1 Pinned Dependencies

The complete `requirements.txt` is reproduced below:

- Phase 1 core: `numpy==2.4.4`, `scipy==1.17.1`, `pymoo==0.6.1.6`.
- Plotting: `matplotlib==3.10.8`, `seaborn==0.13.2`.
- Tables / analysis: `pandas==3.0.2`.
- Testing: `pytest==9.0.3`.
- Progress UI: `tqdm==4.67.3`.
- Phase 2 surrogate: `torch==2.11.0`.
- Phase 3 RL: `stable-baselines3==2.8.0`, `gymnasium==1.2.3`.
- Phase 4 LLM: `anthropic==0.96.0`, `python-dotenv==1.2.2`.
- Phase 5 app: `streamlit==1.56.0`, `fastapi==0.136.0`, `uvicorn==0.46.0`, `plotly==6.7.0`, `httpx==0.28.1`.

D.2.2 Setup Commands

From a fresh clone, reproduction starts with:

```
git clone <repo-url> thesis && cd thesis
python3.12 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

# (optional) populate your Anthropic API key for LLM re-runs:
echo "ANTHROPIC_API_KEY=sk-..." > .env

# smoke-test the environment:
pytest -q -m "not slow"         # ~30 s, 70 green
```

The slow tests (the tag `-m slow`) are the five batch-validation tests and can take several minutes; they are excluded from CI and from the 30-second smoke test.

D.3 Random Seeds

Every stochastic run (GA, PSO, NSGA-II, LHS dataset generation, LLM sampling) uses a caller-supplied integer seed. Across Chapter 4 the fixed pool of 10 seeds is:

$$\mathscr{S}_{10} = \{42, 123, 456, 789, 2{,}024, 9{,}999, 11{,}235, 27{,}182, 31{,}415, 8{,}675{,}309\}. \quad \text{(D.1)}$$

The list is chosen to include the three most-cited "famous" seeds in scientific Python (42, 27,182, 31,415) plus seven arbitrary integers so that any correlation between a "lucky" seed and reported median would show up as a distributional anomaly in the full 30-seed sample. For the large-run studies that need 30 seeds (§4.5, §4.8), the pool is repeated three times with three independent initial-population LHS draws (`pymoo LHS(criterion="cm", iterations=50)`). Every pickle stored in `results/` has a filename of the form `<benchmark>_<algo>_seed<seed>.pkl` so the provenance is self-documenting.

D.4 Deterministic Command Sequences

D.4.1 Ch 4.1 — 10-bar benchmark

```
python scripts/run_batch.py --benchmark 10bar --algo ga   --seeds 10
```

```
python scripts/run_batch.py --benchmark 10bar --algo pso  --seeds 10
python scripts/run_batch.py --benchmark 10bar --algo nsga2 --seeds 10
```

These three commands produce 30 pickles in `results/` (named `10bar_{ga,pso,nsga2}_seed*.p` and three per-algorithm aggregate CSVs. Wall-clock time on Apple M1 is $\sim$90 s per algorithm.

D.4.2 Ch 4.2 — 25-bar benchmark

```
python scripts/run_batch.py --benchmark 25bar --algo ga   --seeds 10
python scripts/run_batch.py --benchmark 25bar --algo pso  --seeds 10
python scripts/run_batch.py --benchmark 25bar --algo nsga2 --seeds 10
```

Wall-clock: $\sim$180 s per algorithm.

D.4.3 Ch 4.3 — 72-bar benchmark

```
python scripts/run_batch.py --benchmark 72bar --algo ga   --seeds 10
python scripts/run_batch.py --benchmark 72bar --algo pso  --seeds 10
python scripts/run_batch.py --benchmark 72bar --algo nsga2 --seeds 10
```

Wall-clock: $\sim$240 s per algorithm.

D.4.4 Ch 4.4 — Surrogate training

```
python scripts/train_surrogate.py --benchmark 10bar --n-samples 2000
```

Generates `results/surrogate_10bar.pth`, `results/surrogate_10bar_metrics.csv`, and the `lhs_dataset_10bar.npz` cache. Wall-clock: $\sim$90 s including LHS and 300-epoch PyTorch training (CPU).

D.4.5 Ch 4.5 — 200-bar benchmark

```
bash scripts/run_overnight_200bar.sh
```

A single-command resilient wrapper that launches all $3 \times 30 = 90$ runs, skips any seed that already has a pickle, retries each failure up to three times, and logs to `logs/200bar_overnight.log`. Wall-clock: $\sim$8 h on Apple M1 for the complete matrix.

D.4.6 Ch 4.6 — LLM warm-start

```
python scripts/run_llm_warmstart.py --benchmark 10bar --n-queries 30
python scripts/run_llm_warmstart.py --benchmark 25bar --n-queries 30
python scripts/run_llm_warmstart.py --benchmark 72bar --n-queries 30
```

Each command calls the Anthropic API 30 times and writes its responses + parsed area vectors to `results/llm_cache/{10,25,72}bar_*.json`. These cache files are committed to the repository so the LLM ablation in Section 4.8 can be reproduced without an API key. To force a fresh API call, delete the corresponding cache file and rerun.

D.4.7 Ch 4.14–4.16 — Day-1 post-hoc analyses

```
python scripts/run_mc_dropout_analysis.py
python scripts/run_llm_cache_reanalysis.py
python scripts/run_convergence_rate_fits.py
```

These three scripts read existing artefacts (`results/*.pkl`, `results/llm_cache/*.json`, `results/lhs_dataset_10bar.npz`) and write fresh CSV + PNG + SVG outputs. Each runs in < 60 s.

D.5 Artefact Manifest

All committed artefacts are listed in Table D.1; "size" is as-of the Phase-10 tag.

Table D.1: Artefact manifest. Figures are regenerated on demand (git-ignored); CSV summaries, LLM cache JSONs, and run pickles are committed so numerical results can be recomputed without rerunning optimisation batches.

Path	Content	Committed
`results/*.pkl`	GA/PSO/NSGA-II run pickles	no (reproduce)
`results/*.csv`	summary tables	yes
`results/llm_cache/*.json`	Anthropic API responses	yes
`results/lhs_dataset_10bar.npz`	LHS training cache	no (reproduce)
`results/surrogate_10bar.pth`	trained MLP weights	no (reproduce)
`figures/*.png, *.svg`	Ch. 4 figures (300 dpi)	no (regenerate)
`thesis_writeup/main.pdf`	compiled thesis PDF	yes

D.6 Build Commands for the Thesis PDF

The TeX source compiles with either a full MacTeX/TeX Live installation (via `latexmk -pdf main.tex`) or with Tectonic 0.16.9 [44]:

```
cd thesis_writeup
# Option A --- MacTeX / TeX Live:
latexmk -pdf -bibtex main.tex
# Option B --- Tectonic (network once, self-contained thereafter):
tectonic main.tex
```

The preamble uses `biblatex` with the `bibtex` backend (not `biber`) to maximise portability: all Tectonic versions, old MacTeX distributions, and fresh Ubuntu `texlive-full` packages converge on bibtex-compatible output. The bibliography style is IEEE (`style=ieee, sorting=none`).

D.7 Continuous-Integration Guarantees

The GitHub Actions workflow (`.github/workflows/ci.yml`) runs the non-slow part of the test suite on every push and pull request. At the time of Phase-10 tag creation, the suite comprises 81 tests: 70 fast ($< 30\,\text{s}$) and 11 marked slow (multi-minute validation runs). Only the fast 70 are required by CI; slow tests are executed locally before each release tag.

Concrete test matrix.

- `tests/test_fem_*.py`: 12 tests — truss elemental stiffness, assembly, solver, analytic 3-bar closed-form check to 1×10^{-21} precision.
- `tests/test_constraints_*.py`: 14 tests — each IS 800 clause individually verified against worked examples from Subramanian [32].
- `tests/test_benchmarks_*.py`: 15 tests — every benchmark checked for matching published reference weights (within 1.5 % where reference is verified; structural correctness only, otherwise).
- `tests/test_algorithms_*.py`: 9 tests — GA, PSO, NSGA-II convergence properties (monotone decrease, deterministic under fixed seed, pareto-front shape).
- `tests/test_surrogate_*.py`: 8 tests — MLP training reproducibility, R^2 thresholds.
- `tests/test_app_api.py`: 12 tests — FastAPI endpoint contracts (updated Phase 9 to add `/benchmarks/200bar`).
- `tests/test_llm_*.py`: 11 tests — cache I/O shape checks, LLM warm-start determinism given cache.

Every result in Chapter 4 can be traced to the deterministic output of at least one of these tests, which collectively form the integrity perimeter of the thesis.

D.8 Glossary of Non-Obvious Flags

A short catalogue of command-line flags whose meaning is not self-evident:

- `--seeds N`: runs `run_batch.py` over the first N seeds of $\mathscr{S}_{10}$. If $N > 10$ the pool cycles with LHS redraws.
- `--benchmark B`: one of `10bar,25bar,72bar,200bar`.
- `--algo A`: one of `ga,pso,nsga2,rl,llm`.
- `--n-samples K`: dataset size for `train_surrogate.py` (default 2,000).
- `--n-queries Q`: number of distinct Anthropic API calls for `run_llm_warmstart.py` (default 30).
- `--verbose`: echo every per-generation best fitness to stdout.

Everything else follows `argparse` conventions and can be inspected with `--help`. Any change to a default value is flagged with a warning so stale-default silent regressions are not possible.

Closing note. Nothing in Chapter 4 depends on any numerical constant beyond what is listed in the bullet points above; the thesis is therefore reproducible from first principles given only this appendix, the pinned `requirements.txt`, and the Git tag corresponding to the Phase-10 release.

References

[1] K. Deb, Multi-Objective Optimization using Evolutionary Algorithms. Chichester: Wiley, 2001, ISBN: 9780471873396.

[2] J. Kennedy and R. Eberhart, "Particle swarm optimization," in Proceedings of the IEEE International Conference on Neural Networks, vol. 4, 1995, pp. 1942–1948. DOI: 10.1109/ICNN.1995.488968.

[3] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," IEEE Transactions on Evolutionary Computation, vol. 6, no. 2, pp. 182–197, 2002. DOI: 10.1109/4235.996017.

[4] M. Sunar and A. D. Belegundu, "Trust region methods for structural optimization using exact second-order sensitivity," AIAA Journal, vol. 29, no. 12, pp. 2159–2165, 1991. DOI: 10.2514/3.10856.

[5] V. B. Venkayya, "Design of optimum structures," Computers & Structures, vol. 1, no. 1–2, pp. 265–309, 1971. DOI: 10.1016/0045-7949(71)90013-7.

[6] C. Fleury and L. A. Schmit, "Dual methods and approximation concepts in structural synthesis," NASA Langley Research Center, Tech. Rep. CR-3226, 1980.

[7] N. V. Queipo, R. T. Haftka, W. Shyy, T. Goel, R. Vaidyanathan, and P. K. Tucker, "Surrogate-based analysis and optimization," Progress in Aerospace Sciences, vol. 41, no. 1, pp. 1–28, 2005. DOI: 10.1016/j.paerosci.2005.02.001.

[8] A. I. J. Forrester, A. Sóbester, and A. J. Keane, Engineering Design via Surrogate Modelling: A Practical Guide. Chichester: Wiley, 2008, ISBN: 9780470060681.

[9] F. Persia et al., "Neural network surrogates for post-processing in topology optimization," Neural Computing and Applications, 2025, In press.

[10] K. Hayashi and M. Ohsaki, "Reinforcement learning and graph embedding for binary truss topology optimization under stress and displacement constraints," Frontiers in Built Environment, vol. 6, 2020. DOI: 10.3389/fbuil.2020.00059.

[11] N. K. Brown and C. T. Mueller, "Reinforcement learning for the topology optimization of continuum structures," Materials & Design, vol. 218, 2022. DOI: 10.1016/j.matdes.2022.110659.

[12] Y. Geng et al., "Integrating large language models with OpenSeesPy for structural analysis," arXiv preprint, vol. arXiv:2504.09754, 2024.

[13] L. Makatura et al., "How can large language models help humans in design and manufacturing?" MIT Exploration of Generative AI, 2024.

[14] A. Kaveh and S. Talatahari, "A novel heuristic optimization method: Charged system search," Acta Mechanica, vol. 213, no. 3–4, pp. 267–289, 2010. DOI: 10.1007/s00707-009-0270-4.

[15] J. Blank and K. Deb, "Pymoo: Multi-objective optimization in python," IEEE Access, vol. 8, pp. 89 497–89 509, 2020. DOI: 10.1109/ACCESS.2020.2990567.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proceedings of the 3rd International Conference on Learning Representations (ICLR), 2015.

[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint, vol. arXiv:1707.06347, 2017.

[18] L. A. Schmit and B. Farshi, "Some approximation concepts for structural synthesis," AIAA Journal, vol. 12, no. 5, pp. 692–699, 1974. DOI: 10.2514/3.49321.

[19] L. A. Schmit and H. Miura, "Approximation concepts for efficient structural synthesis," NASA Contractor Report, no. CR-2552, 1976.

[20] F. Erbatur, O. Hasançebi, İ. Tütüncü, and H. Kılıç, "Optimal design of planar and space structures with genetic algorithms," Computers & Structures, vol. 75, no. 2, pp. 209–224, 2000. DOI: 10.1016/S0045-7949(99)00084-X.

[21] C. V. Camp and B. J. Bichon, "Design of space trusses using ant colony optimization," Journal of Structural Engineering, vol. 130, no. 5, pp. 741–751, 2004. DOI: 10.1061/(ASCE)0733-9445(2004)130:5(741).

[22] G. Bekdaş, S. M. Nigdeli, and X.-S. Yang, "Sizing optimization of truss structures using flower pollination algorithm," Applied Soft Computing, vol. 37, pp. 322–331, 2015. DOI: 10.1016/j.asoc.2015.08.037.

[23] C. A. Coello Coello, "Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: A survey of the state of the art," Computer Methods in Applied Mechanics and Engineering, vol. 191, no. 11–12, pp. 1245–1287, 2002. DOI: 10.1016/S0045-7825(01)00323-1.

[24] Y. Sun et al., "Deep neural network surrogate plus genetic algorithm for optimization of composite structures," Biomimetics, vol. 8, no. 4, 2023.

[25] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016, ISBN: 9780262035613.

[26] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018, ISBN: 9780262039246.

[27] Y. Zhao et al., "Deep reinforcement learning for truss topology optimization with size constraints," Structural and Multidisciplinary Optimization, 2021.

[28] T. Rochefort-Beaudoin et al., "SOgym: A reinforcement learning environment for structural optimization," arXiv preprint, vol. arXiv:2407.07288, 2024.

[29] J. Wei, X. Wang, D. Schuurmans, et al., "Chain-of-thought prompting elicits reasoning in large language models," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[30] Anthropic. "Claude api documentation." (2026), [Online]. Available: https://docs.claude.com/.

[31] IS 800:2007 general construction in steel — code of practice, Bureau of Indian Standards, New Delhi, 2007.

[32] N. Subramanian, Design of Steel Structures. New Delhi: Oxford University Press, 2008, ISBN: 9780195676815.

[33] S. K. Duggal, Limit State Design of Steel Structures, 2nd ed. New Delhi: McGraw Hill Education, 2014, ISBN: 9789339203719.

[34] SP 6(1):1964 handbook for structural engineers — structural steel sections, Bureau of Indian Standards, New Delhi, 1964.

[35] D. L. Logan, A First Course in the Finite Element Method, 6th ed. Boston: Cengage Learning, 2017, ISBN: 9781305635111.

[36] R. D. Cook, D. S. Malkus, M. E. Plesha, and R. J. Witt, Concepts and Applications of Finite Element Analysis, 4th ed. New York: Wiley, 2002, ISBN: 9780471356059.

[37] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 1050–1059.

[38] K.-J. Bathe, Finite Element Procedures, 2nd. Englewood Cliffs, NJ: Prentice Hall, 2014, ISBN: 978-0979004902.

[39] W. Achtziger, "Global optimization of truss topology with discrete bar areas — Part I: Theory of relaxed problems," Computational Optimization and Applications, vol. 17, no. 1, pp. 5–40, 2000.

[40] M. P. Bendsøe and O. Sigmund, Topology Optimization: Theory, Methods, and Applications, 2nd. Berlin: Springer, 2003.

[41] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in ICLR 2019 Workshop on Representation Learning on Graphs and Manifolds, 2019.

[42] R. T. Haftka and Z. Gürdal, Elements of Structural Optimization, 3rd. Dordrecht: Kluwer Academic, 1992, ISBN: 978-0792315049.

[43] A. Kaveh and V. Kalatjari, "Size/geometry optimization of trusses by the force method and genetic algorithm," Zeitschrift für Angewandte Mathematik und Mechanik (ZAMM), vol. 84, no. 5, pp. 347–357, 2011.

[44] P. K. G. Williams and Tectonic Contributors, Tectonic: A modernised, complete, self-contained TEX/LATEX engine, `https://tectonic-typesetting.github.io/`, 2024.

[45] J. S. Arora, Introduction to Optimum Design, 4th. New York: Academic Press, 2016, ISBN: 9780128008065.

[46] S. S. Rao, "Engineering optimization: Theory and practice," John Wiley and Sons, 2009.

[47] D. E. Goldberg and J. H. Holland, "Genetic algorithms and machine learning," Machine Learning, vol. 3, no. 2, pp. 95–99, 1988.

[48] Z. Michalewicz, Genetic Algorithms + Data Structures = Evolution Programs, 3rd. Berlin Heidelberg: Springer, 1996, ISBN: 9783540606765.

[49] K. Deb and R. B. Agrawal, "Simulated binary crossover for continuous search space," Complex Systems, vol. 9, no. 2, pp. 115–148, 1995.

[50] Y. Shi and R. Eberhart, "A modified particle swarm optimizer," IEEE ICEC, pp. 69–73, 1998.

[51] M. Clerc and J. Kennedy, "The particle swarm — explosion, stability, and convergence in a multidimensional complex space," IEEE Transactions on Evolutionary Computation, vol. 6, no. 1, pp. 58–73, 2002.

[52] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach," IEEE Transactions on Evolutionary Computation, vol. 3, no. 4, pp. 257–271, 1999.

[53] C. A. Coello Coello, "Recent trends in evolutionary multiobjective optimization," ACM Computing Surveys, vol. 32, no. 2, pp. 109–143, 2000.

[54] J. Knowles, "ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems," IEEE Transactions on Evolutionary Computation, vol. 10, no. 1, pp. 50–66, 2006.

[55] A. Sóbester, S. J. Leary, and A. J. Keane, "On the design of optimization strategies based on global response surface approximation models," Journal of Global Optimization, vol. 33, no. 1, pp. 31–59, 2005.

[56] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," Journal of Global Optimization, vol. 13, no. 4, pp. 455–492, 1998.

[57] C. E. Rasmussen and C. K. I. Williams, Gaussian Processes for Machine Learning. Cambridge, MA: MIT Press, 2006, ISBN: 9780262182539.

[58] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," Neural Networks, vol. 2, no. 5, pp. 359–366, 1989.

[59] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[60] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," Advances in Neural Information Processing Systems, vol. 32, 2019.

[61] M. Stein, "Large sample properties of simulations using Latin hypercube sampling," Technometrics, vol. 29, no. 2, pp. 143–151, 1987.

[62] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," Technometrics, vol. 21, no. 2, pp. 239–245, 1979.

[63] B. Schölkopf and A. J. Smola, Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond. Cambridge, MA: MIT Press, 2001, ISBN: 9780262194754.

[64] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015.

[65] D. Silver et al., "Mastering the game of Go without human knowledge," Nature, vol. 550, no. 7676, pp. 354–359, 2017.

[66] A. Raffin et al., "Stable-Baselines3: Reliable reinforcement learning implementations," Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.

[67] G. Brockman et al., "OpenAI Gym," arXiv preprint arXiv:1606.01540, 2016.

[68] A. Vaswani et al., "Attention is all you need," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[69] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," NAACL-HLT 2019, 2019.

[70] T. B. Brown et al., "Language models are few-shot learners," Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020.

[71] L. Ouyang et al., "Training language models to follow instructions with human feedback," Advances in Neural Information Processing Systems, vol. 35, 2022.

[72] J. Hoffmann et al., "Training compute-optimal large language models," arXiv preprint arXiv:2203.15556, 2022.

[73] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," arXiv preprint arXiv:2303.12712, 2023.

[74] J. Wei et al., "Emergent abilities of large language models," Transactions on Machine Learning Research, 2022.

[75] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[76] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive nlp tasks," Advances in Neural Information Processing Systems, vol. 33, 2020.

[77] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," arXiv preprint arXiv:2303.11366, 2023.

[78] J. S. Park et al., "Generative agents: Interactive simulacra of human behavior," UIST 2023, 2023.

[79] E. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," ICLR 2023, 2023.

[80] S. Yao et al., "Tree of thoughts: Deliberate problem solving with large language models," in NeurIPS 2023, 2023.

[81] S. S. Rao, "Optimization of structures under shock and vibration environment," The Shock and Vibration Digest, vol. 17, no. 10, pp. 21–33, 1985.

[82] P. Dumonteil, "Simple equations for effective length factors," Engineering Journal (AISC), vol. 29, no. 3, pp. 111–115, 1992.

[83] J. Perry, "On the strength of columns," Proceedings of the Institution of Civil Engineers, vol. 97, pp. 1–6, 1886.

[84] S. P. Timoshenko and J. M. Gere, "Theory of elastic stability," McGraw-Hill, 1961.

[85] O. C. Zienkiewicz, R. L. Taylor, and J. Z. Zhu, The Finite Element Method: Its Basis and Fundamentals, 7th. Oxford: Butterworth-Heinemann, 2013, ISBN: 9781856176330.

[86] M. P. Saka, K. B. Geleş, and A. E. Kayabekir, "Metaheuristic algorithms in optimum design of steel truss structures," Archives of Computational Methods in Engineering, vol. 23, no. 3, pp. 337–366, 2016.

[87] K. S. Lee and Z. W. Geem, "A new meta-heuristic algorithm for continuous engineering optimization: Harmony search theory and practice," Computer Methods in Applied Mechanics and Engineering, vol. 194, no. 36–38, pp. 3902–3933, 2005.

[88] A. Kaveh and M. Ilchi Ghazaan, "Enhanced colliding bodies optimization for design problems with continuous and discrete variables," Advances in Engineering Software, vol. 77, pp. 66–75, 2014.

[89] O. Hasançebi, S. Çarbaş, E. Doğan, F. Erdal, and M. P. Saka, "Performance evaluation of metaheuristic search techniques in the optimum design of real size pin jointed structures," Computers and Structures, vol. 87, no. 5–6, pp. 284–302, 2009.